Noroff University College

Bachelor in
Cyber Security

Cybersecurity Challenges in Web Applications Deployed on Azure Cloud

KAMAL KIRTI SHARMA

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR IN **CYBER SECURITY**

SUPERVISOR

Noroff University College, Norway
May, 2024

Mandatory Declaration

Declarations

The individual student is responsible for familiarising themselves with the rules and regulations regarding the use of sources, generated text and academic misconduct. Failure to declare does not release the student from their responsibility.

1.	I hereby declare that the submission answer is my own work, and that I have not used other sources other than as is referenced and cited correctly, or received help other than what is specifically acknowledged.	Yes
2.	**I further declare that this submission:** • Has not been used for another exam in another course at Noroff University College, at another department/university/college at home or abroad. • Does not refer to or make use of the work of others without acknowledgement. • Does not refer to my own previous work unless stated. • Has all the references given in the bibliography. • Is not a copy, duplicate or copy of someone else's work or answer. • Is not generated using AI generation tools.	Yes
3.	I am aware that a breach of any of the above is to be regarded as cheating and may result in cancellation of the exam and exclusion from universities and colleges in Norway, cf. University and College Act §§4-7 and 4-8 and Regulations on examinations §§ 31.	Yes
4.	I am aware that all components of this assignments may be checked for plagiarism and other forms of academic misconduct.	Yes
5.	I hereby acknowledge that I have been taught the appropriate ways to use the work of other researchers. I undertake to paraphrase, cite, and reference according to the acceptable academic practices, in accordance with the rules and guidelines, as taught.	Yes
6.	I am aware that Noroff University College will process all cases where cheating is suspected in accordance with the college's guidelines.	Yes

Publication Agreement

Authorisation for electronic publication of the thesis: Through submission you are accepting that Noroff University College has a perpetual, and royalty free right to retain a copy of work for its own internal use, and has the right to make work publicly available - considering any restrictions to publication.

Acknowledgements

I would like to express my sincere thanks to my supervisor Professor Barry Irwin for being a fantastic mentor throughout this research work. The insights and feedback provided by him were crucial in helping me develop my research skills and refine my thoughts. I am sincerely grateful for his patience, encouragement, and willingness to share his knowledge in the research domain.

Abstract

Securing cloud computing demands a thorough grasp of its structural components and associated threats. The primary step in fortifying any technology's security is identifying its potential threats. This research endeavors to uncover common cybersecurity challenges faced by web applications deployed on the Azure cloud platform and provide mitigation strategies to reduce these vulnerabilities. The widespread use of cloud computing has transformed the distribution of applications while also posing security risks. Although Microsoft Azure has strong security capabilities, managing web apps becomes more complex due to its flexibility. Because of this complexity and the always changing threat landscape, web application security in the Azure cloud environment requires a comprehensive solution. This research analyzes these issues, evaluates existing security testing techniquesand suggests a practical mitigation stratergies in utilizing Peffer's approach as methodology. It assesses the effectiveness of vulnerability scanning, penetration testing, and security best practices like access controls and data encryption in addressing identified vulnerabilities, subsequently analyzing their impact on the web application's overall security posture. Through the implementationa nd testing it illuminates the cybersecurity landscape of web applications, particularly hosted on Azure and bridges the theory and practical aspects. By using Damn Vulnerable Web Application (DVWA) to demonstrate the security principles and web application vulnerabilities it offers a practical approach to secure web application on Azure. The research highlights the importance of Role-Based Access Control (RBAC) and examines how to mitigate such vulnerabilities proactively by testing Web Application Firewalls (WAFs) and Network Security Groups (NSGs). The research assesses how vulnerability scanning, penetration testing, data encryption, and access controls, together with other security best practices, resolve vulnerabilities found and affect the overall security posture of the online application. This research intends to help organizations hosting web applications on Microsoft Azure improve their cloud security posture by offering insightful analyses of Azure security vulnerabilities and workable remedies.

Contents

List of Figures

CHAPTER 1

Introduction

1.1 Introduction

From the time when the internet became popular in the 1990s to today's widespread use of computers everywhere, the internet has made significant changes in how computers work. This evolution started with something called parallel computing and then moved on to distributed computing, grid computing, and now cloud computing. Even though people have known about cloud computing for a while, cloud computing is still a new and growing area of computer science (Jadeja & Modi, 2012). As stated by IBM, a prominent company in the IT sector, cloud computing offers numerous advantages to organizations, including increased flexibility, enhanced efficiency, and strategic value (IBM, 2023). The introduction of cloud computing has transformed the deployments of applications and it is a significant shift in how things work in the IT world. However, the advantages come at a cost as these benefits of cloud computing can lead to serious security issues such as data breaches, computation breaches, and flooding attacks (Takabi & Ghasemigol, 2019). One of the recent researches done by Trendmicro a global cybersecurity leader, identified sensitive environmental variables in the Microsoft Azure environment that, if exposed, could potentially be exploited by malicious actors to compromise the entire serverless environment (Fiser & Olivera, 2023).

The project explores the issues of securing web application deployments in the Azure cloud environment using the practices of cybersecurity. This project seeks to provide a robust and integrated guide to the cybersecurity framework that not only facilitates the deployment of web applications but also strengthens defense against various cyber threats. The project will explore cloud-based web application deployment; cybersecurity best practices and various cloud-native tools used for security and follow a journey to bridge the gap between innovation and security. After the completion of the research work, the study

will yield not only a more profound comprehension of the cybersecurity challenges specific to Azure-hosted web applications but also the development of a practical guide capable of aiding organizations in enhancing the security of their cloud-based web applications effectively.

In the next section, the exploration of background information on the problem domain will be done. This will make it easier for readers to understand the cybersecurity challenges in web applications deployed in the Azure cloud environment.

1.2 Background and Related Work

In this section, the research work explores the foundational aspects of cybersecurity challenges within web applications hosted on cloud platforms. Additionally, it reviews related studies that have investigated similar challenges in the realm of cybersecurity for web applications hosted on cloud platforms.

1.2.1 Background

In the era of the fast-growing digital landscape, organizations and IT sector businesses are constantly under pressure to adapt and innovate to stay in the business competition. Every business aspires to attract more clients and get new business opportunities for growth which in return has generated an increased demand for additional IT infrastructure and services, like data storage and implementation of various essential processes. The emergence of cloud computing can be attributed to the increasing demand for IT infrastructure. Cloud computing has seen remarkable growth and adoption, attributed in part to its roots in the maturation of Grid Computing and the advent of Web 2.0 technologies. Chang et al. (2016) analyzed these developments have simplified intricate processes while maintaining exceptional performance and delivering user-friendly web interfaces.

Cloud Computing and Azure Cloud Services

Cloud computing has a fascinating history dating back to the 1960s when John McCarthy envisioned a future where computation would function as a public utility. He famously stated, Computing may one day be organized as a public utility (Jadeja & Modi, 2012).

Historically, computers occupied entire rooms with bulky electronics, consuming significant power while offering limited processing power. Today, we benefit from compact hard drives and cost-effective network devices, enabling powerful distributed systems for tasks like scientific simulations. These systems, known as clusters and grids, have become integral in distributed computing (Stanoevska-Slabeva & Wozniak, 2010). The clusters and grid are different in approaches as the cluster model supports a homogeneous network and the grid system supports a large and heterogeneous network. Cloud

computing, which emerged in 2008, aimed to realize computing as a utility, a concept introduced by Corbato and Vyssotsky in 1965. It offers on-demand access to computing resources, allowing customers to select the resources they need, similar to how the public accesses services like water and electricity (Singh & Chatterjee, 2017). In formal terms, cloud computing refers to the practice of storing and accessing data and software applications via the Internet, rather than relying solely on local computer storage (Rashid & Chaturvedi, 2019). According toMell and Grance (n.d.), it's described as a model that enables ubiquitous, convenient, on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services. These resources can be rapidly provisioned and released with minimal management effort or service-provider interaction.

What makes cloud computing particularly attractive is its ability to deliver resources as needed, allowing for flexibility in scaling resources up or down based on user demand, making it cost-effective for businesses of all sizes (Sharma & Trivedi, 2014). The "pay-as-you-go" model is especially advantageous, as clients only pay for the services they utilize (Diaby & Rad, 2017). While Cloud Computing offers numerous advantages, its adoption is hindered by notable barriers. Foremost among these is security, with compliance, privacy, and legal concerns also posing significant challenges.

However, cloud computing faces a significant challenge in terms of security due to its data storage and resource-sharing capabilities. This security concern can deter users, especially when it comes to personal data storage in the cloud (Kaur & Singh, 2015).

Challenges in Cloud Computing

The world has evolved a lot in the last few decades, this evolution process caused the exponential rise of IT systems. Organizations' demand for faster and scalable access to computing powers is being fulfilled by cloud computing. Cloud computing brings many benefits like the latest technologies, unlimited data storage, compute power, reduced prices, and savings in IT asset capital expenditure. Cloud solves many problems but has many challenges especially related to security (Singh & Chatterjee, 2017). Cloud computing is entirely dependent on the internet so a lot of data saved in the cloud flows through the network and there are a lot of attackers who continuously try to occupy and exploit the private data (Tabrizchi & Kuchaki Rafsanjani, 2020). Some common challenges in cloud computing are data breaches, data loss, DDoS attacks, compromised credentials, access control, limited knowledge causing high cost, and inadequate diligence (Subramanian & Jeyaraj, 2018). The cost-effectiveness of a cloud environment is advantageous not only for users but also for potential intruders. The availability of affordable computing resources can simplify penetration testing for attackers, enabling them to exploit vulnerabilities in cloud customers' systems (Zhang et al., 2014). The absence of interoperability can be a source of security concerns when it comes to APIs. Interoperability, in this context, refers to the capability of different components or platforms within a system to communicate smoothly using an API. Several scenarios highlight the importance of

this capability, including migrating from one Cloud Service Provider (CSP) to another or upgrading services on either the CSP or customer side (Machado et al., 2009). As organizations continue to navigate the complexities of cloud computing, a proactive approach to security and a commitment to staying updated on evolving threats will be essential in overcoming these challenges and harnessing the full potential of the cloud.

Cloud Computing Security Threats

Cloud computing threats may challenge three key components of security i.e. Integrity, Confidentiality, and Availability (Belal & Sundaram, 2022).

Cloud vulnerabilities that can threaten confidentiality can be classified into 3 categories. First, is when external attacker uses remote tools resulting data exposure or unauthorised access of cloud applications for malicious intentions (Varun et al., 2014). Second is when internal attacker probably an employee uses an authorized system to harm cloud applications, exposes data or get anathorised access for malicious purpose (Alsolami, 2018). Third is when misconfigurations, human error, lack of security components, secure access failures or bad architecture exposes the data of application (Butt et al., 2020).

Data Integrity threats are connected to risk of information separation, issues with access control, preserve data from modification and authentication challenges. The risks associated with information separation can impact the data's integrity as changes occur within the cloud environment, affecting assets connected to clients (Kazim & Zhu, 2015). In the analysed view from Nadeem (2016) Issues related to authentication and access control emerge when cloud access policies are not followed or when security measures related to usernames, passwords or biometric authentication methods are not followed.

Availability threats can affect the availability of applications due to inaccessibility caused by DNS servers, software issues, applications, or any components in cloud. Availability can also be affected by network providers, IT administrators or network hardware failures (Nicho & Hendy, 2013).

1.2.2 Related Work

Security is a widely discussed topic in both industry and research domains. Due to cybersecurity challenges, risks, and threats in web applications deployed on Azure cloud, numerous systematic research efforts have been conducted to explore challenges, vulnerabilities, and threats in cloud computing.

The following research articles and papers have explored pertinent security issues and challenges within the realm of cloud computing.

1. In the analysis done by S. N. Kumar and Vajpayee (2016) on the issues of data security and privacy issues during communications on the clouds, identified the necessity of implementing data encryption and message signing as essential measures to ensure confidentiality and integrity. The paper also found that optimal security services

can gained by applying both encryption and authentication on data processing over the cloud.

2. Tabrizchi and Kuchaki Rafsanjani (2020) did an analysis survey to analyze and examine various aspects of cloud computing, highlighting security and privacy issues. The authors introduce a novel classification of recent security solutions and discuss emerging security threats. The paper offers an in-depth exploration of security challenges affecting cloud entities, including cloud service providers, data owners, and users, while also addressing open issues and proposing future directions.

3. In the 2022 Vellela et al. (2022) conducted a brief analysis and presented a strategic review of confidentiality and privacy approaches within the context of cloud computing. The study focuses on the significant technological advancement of Cloud Computing (CC), which enables data access from anywhere at any time. Cloud computing processes have rapidly evolved, with large organizations adopting cloud infrastructure for efficient data storage and flexible access. Privacy methods are employed in this cloud computing environment to address security concerns. Various privacy-preserving techniques are introduced to enhance data integrity in cloud storage and ensure client data security. Consequently, the research suggests that privacy-preserving techniques contribute to improved security.

1.3 Problem Statement

Web apps on Microsoft Azure face intricate cybersecurity challenges due to its flexible nature, leading to heightened risks as infrastructures expand. Despite Azure's security features, users often struggle to effectively utilize tools against evolving threats arising from web app vulnerabilities and the dynamic cybersecurity landscape (Caraballo, 2024). An exploration and analysis, along with the identification of potential countermeasure solutions, are necessary for this domain to address specific challenges effectively.

1.4 Research Objectives

The primary objectives of this research work are:

1. To identify common cybersecurity challenges specific to web applications deployed on Azure cloud.

2. To assess various security testing methodologies, including vulnerability scanning and penetration testing, to determine their effectiveness in identifying potential vulnerabilities in the Azure-hosted web application.

3. To explore and evaluate cybersecurity mitigation strategies and measures, including configuration changes, access controls, data encryption, network security, and monitoring, that can be applied to address vulnerabilities identified in the Azure-hosted web application.

4. To analyze the impact of implementing security measures on the overall security posture of the web application in the Azure cloud. Examine how these measures contribute to the reduction of cybersecurity risks.

1.5 Research Methodology

This section provides an overview of the research methodology employed in the research project. It outlines the systematic approaches and tools utilized to collect data, analyze and interpret data. In this section, the reader will gain insights into the research design, data collection methods and analysis techniques that underpin the research process. This comprehensive overview enables a clear understanding of how the study will be conducted and the derivation of the results.

1.5.1 Introduction

This research work aims to explore and analyse the cybersecurity challenges faced by the web applications hosted on the Azure cloud platform. The research work will follow a Design Science Research methodology to find out the challenges in the security of web applications deployed on Azure Cloud.

The research will be carried out in phases. First and foremost, the emphasis is on identifying and analyzing cybersecurity risks that arise in web applications hosted on Azure. This is followed by a thorough literature analysis to integrate current information, laying the groundwork for a strong theoretical foundation. Then, real-world case studies will be examined to provide insight into unique difficulties, effective techniques, and potential solutions. Practical cybersecurity solutions will be suggested based on the findings of the literature research and case studies, including countermeasures and complete guidance suited to Azure-hosted web apps. These solutions will be applied to a deployed web application on Azure, undergoing rigorous monitoring, analysis, and assessment, including vulnerability scanning, penetration testing, and manual configuration testing. The efficiency and performance of these precautions will subsequently be explored using the deployed web application as a testing ground. Finally, the research will go through a reflection and revision phase, when it will analyze results, seek input, and fine-tune potential solutions based on real-world observations and insights acquired throughout the process.

1.6 Summary

In summary, the research work explores the growing importance of cloud computing, particularly with Microsoft Azure's rise. It examines how cloud computing transforms application deployment but also introduces new security concerns. Highlighting recent research, the summary emphasizes the need to address vulnerabilities specific to Azure web apps. It then outlines a comprehensive research project to find and mitigate these risks. This

project dives into identifying, analyzing, and resolving Azure security challenges. Finally, it provides a roadmap for the following sections, which will detail the project's background, related research, problem statement, goals, and methodology.

CHAPTER 2

Literature Review and Theoretical Background

2.1 Introduction

The study project explores the fundamentals of cybersecurity issues in cloud-hosted web applications in this chapter. Furthermore, it examines recent research that has looked into comparable cybersecurity issues for web apps housed on cloud platforms. It also begins with a summary of the research project and explains cloud computing, with a focus on Azure Cloud, along with a detailed breakdown of security aspects from the literature. It then provides a summary of the security issues with web apps hosted on Azure Cloud.

In the initial part of this research, Section 1.1 (Chapter 1) the research project undertakes a thorough exploration of fundamental concepts in cloud computing. Following this, Section 1.2 (Chapter 1), a detailed comparison between various models of cloud computing is outlined, providing an overview of the basic principles that influence the modern IT landscape. The study aims to highlight specific cybersecurity considerations by closely examining these diverse cloud models, Section 1.3 (Chapter 1), aiming to establish a framework for understanding the challenges associated with deploying web applications on the Azure cloud platform.

Further, the literature review proceeds to dive deep into Azure Cloud from Section 1.4 to Section 1.5 (Chapter1) by outlining an in-depth examination of Azure's architecture, services, and features. Contributing in the next sections, the literature review explores some difficult aspects associated with web application security, as well as common threats, vulnerabilities, and attack vectors. These include traditional security issues and those that are distinct to the cloud environment thus providing a background for a detailed exploration of problems facing web applications deployed on Azure cloud.

2.2 Related Work

Security is a widely discussed topic in both industry and research domains. Due to cybersecurity challenges, risks, and threats in web applications deployed on Azure cloud, numerous systematic research efforts have been conducted to explore challenges, vulnerabilities, and threats in cloud computing. Notable international conferences such as the European Conference, ACM Workshop on Cloud Computing Security, and the International Conference on Cloud Security Management exclusively focus on cloud security. Additionally, numerous international journals specialize in cloud security.

The following research articles and papers have explored pertinent security issues and challenges within the realm of cloud computing.

1. In the analysis done by S. N. Kumar and Vajpayee (2016) on the issues of data security and privacy issues during communications on the clouds, identified the necessity of implementing data encryption and message signing as essential measures to ensure confidentiality and integrity. The paper also found that optimal security services can gained by applying both encryption and authentication on data processing over the cloud.

2. Tabrizchi and Kuchaki Rafsanjani (2020) did an analysis survey to analyze and examine various aspects of cloud computing, highlighting security and privacy issues. The authors introduce a novel classification of recent security solutions and discuss emerging security threats. The paper offers an in-depth exploration of security challenges affecting cloud entities, including cloud service providers, data owners, and users, while also addressing open issues and proposing future directions.

3. In the 2022 Vellela et al. (2022) conducted a brief analysis and presented a strategic review of confidentiality and privacy approaches within the context of cloud computing. The study focuses on the significant technological advancement of Cloud Computing (CC), which enables data access from anywhere at any time. Cloud computing processes have rapidly evolved, with large organizations adopting cloud infrastructure for efficient data storage and flexible access. Privacy methods are employed in this cloud computing environment to address security concerns. Various privacy-preserving techniques are introduced to enhance data integrity in cloud storage and ensure client data security. Consequently, the research suggests that privacy-preserving techniques contribute to improved security.

4. After analyzing almost 41 studies that were published between 1994 to 2014, Rafique et al. (2015) proposed solutions for web application security. In this research work, the researchers did a systematic mapping study on the web application vulnerabilities and proposed some solutions. These solutions were based on two main components:

   (a) Software Development Stages: In this, the solutions suggested were aligned with different phases of software development where they were designed to be

implemented and helped the researchers to find out the most effective approach.

(b) The solutions were also categorized based on common web application vulnerabilities that are framed by OWASP Top 10 that also provide countermeasures for handling the most prevailing security issues.

This study also found many techniques, analysis, and hybrid approaches for the solutions and also revealed the gaps and opportunities for future work to improve the scalability and real-time threat detection.

5. A study research done by S. Ahmad et al. (2022) provides a view on how to secure cloud data effectively and also discusses the various challenges and constraints in the cloud environment. This article explored a critical aspect of data security in the cloud and mainly focuses on the challenges of cloud service providers and clients or organizations. The solutions suggested were mainly on the data encryption algorithms.

2.3 Introduction to Cloud Computing

Cloud Computing system is an emerging Infrastructure which is based on the internet and serves services virtually. Cloud computing provides cost-efficient access to a large selection of information technology and computing resources according to the demand. These services and resources include operating systems, storage services, network infrastructure, hardware equipment, and complete software applications (Milani & Navimipour, 2016). In simpler terms, cloud computing is storing, saving, and retrieving data on the internet rather than one's local machine or local hard drive (Rashid & Chaturvedi, 2019).

Cloud computing has emerged as a realm in the world of information technology with the business world for providing multi-way customized resources. The on-demand gain of network and infrastructure entry to a shared or unshared reservoir of scalable IT resources and services makes cloud computing the most popular in the IT industry. Organizations including the public sector depend on cloud system services for even daily tasks, like document formation, data storage, business management online games communication, and collaboration. An individual's daily use of cloud storage involves storing personal photos and documents on services like Google Drive or Dropbox, ensuring easy access from any internet-connected device. Cloud has created the foundation and groundwork for major technological advances like mobile computing, the Internet of Things (IoT), Big data, and Artificial Intelligence (AI) which as a result speeds up the changes and development in industries and Business models and creates a strong digitalization (Sunyaev, 2020).

2.3.1 Definition and fundamental concepts of cloud computing

Cloud computing has drastically changed and revolutionized the way businesses and individuals handle data storage, access, and processing. It has become a part of our lives

allowing us to store and retrieve information from anywhere anytime. 'Cloud' in the context of cloud computing refers to the service provider that provides services via the internet, and 'Computing' refers to the computation or processing of power and resources that computers give (Surbiryala & Rong, 2019). As defined by NIST, it is described as a model that enables ubiquitous, convenient, on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services. The resources here can be properly deployed fast with no time and almost no management efforts are required for the services. Technically cloud computing is the practice of using the Internet to store and retrieve software applications, technologies, and data instead of depending on a local computer storage (Rashid & Chaturvedi, 2019).

Essentially, cloud computing is a system for managing computing resources, involving the technique of massively sharing and pooling hardware infrastructure resources. Different demands share limited hardware resources, providing each user with the appearance of sole authority over the actual environment and removing the need for the user to understand the physical resource configuration. Because it functions as a production environment, and a worldwide marketplace all at once, cloud computing is innovative (Kushida et al., 2015). Gartner Gong et al. (2010) has defined cloud computing as, 'a way and style of computing where huge scalable IT-enabled capabilities are delivered 'as a service' to external customers using Internet technologies.'

2.3.2 Historical evolution of cloud computing

In general, it is believed that cloud computing is a new concept, however, it dates back to the middle of the 1940s. In the 1930s, after Alan Turing's pioneering work on the first computers, big and costly general-purpose machines like the ENIAC were developed. The computing of this type used huge shared systems that could perform calculations by manipulating components such as vacuum tubes and relays (Maayan, 2021).

In the stages of computing, organizations relied on mainframe computers for their data processing and storage needs. This concept laid a foundation for cloud computing by introducing the concept of resource sharing, among users. In the 1960s John McCarthy founded and coined the cloud computing concept that in the future computing would exist in a public-oriented structure and utility (Surbiryala & Rong, 2019).

In 1963, MIT received 2 million USD from the Defense Advanced Research Projects Agency (DARPA) DARPA for Project MAC, which aimed to develop technology for multiple users on a computer and provided the foundation for Cloud computing. In 1969, psychologist and computer scientist JCR Licklider played a key role in ARPANET's development, envisioning a global interconnected computer network (Foote, 2021).

In the 1970s virtualization made progress alongside ARPANET. It involved creating versions of computing resources, like hardware, storage, and operating systems. This breakthrough allowed multiple users to share resources while keeping their environments separate which became an aspect of cloud computing. As the internet expanded organiza-

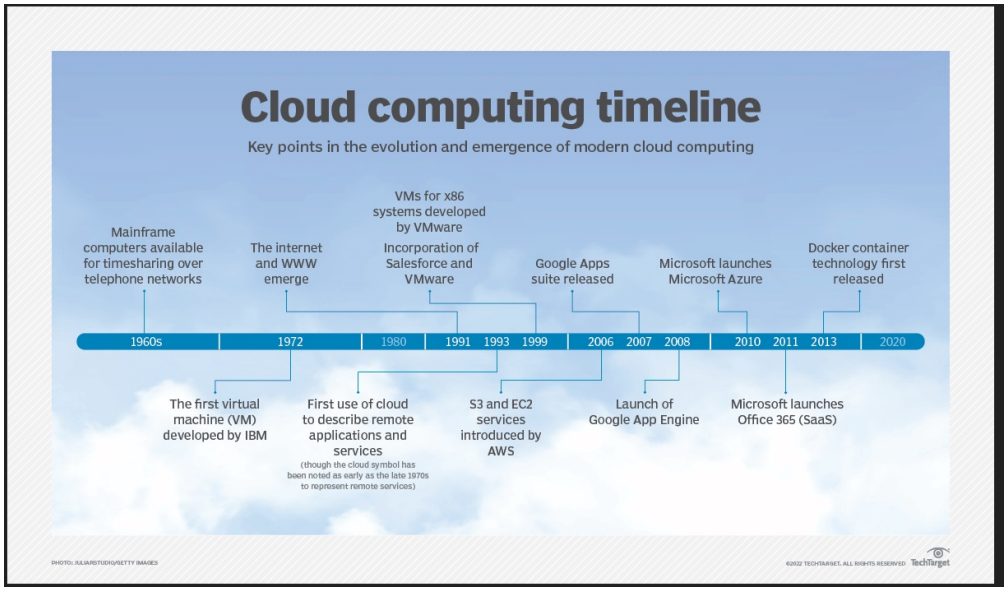

Figure 2.1: History of Cloud Computing taken from (Radoff, 2023)

tions and businesses started offering and providing private networks in tandem with the evolution of Virtualization kept growing from the 1970s to the 1990s, with important tech improvements leading to real cloud computing. IBM played a big part in 1972 by releasing the VM operating system. In the 1990s, telecom companies added to this progress with their versions of virtual private networks (VPNs). All these changes shaped virtualization and set the stage for today's advanced cloud computing systems (Ipacs, 2023). In 1989, the World Wide Web was born, and it dramatically increased data sharing and access resulting in a high demand for Internet services and infrastructure. This information boom provided huge scalability, accessibility, and connectivity (Ipacs, 2023).

In 1999, Salesforce started offering apps on the internet, making Software as a Service (SaaS) a thing. Three years later, lots of videos, music, and other stuff started being put and sent out worldwide. In 2006, Amazon brought in the initial corporate cloud called Elastic Computer Cloud (EC2). Soon after, in 2008, Google did the same, giving out free basic plans and cheap services for computing and storage with its Google App, and in 2010, Microsoft Azure stepped into the cloud market and continued supporting mobile and web apps to grow efficiently (Davies, 2021).

From the 2010s and further cloud computing has revolutionized to new from the primary concept of business and individuals, this competition between big companies is to develop and provide an extensive range of cloud services, virtually every industry (Ipacs, 2023). Figure 2.1 shows the timeline of the emergence of Cloud Services.

2.3.3 Cloud Computing's Significance in Modern IT

In today's IT environment, cloud computing is crucial since it usually results in positive outcomes for both the service provider and the customer. Cloud computing provides scalability, manageability, and reliability to businesses and has changed the literal meaning of the businesses by providing on-demand operations, universality, economic benefit

and availability of the IT infrastructure to both the provider and the consumer (Alhomdy et al., 2021).

I. Ahmad (2017) analyzed that, customers can always access cloud-based services according to their needs and in a customized way via the Internet from any location (ubiquitous) across a variety of devices, including computers, personal digital assistants (PDAs), and smartphones, which makes cloud computing as a Broad network Access and provides the feature of mobility. Cloud services like storage, server time, computing power, and networking can be provided to the consumers according to their requirements without any human interaction thus reducing the manpower and physical storage (Uzoma & Okhuoya, 2022). One of the characteristics of cloud computing is resource-pooling in which physical computing resources like CPU, RAM, storage, and virtual computing resources for example virtual machines, hypervisor are pooled and merged into the cloud as these resources are not location-dependent and the client doesn't need to have control or awareness of the position of these resources.

Cloud computing provides consumers the benefit of pay-as-you-go service in which the client similarly uses them as other utilities are used and billed by the usage like water, fuel, and electricity making cloud computing a measured service (Rashid & Chaturvedi, 2019). What makes cloud computing services particularly attractive is the ability to deliver resources as needed, allowing for flexibility in scaling resources up or down based on user demand, making it cost-effective for businesses of all sizes (Sharma & Trivedi, 2014). Although cloud computing has much to develop in the future, the three main core fundamental technical aspects, virtualization, multitenancy, and web services have been taking shape quickly and smoothly (Marston et al., 2011).

The "pay-as-you-go" model is especially advantageous, as clients only pay for the services they utilize (Diaby & Rad, 2017). While Cloud Computing offers numerous advantages, its adoption is hindered by notable barriers. Foremost among these is security, with compliance, privacy, and legal concerns also posing significant challenges. However, cloud computing faces a significant challenge in terms of security due to its data storage and resource-sharing capabilities. This security concern can discourage the users, especially when it is the case to store personal data in the cloud (Kaur & Singh, 2015).

2.4 Types of Cloud Services

In cloud computing, there are various kinds of cloud services and These models specify what kind of accountability and control users have over their computer services. Mainly three main types of service models are IaaS, PaaS, and SaaS. The figure 2.2 shows the cloud architecture of the cloud system in layers of the service models.

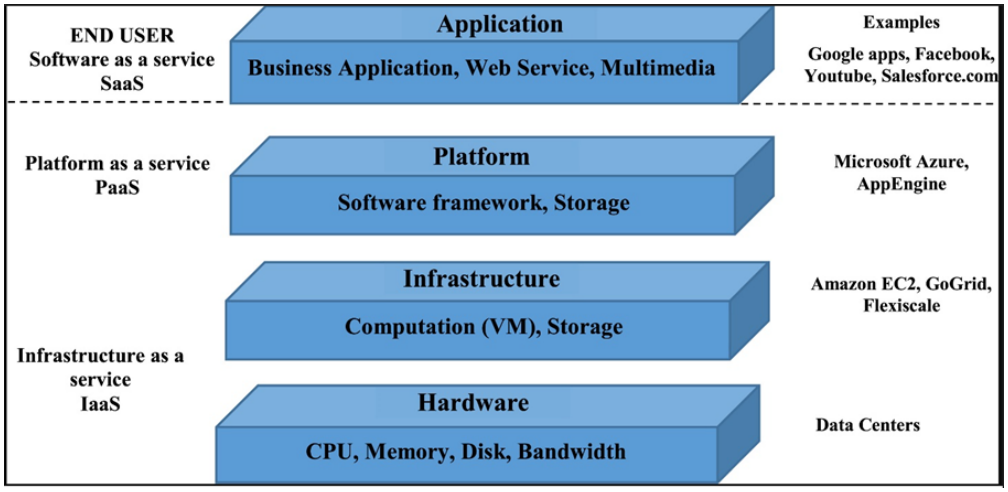

Figure 2.2: Layered Architecture of cloud system adapted from (Abdullayeva, 2023)

2.4.1 Infrastructure as a Service (IaaS)

Rashid and Chaturvedi (2019) discuss the basic cloud computing models and mainly focus on Infrastructure as a Service (IaaS) with an explanation that in IaaS, the cloud service providers give and offer virtual computing resources and services like CPU, memory, Operating system, and different applications. Infrastructure as a service (IaaS) is a uniform, highly efficient service in which a company outsources mainly IT infrastructure like servers, networking components, storage, hardware, and other equipment needed to support operations, and then provides the service to customers on a demand basis. Customers can use a web-based interface to get oversight over this infrastructure (Almubaddel & Elmogy, 2016).

According to Mohammed and Zeebaree (2021) The user's ability to provide essential storage, networking, and other services enables them to install and have an overview of virtual machines, which includes operating systems and applications. With Infrastructure-as-a-Service (IaaS), users can manage operating systems, storage, and apps, but they may not have as much control over networking elements like firewalls (Miyachi, 2018). BigCommerce (2021) defines IaaS, or infrastructure-as-a-service, as a service that provides cloud-based alternatives to on-premise physical infrastructure, allowing companies to obtain resources as needed instead of incurring the greater expenses associated with having to buy and maintain hardware.' IaaS has greater flexibility and scalability than traditional on-premise-based solutions. IaaS service-providing companies generally provide services based on consumption, that is pay-as-you-go for storage, virtualization, and networking.

The benefits of Infrastructure as a Service as outlined by Rashid and Chaturvedi (2019) are:

1. Cost Reduction: IaaS reduces capital expenses by reducing the need for large initial investments.

2. Pay-as-You-Go: It provides an efficient payment model in which users pay only for

the services they want to use that are also cost-effective.

3. Access to Quality Resources: Users gain access to enterprise-grade IT resources, ensuring the utilization of the best infrastructure.
4. Flexibility: IaaS provides flexibility and adaptability by allowing consumers to adjust their computing resources in accordance with their needs at any given moment.

2.4.2 Platform as a Service (PaaS)

Marinos and Briscoe (2009) define Platform as a Service (PaaS) functions as a higher level of abstraction utilizing Google-App Engine as a technical thing for the developers. Here in this enhanced framework and service, the developers are free from the complexities of technical details and machine instances, allowing them to focus on their programming tasks. The programs they create are run within data centers, which reduces the issues of resource allocation. Through PaaS, customers have a platform for developing software applications, and, Microsoft Azure and Google are examples of PaaS (Surbiryala & Rong, 2019). In PaaS, the user does not handle the operating systems and the network control access but has some control over the applications deployed and to some extent the configurations of the application hosting the environment (Katzan, 2011).

Rashid and Chaturvedi (2019) define PaaS as a complex cloud computing model in which the service provider manages all the system software and computational resources, and handles everything from the creation of applications to its hosting. PaaS services also include tools for collaboration, data integration, safety features, and scalable options, relieving customers of the complexity of handling hardware and software. The benefits are the system's scalability and flexibility in installing software. The problem in PaaS arises due to the limitations in the interoperability and migration between different service providers. Examples of PaaS are Database, Development and security testing, Database and Business Intelligence (Odun-Ayo et al., 2018).

While Rashid and Chaturvedi (2019) concentrates on the service provider's management of the system software and computing resources, Marinos and Briscoe (2009) highlights Google-App Engine's function as a technological platform for developers. Additionally, Rashid and Chaturvedi (2019)) underline the capabilities and tools that PaaS services provide to clients, while Marinos and Briscoe (2009) define the user's control over the applications and the hosting environment. In this research work, PaaS is used as the cloud service as it has numerous benefits such as speeding up the deployment process, and scaling up and down of the resources and performance according to the need and configuration. Azure provides access to numerous automated tools and it is cost-effective in building and maintaining infrastructure.

2.4.3 Software as a Service (SaaS)

According to Stanoevska-Slabeva and Wozniak (2010) Software as a Service is software that is controlled, supplied, and managed globally by several service providers on pay-as-

you-go. It is the most prominent layer of Cloud Computing for end users since it requires the use of software and programs. For the consumers and users, it is it has the special feature of being a low-cost utility that depends on the payment structure that removes the need for initial expense for the infrastructure. Salesforce.com, and Google Apps like Gmail, and Google Docs are examples of SaaS. Gong et al. (2010) explains that some companies and businesses proclaim to be cloud computing providers rather than that they are SaaS suppliers. When compared, it is evident that cloud computing is essential for a provider to supply scalable resources. In essence, it is a crucial side of any SaaS application, allowing the service providers to provide the same code and data to all subscribers. Susanto et al. (2012) finds that in SaaS, customers can have access to a service or application that is cloud-based for example Salesforce.com or google where critical customer service data is maintained as a component of the cloud service. As per Katzan (2011) consumers can deploy applications and services generated by the user or acquired from somewhere on the cloud. The required applications need to be created with the languages and technologies that the service provider has approved.

IaaS provides a considerable amount of control over the basic IT infrastructure, but the user is responsible for the security. Companies must implement strong security measures to guard their infrastructure from any kind of attack. This covers the management of access control, network security and safety policy enforcement. As PaaS conceals the foundational infrastructure, security is still a top priority. Companies must ensure the security of the apps and must follow best practices for data security that include using strong authentication, and encryptions of data during transmission and storage. As SaaS provides sophisticated software solutions by giving users minimal control over the infrastructure and security. Again, service providers must ensure that they are using a trustworthy SaaS provider with proper security. This consists of checking the provider's relevant security standards and measures such as ISO 27001 and proper access controls (Vordel, 2011).

2.5 Deployment Models

Parast et al. (2022) explains that the deployment models specify the level of exclusive access to the shared resource on the cloud and they are centered on the demand of the users as they can choose them according to their needs. Cloud Computing can be categorized according to its deployment method, and as emphasized by Thu et al. (2023), various cloud deployment models exist, such as:

1. Public Cloud: The public cloud is a collaborative cloud computing ecosystem in which many clients use the infrastructure on the platform of the service providers. Clients register with the service provider and pay-as-you-go concept for the pricing policy. Nowadays, the public cloud is the most widely used and dominant system.
2. Private Cloud: A private cloud is an exclusive facility that clients can use through their private network. it provides independent services, scalability, and growth which

mirrors the advantages of the public cloud. Private cloud provides higher security, privacy, and control by utilizing internal firewalls and storages and more IT infrastructure that helps to save sensitive data from third-party access and provides features such as control and customized support through dedicated on-premises resources.

3. Hybrid Cloud: A Hybrid cloud integrates the features and quality of public cloud and private clouds, which allows companies to capitalize on the advantages of each model while giving the best solutions to the consumers. The roles and responsibilities of the management are mainly specified by the companies and the service provider.
4. Multicloud: Multicloud refers to the cloud services that are shared and used by organizations or users to eliminate the dependency on one cloud service provider. Rather the companies use their unique capabilities for their business needs, for example, Amazon Web Services, Azure Microsoft, and Google Cloud Platform.

Each cloud deployment model has its own specific advantages and disadvantages when it comes to deployment, administration, and securing web applications. The public cloud provides simple access to resources and services yet lacks the control offered in the Private cloud. The private cloud gives clients more control over the IT infrastructure but needs more inclusion of clients in the management and updates. Conversely, hybrid clouds combine the benefits of private and public cloud computing, but they can occasionally need more intricate management and control.

2.6 Overview of Azure Cloud

Microsoft Azure also known as MS Azure is a sophisticated cloud computing service that is above a storage service. Apart from storage work, it includes a wide range of tools and services that are meant to help organizations and businesses including databases, VMs, and user directories. Ms provides hundreds of services like other cloud service vendors that users can choose according to their needs (“What Is Microsoft Azure Cloud?”, n.d.). According to Takabi and Ghasemigol (2019) Azure is a cloud platform that uses virtualization technology for the development and management of applications. It uses a hypervisor to separate hardware from the operating system which allows several VMs to run concurrently with different operating systems such as Windows or Linux. Microsoft Azure is a flexible cloud computing platform that offers IaaS, PaaS, and SaaS solutions. For IaaS, VMs, storage, and networking are the built part, but the client has to create and deploy the applications, Due to the facility of the hypervisor, Multiple operating systems are supported by Azures’s Hyper-hypervisor. PaaS provides pre-configured environments and auto-scaling through services like Azure Functions and App services. With SaaS products like Dynamics 365 and Office 365, Azure handles every step of the application from deployment to scaling. Al-Sayyed et al. (2019) characterizes Azure as a service provider that offers platform and facilities in addition to services with excellent scalability, affordability, and reliability. Some examples of services offered by Azure are Virtual Networks, Virtual Machine, Structure Query Language (SQL) Database, Azure Name Resolution,

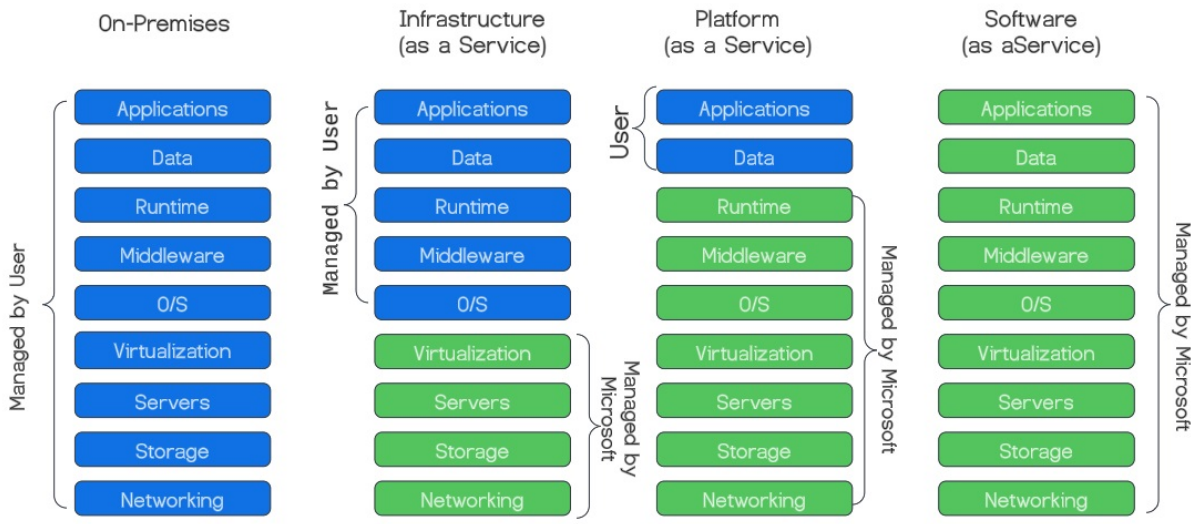

Figure 2.3: Azure Cloud Overview derived from (Goedtel, 2023)

Traffic Manager, Storage, etc.

Microsoft Azure seamlessly integrates with on-premises infrastructure that as a result allows businesses to use cloud resources without any difficulty and disturbance with their current IT infrastructure. With control and flexibility and control, MS Azure allows businesses to increase their investment and accomplish business goals and achievements. Figure 2.3 shows an overview of Azure Cloud.

2.7 Cloud Security Issues, Threats and Attacks

According to Rosencrance (2023), Cloud Security Alliance (CSA) is a leading organization that is working globally and is dedicated to defining and raising awareness for the guidelines to secure and protect the cloud computing environment. A vulnerability is considered different from software bugs as they are not self-exposed until triggered by an external source intentionally. Some threats that were mentioned in the annual report titled 'TOP Threats to Cloud Computing-Pandemic Eleven' in 2022 after the work contribution of 700 Cloud professionals. the threats mentioned in the report were:

1. Insufficient Identity, Credentials, Access, and Key Management: In Cloud security, Identity management plays an important role as it means giving users access to the assets, resources, accounts and many locations. Gratner's research predicted that by 2023 75 percent of the security fails will be the result of insufficient management in Identity, access, and privileges. Identity here relates to the systems, users, and servers (Kenner, 2022). The consequences of this threat can be negative performance and productivity in business, and employee testing fatigue which leads to the lack of compliance, data replacement, or disclosure by unauthorized access ((CSA), 2022).

2. Insecure Interface and APIs: According to Kenner (2022) with other components, API and other similar interfaces can also have vulnerabilities as a reason of misconfiguration, coding flaws, lack of authentication, and privileges. Some important takeaways to avoid the consequences are tracking of attack surfaces that are supported by the API, All the traditional policies must be updated to keep in step with changes in API, and automation in the organization should be implemented.

3. Misconfiguration and Inadequate Change Control: A report on the top Cloud Computing Computing Threats by Cloud Security Alliance (CSA) (Torkura et al., 2021) found that misconfiguration and inadequate change control were the most crucial security risks. The seamless deployment of resources like servers, and databases provided by service providers also brings the risk of misconfiguration and inadequate.

4. Lack of Cloud Security Architecture: The incorporation of the technical and commercial considerations to create a purposeful design is challenging because of the rapid growth of technology and the availability of self-service cloud infrastructure. Insufficient planning can leave cloud infrastructure open and prone to cyberattacks and reduce resilience (Michael Roza et al., 2022).

5. Insecure Software Development: Insecure software development means when developers choose not to create the infrastructure and layers of the platform themselves from the base level, instead they give the responsibility and management of these layers to the service provider (Mello, 2022).

6. Unsecured Third-Party Resources: An exploit can be initiated at any point and link in the chain and carry it to other products and services. This interprets that the hacker or intruder needs to find the weakest link to get access (IG, 2023).

7. System Vulnerabilities: The system vulnerabilities can hinder the availability, confidentiality, and integrity of the services, The vulnerabilities can include, vulnerable misconfigurations, use of default or weak credentials, and missing patchwork (IG, 2023). System vulnerabilities are those defects in the components of the system that are generally caused by human errors, which makes way for hackers to target the cloud services (Mello, 2022).

8. Accidental Cloud Data Disclosure: Data disclosure is a common and major problem in cloud users as a survey report shows that around 55 percent of companies have a minimum of one database that has been accessible to the public connection. As many databases hold weak and no credentials they become an easygoing target for attackers (Mello, 2022).

9. Organised Crime/ Hackers /APT: An attack event in which an attacker or group of attackers attempt to continue long unauthorized access in a network to acquire sensitive data is in a combined way known as APT or Organised crime. APT groups have a wide range of motivations that depend on and differ from one group to another group, for example, APT38 is known to be a state-sponsored group from North Kores that has a target to conduct cyberheists against international financial organi-

zations (Ferda, 2023).

10. Cloud Storage Data Exfiltration: Data that is private, protected, or sensitive can be exfiltrated from cloud storage systems. It is possible for someone who is not employed by the company to access, expose, steal, or use this data. Data exfiltration can be the main goal of a targeted attack and can be caused by application flaws, poorly implemented security procedures, or exploited vulnerabilities or misconfigurations. Exfiltration like this could be caused by misuse or human error, like misconfigured PaaS services. Storage objects may also reveal private information or files which are accessible externally through personal cloud storage services (Ferda, 2023).
11. Misconfiguration and Exploitation of Serverless and Container Workloads: In a recent report named *Top Threats to Cloud Computing: The Pandemic 11*, it was highlighted that developers are continuously facing difficulties in managing and developing the infrastructure to execute the apps and a strong responsibility must be taken for network and application security controls (Mello, 2022).

2.8 Azure Security Features

Since Azure ensures the privacy, integrity, and availability of their customer data, it is important to discuss the security features that guard against and lessen the likelihood of cybersecurity flaws in web applications. According to (Microsoft, 2023a) the responsibility for the security of an application or the services is dependable on the cloud service models used. According to (Troscianecki, 2023) Microsoft Azure offers a set of native security called Azure Security that work collectively to protect an Azure cloud platform. This protection includes physical framework, and network components, which include embedded services for identity, data, network, and applications. The client should be mindful of personal obligations to further strengthen the security of the private data used on an Azure cloud. Cobbins (2022) explores Microsoft Azure as one of the flexible and safe cloud platforms in the current business area. It contains a wide range of security tools that can be configured and provide support to further different types of services and features that are based on security purposes. The Azure platform provides built-in features and permits the implementation of solution partnerships under an Azure subscription. Six distinct functional sectors comprise the built-in features: Operations, Applications, Storage, Networking, Compute, and Identity.

2.8.1 Azure security tools

1. **Sentinel:** Sentinel from Microsoft provides a flexible based on the cloud platform for security orchestration, automation, and response (SOAR) and security information and event management (SIEM). For identifying attacks, threat exposure, and threat response, it offers cognitive security analytics and threat information to the entire company (Brook, 2023).

2. **Defender for Cloud:** Microsoft Defender increases security for Azure resources and offers advanced threat detection and response capabilities. With the help of integrated monitoring and policy management through Azure subscriptions, it rapidly identifies possible threats, deploys security solutions, and facilitates one-click remediation with a central dashboard within the Defender for the cloud console (Brook, 2023).

3. **Application Insights** Application Insights, is a web application performance management (APM) service and feature that offers for complete website performance assessment. It is designed to ensure maximum performance and gives the best-in-class possible experience to the users of the website developed. Additionally, it also contains a powerful analytics tool that helps to identify vulnerabilities and provides insight into the interactions of the web applications and user (Cobbins, 2022).

4. **Resource Manager:** Azure Resource Manager (ARM) serves as Azure's deployment and management service, that allows the creation, modification, and deletion of resources in an Azure account. It utilizes Infrastructure as a code which is based on JSON as a concept to enable the resource management and deployment (Troscianecki, 2023).

5. **Azure Monitor:** Azure monitor is a powerful tool that maximizes the availability and performance of applications. It provides an all-inclusive solution for collection, evaluation, and reaction to analytics from on- on-premises to cloud-based environments. Azure Monitor assists in effectively identifying and resolving issues affecting the application's performance. By providing users the ability to comprehend the surroundings and how apps behave, this facility enables users to respond effectively to system errors ("A Comprehensive Guide to Azure Security Tools and Features", 2024).

6. **Web Application Firewall (WAF):** The Web Application Firewall is a component provided by Microsoft in the Azure Application Gateway. It helps the applications to be protected from attacks like session hijacking and SQL injection (Cobbins, 2022).

2.8.2 Different Layers of Azure Security

Microsoft Azure offers a multi-layered security solution to its users to protect the applications and data. The different layers of Azure security are as follows:

Physical Security:

The starting point of any protection should be physical security. The breaches in physical security result in total vulnerability in contrast to the other layers that may fail with no consequences (Hoseini et al., 2022). With its advanced data centers located all over the world and its strong physical access restrictions, Microsoft Azure boasts an effective physical security framework that keeps its infrastructure safe from unwanted physical access ("A Comprehensive Guide to Azure Security Tools and Features", 2024).

Network Security:

The process of implementing the limitations to the network traffic to restrict access from the unauthorized users of a party or attack is known as network security. Azure provides a strong networking infrastructure that is tailored according to the requirements of the service and applications asked by clients (Lanfear, 2023). Network settings are separated and secured due to DDoS prevention, virtual networks and network security groups (NSGs). The Azure Virtual Network Gateway mainly protects communications among local networks and Azure resources ("A Comprehensive Guide to Azure Security Tools and Features", 2024). Microsoft strongly recommends in the case of web workloads to leverage Azure DDOs protection and implement a web application firewall to protect against DDoS attacks. Alternatively, platform-level defense against network-level DDoS attacks can be achieved by integrating Azure Front Door with a web application firewall.

According to Dahan (2022) nowadays after COVID 19, organizations have generated a need for a new security model that can better accommodate the multifaceted nature of today's workplace, to protect devices, apps, and data irrespective of location. Microsoft's Zero Trust Framework has the role of protecting the assets from any location by following three principles:

1. **Verifiy explicitly:** authorization and authentication must be carried out based on all the data points that are available and should encompass the user identity, devices, location, services, data sorting, and any deviations.
2. **Use least privileges access:** User access through just-in-time and just-enough-access(JIIT and JEA) must be restricted, risk-based adaptive policies should be employed and data protection measures to improve the security of data and productivity should be implemented.
3. **Assume breach:** In this principle, organizations it is encouraged to operate with a mindset that a security breach is always a possibility, and for a response, it is suggested segmenting the access and lowering the 'blast radius' to minimize the possible consequences of the breach. This principle also stresses that end-to-end encryption should be ensured.

Additional Network Security Features in Microsoft Azure

Apart from the zero trust framework Microsoft Azure has some other components that are offered in the area of network security (Lanfear, 2023). These are:

(a) Azure networking: Virtual machines should be connected to an Azure Virtual Network, which is a logical structure based on the high point of the Azure network fabric. To increase the security, every virtual network is segregated from other Azure users, to protect data and network traffic (Lanfear, 2023).

(b) Network access control: It is the process of restricting access to and from particular devices and subnets and the objective is to allow access only to authorized

users and devices (Lanfear, 2023). Azure provides various types of network access controls like Networking layer control, route control and forced tunneling, and virtual network security appliances.

(c) Azure Firewall: Azure Virtual networks are protected by Azure Firewall. It has an emphasis on high availability and the removal of the need for additional infrastructure elements. It is considered a reliable and scalable security solution for Azure Virtual Network services (Agarwal, 2021).

(d) Secure remote access and cross-premises connectivity: Azure resource setup, configuration, and management are remote activities that are considered to be accomplished with priority. Moreover, users planning to deploy hybrid IT systems that have the components on a cloud as well as on-premises (Lanfear, 2023).

(e) Availability: Availability is one of the crucial components in the cybersecurity domain. Azure provides high availability strategies by networking technologies:

- HTTP-based load balancing
- Network level load balancing
- Global load balancing

(f) Name resolution: The Azure DNS private resolver is a service that allows users to query Azure DNS private zones from an on-premises environment without the need for deployment of VM-based DNS servers (Kelleher, 2022).

(g) Perimeter network (DMZ) architecture: The segment of the network that provides an extra degree of security between assets and the internet to allow authorized access to the resources (Lanfear, 2023).

(h) Azure DDoS protection: It ensures constant application availability simultaneously protecting the excessive IP traffic costs (Copeland et al., 2015).

(i) Azure Front Door: It empowers users to define, manage, and monitor the routing of the web traffic while keeping the performance first and providing rapid worldwide recovery for increased high availability (Copeland et al., 2015).

(j) Traffic manager: The traffic that is based on the DNS worldwide, while maintaining high availability and responsiveness (Copeland et al., 2015).

Identity and Access Management (IAM)

Identity and access management are used as best practices to offer security in cloud systems. Cloud Security Alliance (CSA) offers a set of best practices that ensures identities and safe access through the resources. A report was generated that includes user access, division of responsibilities, identity and access reporting, and role-based access controls as components (Hashizume et al., 2013). Identity and access management as a framework allows users or systems the authorized access to resources. It consists of the process of identifying, authentication, and authorization by limiting permission (Matthews,

2024). According to TerryLanfear (2024), the process of authorization and verifying security measures comes under Identity management. These measures or identities can be users, groups, services, and apps. Microsoft protects the cloud and corporate data centers from user access using Microsoft's IAM and improves security by providing multifactor authentication and conditional access rules. The core functionality of IAM is to create, store, and manage identity information, It also allows users from other sources with passwords to access the system. It also manages user accounts and permissions and confirms the user identity using multifactor authentication. The access levels and user access control are also managed by IAM which also sets the privileges.

Threat Detection and Monitoring

Microsoft's users look for methods for the evaluation of the security standards of the environment and resources they plan to use. Azure Security Center (ASC), is a built-in platform that is designed for threat detection solutions. ASC provides machine learning techniques that intend to identify suspicious behavior patterns on different workloads like databases, storage accounts, VMs, and some malicious activities such as brute force attacks. It also provides sophisticated analytical capabilities that sensitize users about system threats and offer them to take preventive measures before any loss (George & Sagayarajan, 2023). In Microsoft Azure, Endpoint Detection and Response(EDR) plays a crucial role in handling security issues and protects against attacks like ransomware. It is designed to automatically defend user's endpoints and assets of the organizations from threats that can evade traditional antivirus programs and other security tools. The main key points according to Aarness (2023) of Endpoint Detection and Response are:

1. Continuously monitors endpoints: EDR installs software agents on the organization's components and logs continuous activity on the devices and provides visibility in the digital environment.
2. Telemetry Data Aggregation: The data from each device is collected and sent back to EDR, the data here can be event logs, application usage, authentication attempts, etc.
3. Data Analysis and Correlation: EDR helps companies to protect against attack by uncovering the Indicators of Compromise (IOCs) and utilizing machine learning and AI, providing tailored behavioral analytics.
4. Threat Identification and Remediation: When EDR solutions find a threat and mark it, alerts are sent to the security team and the EDR also isolates the affected endpoints to avoid the spreading of the threat during the investigation process.
5. Data Storage for forensics: Forensics records are maintained of previous events to get a thorough view of those threats that were undetected.

Microsoft has an important tool in the security sector which is Microsoft Security Copilot which employs data signals from multiple sources and places including Microsoft Sentinel, Microsoft Defender Threat Intelligence, and Microsoft Intune to identify threats and risks

and provide swift resolution. It also offers Enhanced and clear visibility and control over security posture with detailed reports and timely alerts. Microsoft pilot also reduces the cost and complexity of the security tools and workflows to provide a better and unified platform. With these incorporated models threat detection and monitoring in Azure makes it effective to detect threats (Hyun-Jin & Im, 2023).

Compliance and Governance

The main goal of Azure Compliance and Governance is to ensure the resources with proper management and compliance with regulations. The important components of defined by (Grimshaw, 2024) are:

1. Azure Blueprints: The facility helps the cloud and security teams to define and create repeatable sets of regulations for Azure resources that follow the standards, rules, and requirements of an organization. It also helps to bundle the different resources, patterns, and artifacts for access control, policy assignments to enforce compliance rules and regulations and Azure Resource Manager (ARM) templates to outline and define the infrastructure.
2. Azure Policy: This outlines the policies for each resource that ought to be followed to preserve consistency, and it permits the enforcement and automation of governance responsibilities and decisions.
3. Microsoft Defender: It improves the security features by automating threat detection and response and strengthens compliance with external regulations.

2.9 Web Application Vulnerabilities

Over the past many years, web applications have developed and transformed from being simple and static to dynamic, complicated, and interactive platforms are now have become essential for daily activities like the banking sector, e-commerce business, learning platforms tax filing, and various other activities because of the availability and accessibility globally due to internet. The management of sensitive data creates a security difficulty due to the popularity and emerging needs of businesses and humankind's dependency which makes web applications a prominent target for attackers. Web-based software vulnerabilities create a constant risk to companies and businesses as they can end up in data loss and disruptions in the services and access to resources. To get an oversight of the maintenance of software systems, companies need a proper method for the evaluation process (Wen & Katt, 2023). A vulnerability can be defined as a security detection that goes unnoticed in a program or an application. Attackers attempt to take advantages like unauthorized access to the application data by exploiting a vulnerability (Alazmi & De Leon, 2022).

Vulnerabilities are coding flaws that can seriously harm programs and applications when taken advantage of. The reason for these vulnerabilities is generally incorrect handling of

user input, which can lead to malicious code injection. This code can send out improper queries. incorrect syntaxes, and altered HTTP responses to obtain information about the user's session (SecurityScorecard, 2021).

Wen and Katt (2023) finds that Irrespective of the usage of firewalls and Secure Sockets Layer(SSL), and network and host security devices on web applications security the attackers mainly target the apps that these technologies cannot fight and stop. With the recent report from The Information-Technology Promotion Agency(IPA). As web applications are becoming very vulnerable to various attacks, the development of the software and technologies play a crucial role in it at various levels. The vulnerability of XML in enabling relations between heterogeneous web applications, many factors affect the security like lack of the security support in major frameworks and developer's preferences towards the need for security features all together contribute towards the security considerations and finally the security is degraded. The majority of attacks can be made possible by a variety of vulnerabilities that compromise the availability and functioning of the application, such as poor session information management, inappropriate authentication and authorization, and insufficient input validation (Deepa & Thilagam, 2016).

Web application vulnerabilities as explained by Deepa and Thilagam (2016) can be classified into three primary types:

1. Injection Vulnerabilities- These occur when an attacker modifies user-input parameters in a query to change its syntax. The insecure communication of data and application security may occur from these variables entering a trustworthy web page without required validation. The absence of validation or improper validation for user-controlled data is the main reason for injection vulnerabilities. There are various types of injection vulnerabilities that are based on the content of the injection like, SQL queries, HTML responses, LDAP statements, HTTP headers, etc.

2. Business Logic Vulnerabilities- These types of vulnerabilities provide attackers the ability and opportunity to change an application's business logic. These can be readily exploited, and attackers who take advantage of vulnerabilities employ legitimate application transactions to carry out undesired actions that are not part of routine business tasks or processes. For example, if a client on an online shopping cart has a coupon to redeem that can be used only once, a malevolent user could be able to use it more than once or unlimited times to receive a huge discount. Application flow bypass, parameter manipulation, and access-control vulnerabilities are some of the most prevalent and well-known forms of business logic vulnerabilities.

3. Session management Vulnerabilities - These suggest mishandling of the session variables, which are essential to preserving the state of an application. Attacks like clickjacking, Cross-Site Request Forgery (CSRF), session hijacking, and session fixation result from this.

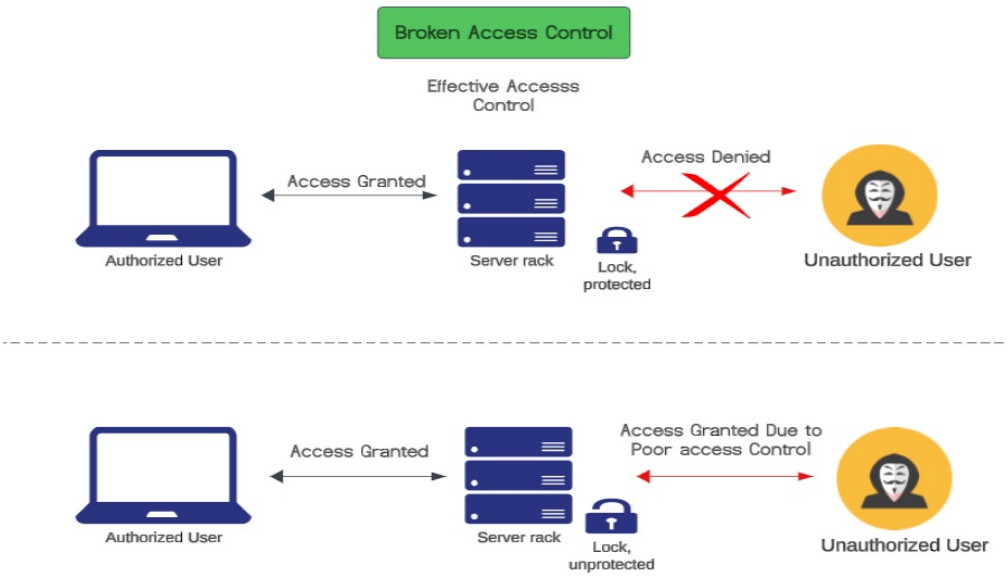

Figure 2.4: Broken Access Control derived from (Security, 2022)

2.10 Application based Vulnerabilities

The Open Web Application Security Project (OWASP) is a nonprofit group that works on online web security. It is a report that is regularly updated and highlights the top 10 security issues that are connected with web application security (Cloudflare, 2024). The OWASP is defined by Linskens (2023) as a standard document that contains the list of most significant security threats to web applications for the web application security and for the developers. The top 10 list of web application security risks are:

2.10.1 Broken Access Control

Access control ensures that the users should not get more permission than they require, as it exceeds the permission and can lead to unauthorized disclosure of data, modification of data, or destruction of data. An example of Broken access control is metadata manipulation including changes in cookies or hidden information, manipulation or replaying of tokens, and increasing privileges. Access to the required resource is a must but the principle of least privilege should be adopted (BasuMallick, 2022). Common broken access control vulnerabilities according to Linskens (2023) consist:

1. Violation of the principle of the least privileged as the access is granted beyond the requirement and capabilities, roles, and denial by default could be compromised.
2. Allowing unauthorized users to view or edit any account through insecure objects.
3. Intentionally and forcibly browsing the privileged pages or data as a regular user or accessing authenticated data as non-privileged or unauthenticated user. Figure 2.4 shows a Broken Access control attack.

2.10.2 Identification and Authentication Failures

Identification and authentication vulnerability was previously known as Broken authentication. It is a web-based vulnerability that happens as a result of the misconfigured session management. When the authentication procedure is completed, a session is generated and it becomes active for the exchange of data between the server and the user. below is a picture showing general broken authentication and session management exploitation (Hassan et al., 2018). The authentication failures may occur due to the following reasons as explained by OWASP (2024) are:

1. In this attacker employs a list of valid usernames and passwords, that as a result leads to automated attacks like credential stuffing.
2. It also allows attacks such as brute-force attacks.
3. Use of default, weak or famous passwords like '123456', 'QWERTY', 'Password', or 'admin/admin'.
4. Use of weak or those credentials that use knowledge-based answers that are ineffective and unsafe.
5. data is stored as plain text or use of the weakly hashed password is done.
6. Exposure of session identifier in the URL.
7. Session identifiers are reused after the login and user session tokens are not invalidated after logout.

2.10.3 Injection

According to Poston (2020) In an injection attack, the advantage of poor and low sanitization of user input is done as sometimes the developers did not properly handle the user input and it allows them to execute unauthorized commands. Failure to sanitize SQL user input can result in bypass authentication and run unauthorized queries against the database. OWASP defines injection as that vulnerability in which the attacker uses malicious code in an application to systems and compromises both systems such as the back-end and client-side when in connection with a vulnerable application. Injection vulnerability is one of the most persistent online application attacks. In this attack, the attacker sends malicious code or input to do unintended actions and the main reason for this is the absence of validation, and filtering by the application. Injection vulnerabilities are also known as Structured Query Language (SQL) injection in which database manipulation or exploitation is done by attacker (Linskens, 2023). As explained by Deepa and Thilagam (2016) the main reason for the injection attack is the insufficient validation of the user input data. The injection vulnerabilities can further be classified as :

1. SQL injection vulnerabilities: SQL injection is a type of vulnerability in which the attacker interferes in the database with different malicious queries and makes the attacker able to view, modify, edit, or delete the data which they were not indented to

(PortSwigger, 2024). Figure 4.37 shows an overview of SQL injection attack. Types of various SQL vulnerabilities explained by Kiprin (2021) are:

(a) Error-based SQLi: In this type of attack, the attacker is dependent on the error messages from the database to gather information about the database and its structure. This often reveals that much data is enough to enumerate the whole database. It is the most commonly encountered type of vulnerability in which an attacker provides incorrect input to a SQL query, for example, writing a string when an integer is needed (Li et al., 2019).

(b) Blind SQL injection: In this type of injection the attacker asks the database true or false questions that are based on the application's responses. In this generic error messages are shown and they are not mitigated even if they are vulnerable. It is similar to normal SQli but differs in the way of retrieving the data as the data gets stolen by asking questions based on true or false (Kingthorin, 2024).

(c) Content-based SQLi: In this injection vulnerability, the attacker forces the web application to provide different results that are based on true or false. In this, the output determines if the content of the HTTP response is staying the same or changing, in which even if the attacker gets nothing from the database as a result it can determine if the malicious payload is providing true or false (Kiprin, 2021).

(d) Union Based SQLi: In this vulnerability if the web application is vulnerable to SQL injection attack, the UNION keyword can be used by the attacker to retrieve results in the application's responses to the queries. Lack of proper insight to analyze and evaluate the risks in software and systems can lead to poor handling of business. When the crucial steps and phases of establishing necessary security controls, the security is compromised and the application faces a real-time threat(PortSwigger, 2024).

(e) Time-based SQLi: Time-based SQL injection is a form of Blind SQL injection, in which the attacker uses malicious payloads to make the database 'sleep' or pause for a limited number of times and confirms the presence of vulnerability.

2.10.4 Insecure Design

In OWASP, insecure design vulnerability is shown as 'missing or ineffective control design' that straightforwardly means that there is a flaw in the design and implementation of web applications. The insecure design cannot be fixed by easy implementations as it generally lacks proper security controls that can protect the application against any specific attack (Linskens, 2023).

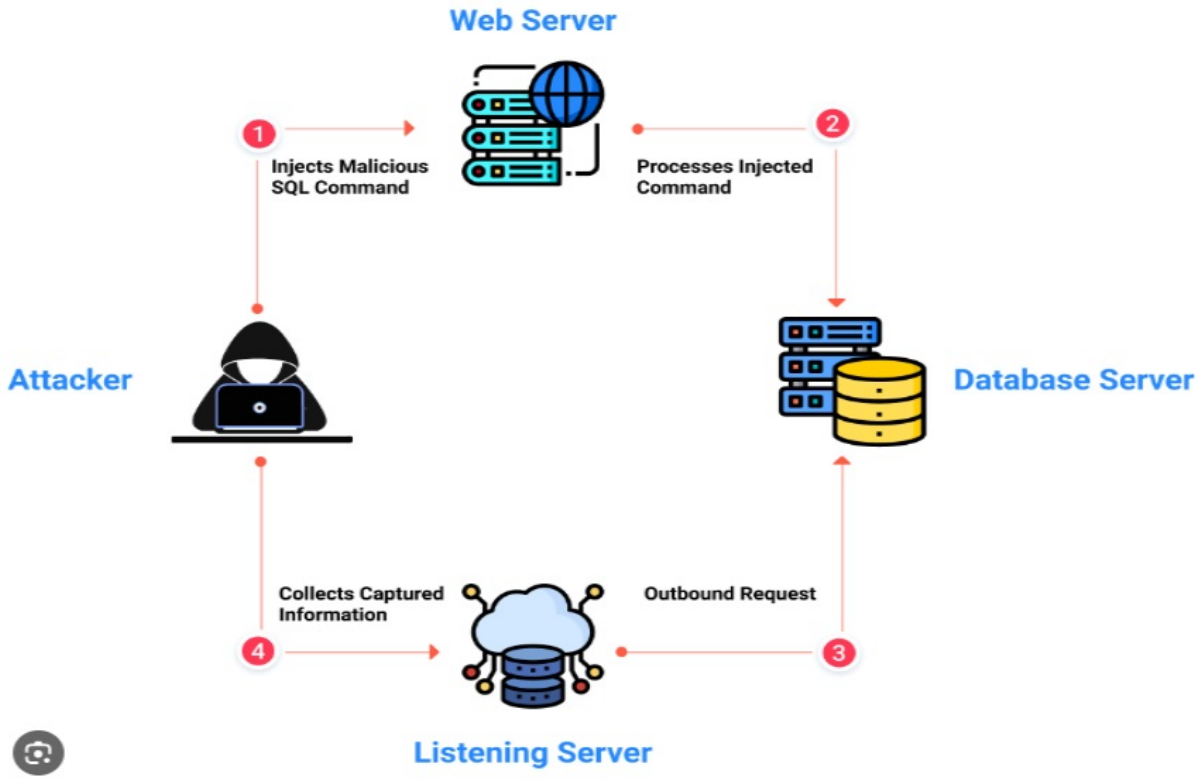

Figure 2.5: SQL injection taken from (Kukutla, 2023)

2.10.5 Security Misconfiguration

Security misconfiguration means that there are unpatched vulnerabilities are present within the open-source frameworks such as WordPress and Joomla. It leads towards, those security issues that are unaddressed in the configurations that can lead to Distributed Denial of Service (DDoS) attacks. The vulnerabilities can be caused by misconfiguration, database settings, custom codes, and libraries within open-source frameworks (Meier, 2006). Security misconfiguration is one of OWASP's vulnerabilities in which the user uses insecure configurations or default configurations which as a result leads the system to open for an attack. It is considered as one of the most prevalent kinds of vulnerabilities (Humayun et al., 2020). Loureiro (2021) explains that misconfigurations are seen as a soft and easy target as misconfigured web servers, and applications can be detected easily which plays as a low-hanging fruit for the attacker. These misconfigurations when exploited can lead to severe and large data leakage problems. The reasons for this vulnerability is simple such as, systems that are unpatched, use of default login credentials, unencrypted files, and insufficient firewall security.

2.10.6 Vulnerable and Outdated Components

The components in a web application like libraries, and frameworks are susceptible to any attack and become vulnerable to attacks, such as Laravel (PHP), and Django (Python). these codes can be further integrated with other components without any security attention which can lead to serious consequences in the business. Third-party components

that are outdated can lead to zero-day vulnerabilities and may allow attackers to break the sensitive system data and also to data breaches. resolution of this is a complicated process and can be a headache for developers (Valezquez, 2022). OWASP explains the main reason for this vulnerability:

1. If the user or developer doesn't know about the versions of all the components in a web application, be it client-side or server-side.
2. If the software is outdated or of an old version, the components here can Operating systems, web servers, libraries, environments, and databases.
3. When the vulnerability scans and assessments are not done timely.
4. When the software is unpatched, not updated, or upgraded.

2.10.7 Software and Data Integrity Failures

When inadequate precautions are taken to protect the data integrity, it is called software and data integrity failures. This vulnerability consists of weaknesses in application security and was introduced in OWASP top ten in 2021 which results in insufficient verification of integrity. There are many reasons that could lead to failures in integrity such as the use of unsupported and outdated third-party software, Insufficient vulnerability scanning, unpatched servers and platforms, and insecure configurations (Sengupta, 2022).

2.10.8 Security logging and Monitoring

According to OWASP, Insufficient logging and Monitoring, when combined with ineffective integration and incident response mechanisms, makes the attacker able to exploit the systems and as a result, illegal data extraction, destruction of data, and unauthorized tampering of data. This type of attack takes a longer time to be detected. Security logging and Monitoring implies the inability of an application to provide logs for security-related occurrences or any suspicious activity. These shortcomings may take the situation towards delay in identification and reaction against any security attack as it makes it easier to invade for attackers (Kelly, 2023).

2.10.9 Server-Side request Forgery

Server-side request forgery (SSRF) is a type of vulnerability in which the attacker inserts unintended requests to an unintended location, like an external resource or internal resource to manipulate the server-side application. The exploitation can result in the access and disclosure of sensitive data, and getting control over the server which poses a significant risk to security as the attacker can execute arbitrary commands (Journey, 2024). The BasuMallick (2022) explains that SSRF vulnerability happens when a web application retrieves an external resource without verifying the URL provided by the user. It works even if the system is protected by a firewall, or virtual private network (VPN) and allows

the attacker to induce a forged request to an unidentified location. The rate of SSRF attacks has been increased due to convenient features provided by developers to the end users like retrieving a URL.

Clements (2023) explains that in SSRF, the attacker after executing unintended commands sometimes makes connections to the internal running services like HTTP-enabled databases or can also send POST requests to fetch internal resources and access data that is not meant for public uses.

In the methodology section, Damn Vulnerable Web Application (DVWA) has been opted for the testing part. The DVWA is a web app that is designed purposefully vulnerable to practice web security testing as it provides a safe controlled environment and can be locally installed or on a VM (Academy, 2023). DVWA offers a diverse list of vulnerabilities, such as SQL injection, cross-site scripting (XSS), remote file inclusion, command injection, and many more. Each of the vulnerabilities is designed to mimic real-world situations and provides a wide range of security issues in web applications. As DVWA provides different vulnerabilities to exploit accessible via a dedicated menu item, it also offers three levels of security that are low, medium, and high each of them provides different levels of protection (Lebeau et al., 2013).

2.11 Case studies

In this section, some real incidents are discussed in which vulnerabilities are found either in web applications hosted on Azure Cloud or in the Azure Cloud itself. When data is used as a behavior:

1. A new critical remote code execution (RCE) vulnerability was found in several Microsoft Azure services, as reported in a report by 'The Hacker News'. Cross-site request forgery (CSRF) is a vulnerability in the popular SCM service Kudu that might be used by a malicious actor to take complete control over an intended application. The vulnerability was discovered and given the moniker EmojiDeploy by the Israeli cloud infrastructure security firm Ermetic. Attackers can insert malicious ZIP files containing a payload into the victim's Azure application by taking advantage of the vulnerability. Additionally, the vulnerability might make it simpler for confidential data to be taken and transferred laterally to other Azure services. Microsoft patched the vulnerability on December 6 2022 after the identification on October 26 2022 and awarded the bug bounty with a prize money of 30000 US dollars ("New Microsoft Azure vulnerability uncovered — EmojiDeploy for RCE attacks", 2023).
2. In an article named 20 Common Security Vulnerabilities and Misconfiguration in Azure by Cheah (2020) 20 most common vulnerabilities were thoroughly discussed. The author emphasized on the necessity of security and cloud experts to evaluate and patch the vulnerabilities in cloud computing. The article enumerates the top 20 most used accounts and configuration vulnerabilities in the Microsoft Azure cloud computing environment. Some of the vulnerabilities include a storage account ac-

cessible from the internet lack of multifactor authentication for privileged users and missing log alerts in Azure Monitor.

3. According to a report by Alspach (2022) in 2022, multiple security vulnerabilities are found in MS Azure including cross-tenant vulnerabilities, The article provides important information on a major cybersecurity incident, An extensively utilized database service on Azure's public cloud platform was effectively compromised by hackers. The attack exposed databases from thousands of client environments, including Fortune 500 companies, raising questions about the common security concerns associated with cloud infrastructure.

2.12 Summary

Apart from several benefits of cloud computing, there are always chances for vulnerabilities to occur. The fast adoption of cloud computing is also opening doors for opportunities for the attacker. The research chapter presents the characteristics, deployment models, and types of clouds of cloud computing. As cloud systems share resources, and infrastructure among the users, cross-tenant issues arise and provide a large surface for attackers to attack. As a result, it becomes difficult to segregate and protect sensitive data and applications. Furthermore, the dynamic nature of cloud computing infrastructure creates the chances of misconfigurations and increases the possibility of security breaches. The security features of Azure Cloud are also discussed to protect against various cyberattacks. However, the security features are only effective when organizations prioritize secure coding practices, implement strong access control, and do continuous monitoring to protect the web application deployed on Azure. The chapter further proposes a unified platform for representing and analyzing attack scenarios. This chapter also has reviewed different types of vulnerabilities based on different layers like application and network layers that are giving rise to the challenges from security breaches that have generated a huge need for security and cloud experts. This chapter aims to address these challenges by analyzing cybersecurity issues faced by web applications deployed on Azure, focusing on practical implications and proposing a structured methodology for security assessment and development.

CHAPTER 3

Research Methodology

3.1 Introduction

This chapter provides an outline and summary of the research methods used in the research project. It outlines the systematic approaches and tools used to collect data, analyze, and interpret data. The study design, data collection strategies, and analysis approaches that support the research process are explained to the reader in this part. This thorough synopsis makes it possible to comprehend the methodology and findings derivation of the study with clarity.

The purpose of this research study is to explore and analyze challenges faced by web applications hosted on the Azure cloud platform utilizing Peffer's approach. The reason for choosing Peffer's approach in the research work is that it highlights the key principles that are important for creating a strong and resilient security guide and framework that are tailored to web applications on Azure deployments. Peffer's approach has been opted for this research because of its problem-solving methodology, which has a clear objective to augment human knowledge and create inventive artefacts (Hevner et al., 2004). Peffer's approach also promotes the iterative development cycles, that help to refine and enhance the output or results (Peffers et al., 2007). . It also, puts considerable importance on the comprehensive evaluation, to measure the performance or output of the security framework using techniques like penetration testing and vulnerability scanning (Venable et al., 2016). As the Peffer's approach outlines a series of phases, the research methodology straightly maps to each of theses phases:

1. Problem Identification and Motivation: This phase emphasizes the necessity for a strong security architecture by emphasizing the difficulties in securing web applications on Azure.
2. Objectives of the research work: This phase significantly aligns with the research

objectives, which include investigating security best practices and detecting vulnerabilities.

3. Development: This phase is in the alignment with the testing procedures, which includes manual and vulnerability scanning, that will verify the efficiency of the designed framework.
4. Evaluation: In this phase, the impact of the framework in reducing vulnerabilities and improving the overall security hold of web applications deployed on Azure is assessed by the research work.
5. Communication: The last phase entails documenting and sharing the findings via a tailored security guide, to ensure that the findings will benefit the security community.

3.2 Problem Identification and Motivation

This phase entails locating the specific problem on which the research is based. The problem identified is the necessity to address the cybersecurity challenges and vulnerabilities in the web application deployed on the Azure cloud. The main aim is to identify and articulate the security threats and risks, faced by web applications like Damn Vulnerable Web Application (DVWA) as deployed on Microsoft Azure. The DVWA is an extreme example of a web application chosen for testing web applications because it is versatile and covers various vulnerabilities like SQL injection, Command injection, XSS, and more. It provides experience and a platform for both manual and automated testing techniques. It also includes recognizing the vulnerabilities and threats, that could compromise the security of the applications and infrastructure they are hosted on (Lebeau et al., 2013).

The another reason for choosing DVWA for the research work is that it provides a well-defined controlled and structured environment for testing with security measures that as a result facilitates reproducibility and comparison of the outputs and findings (Academy, 2023). By deploying DVWA on Azure, security concerns, relevant to Azure can be identified and explored with having familiar and common web application vulnerabilities.

The purpose of defining the problem statement is to highlight the importance of having a strong security solution that is specifically designed to deal with the issues found. The motivation stems from the growing number of cyberattacks that target cloud-based applications and mitigating the potential risks and threats and the necessity of protecting infrastructure and sensitive data.

3.3 Objectives of the Research Work

The primary objectives of this research work are:

1. To research and identify specific cybersecurity challenges inherent in deploying web applications within the Azure cloud environment,

2. To explore and analyze cloud-native security tools provided by Azure, such as Azure Security Center, Web Application Firewall, and Network Security Groups, to evaluate their effectiveness in securing web applications deployed on Azure.
3. To explore and evaluate a comprehensive set of cybersecurity best practices tailored for securing web applications in the context of Azure deployments, emphasizing strategies and measures that effectively address vulnerabilities identified within the Azure-hosted environment.

3.4 Design and Development

Peffer's approach at this phase focuses on the creation of the artefact or solutions that are based on the existing knowledge, and principles as this phase acts as a link between the conceptualization and implementation (Benali & Asri, 2021). The design and development phase helps the research work to develop a customized artefact related to the unique challenges and requirements of the problem addressed. The rationale behind each test and result is to enhance the security posture of DVWA deployment on Azure.

3.4.1 Deployment

In order to simulate a real-world scenario, a vulnerable website known as Damn Vulnerable Web Application (DVWA) will be deployed on Ubuntu 20.04 LTS, Äpache 2.4.54, MySQL 8.0.31 and PHP 8.2 hosted on the Azure cloud platform (A. Kumar & Taterh, 2016). DVWA is a web application that is based on PHP/MySQL and is intentionally made vulnerable to test environments legally (Zech et al., 2017). The deployment will be set up purposefully configured without any initial security measures in place. This configuration enables thorough testing and analysis of security holes and vulnerabilities in the DVWA application. The aim is to gain insightful information about potential threats and risks that can arise in web application deployments in the real world.

3.4.2 Artefact

The design phase of the research involves crafting a comprehensive security assessment guide specifically tailored for evaluating the security of web applications, such as Damn Vulnerable Web Application (DVWA), when deployed on the Azure cloud platform. The peffer's approach is generally employed for the creation of artefacts or assessment of an artefact to enhance current practices and expand research knowledge (Venable et al., 2012a). A research artefact consists of constructs, models, and methods, that are defined as new properties of technical, social, or informational resources and embed research contributions and includes defining functionality and actual creation (Mullarkey & Hevner, 2019). This guide will be meticulously structured to address the necessary security measures essential for safeguarding DVWA against potential threats and attacks. Its primary aim is to identify and mitigate potential vulnerabilities, offering a comprehensive

overview of Azure's security landscape while providing actionable measures to strengthen the defenses against cyber threats. By integrating Azure-native security features with established security principles, the guide seeks to establish a proactive defense strategy against common vulnerabilities. The resulting artefact will serve as a road map for conducting thorough security assessments, including detailed procedures for penetration testing and vulnerability scanning. It will also encapsulate key findings and recommendations derived from these assessments, and insights to enhance the security posture of web applications deployed on Azure. To ensure the DVWA site remains uncompromised during testing, access will be restricted through firewall rules and network security groups. Specifically, only the IP addresses of authorized testing tools hosting VMs will be permitted access to the site. This selective access control prevents unauthorized entities from interacting with the site, thereby reducing the risk of compromise during testing.

A security framework will also be designed that will be tailored for exploring and analyzing cybersecurity challenges faced by the web applications deployed on the Azure cloud. The components of the security framework may include:

1. **Access Control:** Utilize Role-Based Access Control (RBAC) to manage user access, restricting who can see and interact with resources based on their assigned roles and permissions. Access control will be at two levels, one is hosting platform level and the second is on the application itself. Authorised administrators can only access the underlying hosting platform and only authorized users can have access to the application.
2. **Encryption:** To avoid a man-in-middle attack, encrypted communication is required. SSL certificate is needed to secure communication between the client and server.
3. **Network Access Control:** Only allowed ports such as 443 to be allowed to open between client and server. Azure firewall and Network security groups can help in controlling port access.
4. **Vulnerability Management:** Inclusion of scanning helps to identify weaknesses in the system and fix security vulnerabilities in Azure cloud.
5. **Compliance:** Azure policy can be used to deploy compliant workloads. For example, only allowing specific software versions using policy or specific operating system versions.

Overall, the security assessment guide represents a crucial component of the research's design phase, offering a structured approach to address security concerns and fortify web applications against potential threats in the Azure environment.

3.5 Demonstration

The demonstration phase of Peffer's approach plays an important role in validating the viability and usefulness of the artefact to real-world relations (Benali & Asri, 2021). The

primary objective here is to do the testing part on DVWA and highlight the qualities and efficiency of the solutions and countermeasures to handle the threat and vulnerability. The 'Demonstration ' phase of the project involves shows that the security framework designed has significant importance, as it shows the practical demonstration of the testing part and implementation of security measures and the factors contributing towards, the overall security of web applications. The focus is mainly on finding the effectiveness and reliability of the security framework applied to DVWA deployed on Azure. The objectives of this phase are to Identify the vulnerabilities with testing and assessment of security controls, as reviewing of the configurations, and policies will be done. In short, The goal of the demonstration phase is to validate the effectiveness, and reliability of the security measures in protecting the DVWA against the threats.

The following are some of the vulnerabilities (among others) that will be tested in DVWA in the test environment. These vulnerabilities are derived from OWASP's top 10, which is a compilation of the most common and critical web security risks.

1. SQL injection
2. Cross-Site Scripting (XSS)
3. Cross-Site Request Forgery (CSRF)
4. Broken Authentication and Session Management
5. Security Misconfigurations
6. Sensitive Data Exposure
7. Insecure Deserialization
8. File Upload Vulnerabilities
9. Server-Side Request Forgery (SSRF)
10. XML External Entity (XXE) injection

The following steps are involved in the testing :

1. **Penetration Testing:** This testing will aim to uncover vulnerabilities, including but not limited to those mentioned above, through a combination of manual techniques and automated tools such as Metasploit, Nmap, Burp Suite, and SQLMap. These tools will be utilized to identify potential security flaws and weaknesses within the system.
2. **Vulnerability Scanning:** The role of vulnerability scanning tools is to generate a report that include the severity of the vulnerablity found like high level, medium level and low level severity and also helps to prioritize the countermeasures and remediation of the scanning results (Y. Wang & Yang, 2017). Vulnerability scanning will be performed using scanners like Nessus, Nikto, and Acunetix to identify and prioritize the vulnerabilities found within the DVWA deployment on Azure. This scan will provide a detailed report that will outline the identified vulnerabilities and their severity

levels with remediations.

3. **Manual Testing:** As automated testing is not the only criteria for exploiting the vulnerabilities of the web application and human need is always there to perform the attacks manually to find the vulnerabilities that are not low-hanging (Ciupa et al., 2008). A proper review of the output generated bu automated tools and scanners is necessary to identify false positives and prioritize them for further exploitation and investigation.

3.6 Evaluation

In alignment with Peffer's approach, the evaluation phase emphasizes the importance of evaluating the effectiveness and utility of the guide that is designed to provide security measures on DVWA deployments on the Azure cloud (Lawrence et al., 2010). It will also evaluate the correctness of the artefact created and if it goes with the security standard mentioned in the guide as the aim of this phase is to evaluate the artefact and its effectiveness to what extent it is helping to solve the given problem (Venable et al., 2012b). This consists of a comparison research work, comparison study (Benali & Asri, 2021). Data collected will be evaluated and an evaluation report will be generated, that will summarize the findings from the testing and vulnerability scanning. It will also analyze the ability of the framework and guide it to prevent, detect, and respond to the security flaws that target deployment on Azure. Assessment of the framework designed with the tools that are provided by Azure. The evaluation approach seeks to shed light on the effectiveness of the security mechanisms implemented, how well they reduce cybersecurity risks, and how well they can stop, identify, and address security problems in Azure DVWA deployments. The objective of the evaluation is to measure the effectiveness and utility of the guide designed to provide security measures for DVWA deployment on the Azure cloud. This evaluation is important to ensure that the guide created meets the intended objectives of addressing cybersecurity issues unique to the web applications hosted on Azure such as DVWA. The evaluation will also help to assess the security criteria and recommendations that are used to compare the artefact to determine its reliability. It means that security measures mentioned in the guide are aligning with security best practices.

The evaluation will follow some steps which include baseline measurement, control implementation, retesting, and final evaluation. This will provide a thorough assessment of cybersecurity issues and control efficacy while installing Damn Vulnerable Web Applications (DVWA) on Azure. Baseline measurement will first involve identifying and quantifying vulnerabilities in the DVWA instance through vulnerability assessments using penetration testing, vulnerability scanning, and manual testing.

Subsequent to this, cybersecurity controls advised for Azure deployments, including network security groups and measures for hardening DVWA, shall be put into effect. After the implementation of controls, a retest will be performed utilizing the same process as the baseline evaluation to assess the efficacy of the measures in mitigating vulnerabilities.

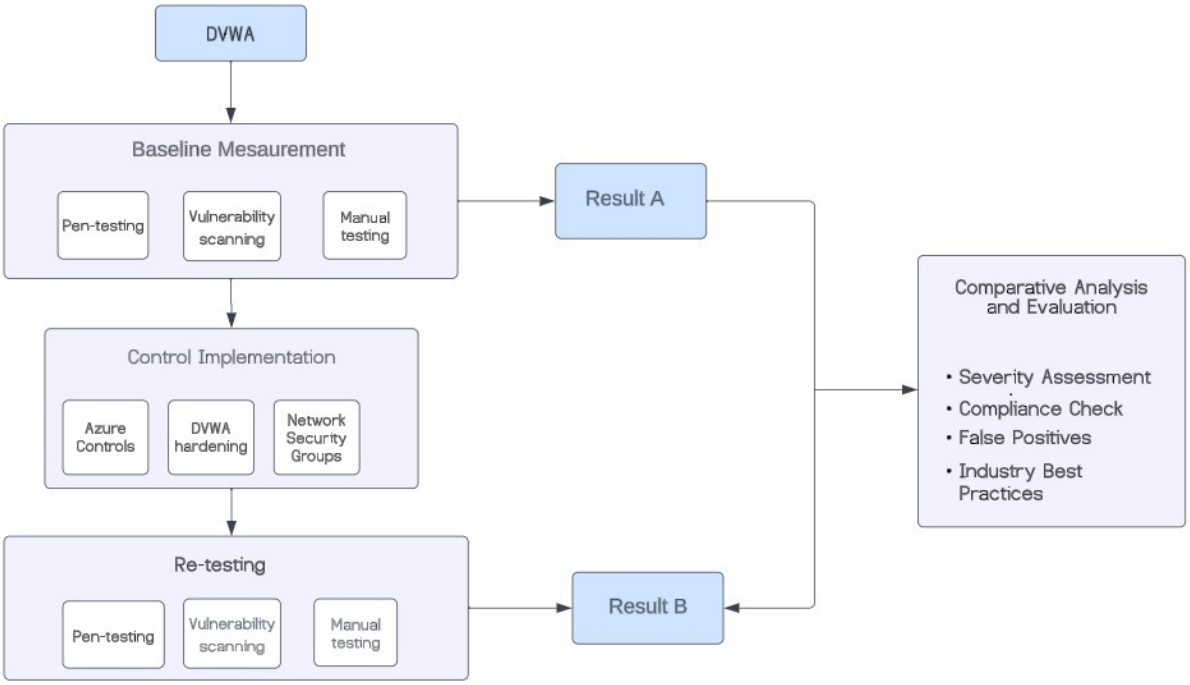

Figure 3.1: Evaluation Phase

Comparative analysis between the baseline and retest results will enable evaluation of the extent to which controls succeed in reducing or eliminating identified vulnerabilities.

The final evaluation will consolidate findings from both phases, taking into account factors such as the severity of remaining vulnerabilities and adherence to industry best practices. This comprehensive assessment phase will determine the overall cybersecurity improvement of DVWA on Azure and the effectiveness of implemented controls. Significant progress in reducing vulnerabilities is anticipated, but the assessment will also point out areas that need improvement, highlighting the significance of continuous vigilance and preventative security measures in cloud-based web application deployments. A foundation for future research and useful implementations aimed at enhancing the security of web applications in Azure will be laid by this evaluation process, which will yield important insights into the complex relationships between cybersecurity challenges and cloud deployment environments. The evaluation phase has been shown in the figure 3.1.

3.7 Communication

The findings and outcome of the research work and results of testing will be documented and communicated through a tailored research paper which is linked in the appendix. This guide will ensure to address the challenges faced by web application deployment on Azure and the countermeasures and mitigations of the vulnerabilities.

3.8 Limitations

The research work is prone to several limitations with one important of limited time in research. The research work could be affected as it will be done in a limited time of scope. Student subscription is generally accompanied by limited access to Azure resources, which can include VMs, storage, and network resources. The limited resources can create hurdles to computational utilization that could have an impact on the scope and depth of the research. As the student subscriptions often have limited allocation of funds for Azure services and resources. Special attention should be paid as spending outside of the limit can affect the research activities. The DVWA is specially designed for testing purposes and has purposefully exposed vulnerabilities that are meant to be instructional. This may not represent the full range of known vulnerabilities, but is a worst case that web applications could face in the actual world that as a result can limit the research work. As only DVWA is tested as a case web application, the research focuses only on the known vulnerabilities, which may not cover and include all the different kinds of threats web applications on Azure could encounter in the real world.

3.9 Summary

In summary, the methodology used in the research is intended to analyze and address security problems encountered by web apps housed on the Azure cloud infrastructure. With its emphasis on systematic problem-solving and iterative development cycles, the selected methodology—which is based on Peffer's approach—is ideally suited for building robust security frameworks customized for Azure deployments.

The phases that make up the research process are in line with the main principles of Peffer's approach. In the context of Azure web applications, the research emphasizes the necessity of a strong security architecture starting with problem identification and motivation. The research's objectives are then outlined in a way that highlights how they connect with Peffer's emphasis on vulnerability detection and investigative techniques. Peffer's emphasis on iterative improvement is mirrored in the development phase, which includes extensive testing techniques including vulnerability scanning and manual testing to verify the effectiveness of the security architecture that has been built. After that, the framework's effect on improving security measures and lowering vulnerabilities in Azure web apps is evaluated during the evaluation phase.

The communication phase is crucial because it places emphasis on sharing research findings with the wider security community via a customized security guide. This ensures that research insights are easily accessible and have practical applications. Essentially, the study technique provides an organized approach to resolving cybersecurity issues with the implementation of Azure online applications. Using Peffer's methodology, the study offers a thorough grasp of security concerns and workable solutions for improving security posture in the Azure cloud platform by fusing theoretical aspects with real-world

applications.

CHAPTER 4

Implementation Testing and Analysis

4.1 Introduction

Web application security is crucial in today's digital life as every aspect of businesses nearly depends on internet and so cloud also. This chapter provides a practical insight of the implementation of the strong security measures to the web applications deployed on the Microsoft Azure platform. In this chapter, the gap between security concepts and real-world applications is bridged with a step by step guide through. This chapter aims to furnish the reader with essential knowledge and tools crucial for effectively securing Azure web applications. It is organized into distinct sections to offer detailed insights into the step-by-step process involved in securing these applications. In the In Section 4.2, the process of deploying DVWA on Azure is presented to initiate the most basic step for testing the web application. real-world example is provided by deploying Damn Vulnerable Web Application (DVWA) to improve understanding and facilitate the implementation process. In Section 4.3, the initial configuration of the DVWA on a VM created on the Azure portal is shown which details in the download of DVWA, transferring the DVWA on the VM, and then creating the environment for the configuration is presented. Further in Section 4.4, web application vulnerabilities identified before the security configuration are discussed, and a testing process to find them is shown. Section 4.5 discusses the outcomes of the pretesting and areas of requirement in configuration. The next Section 4.6 thoroughly explains specific security configurations that are implemented within Azure, such as Network Security Groups (NSGs) and Web Application Firewalls (WAF). These configurations provide the base for securing the web application. Further Section 4.7, focuses on the testing of the overall security posture of the targeted web application. Different tools and technique are utilized to stimulate real-world vulnerabilities and after that assessment of the efficiency of the security measures. This testing after the configuration provides the validation that the security measures as configurations provide the expected

level of security. The Section 4.8 delves into the results of the testing done manually and by tools and provides the insights into the evaluation of the security posture. The finding of strengths and weaknesses of the security framework helps further identifying areas for further improvement. The chapter's main conclusions are outlined in Section 4.9, which highlights the significance of a thorough approach to web application security on Azure.

The entire process of implementation, testing, and analysis is presented in figure 4.1 as a framework which gives overview of the steps and phases that will be followed in this chapter.

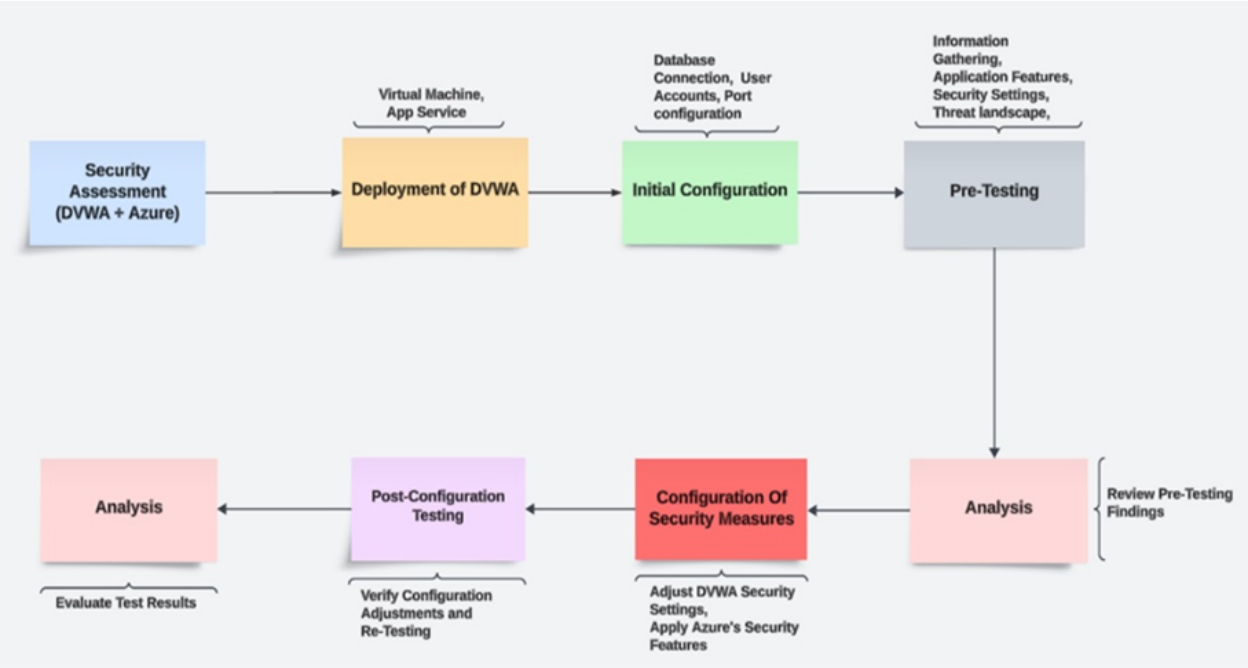

Figure 4.1: Flowchart of Implementation and Testing Process

4.2 Deployment of DVWA on Azure

This section outlines deploying DVWA on Microsoft Azure's cloud platform. It covers the steps to create an Azure resource group and a virtual machine. The process details logging in, configuring the VM (including authentication and network settings), and deploying DVWA. This includes cloning the DVWA repository, configuring the web application and MySQL database, and finally launching DVWA. The detailed explanation has been presented in the Appendix A.

4.3 Initial Configuration

This phase in this chapter focuses on the configurations done to create and set up the environment for the newly deployed Azure VM. Following is the breakdown of each step with details:

Download DVWA

1. Resource: The DVWA web application was downloaded from the official OWSP download page (https://github.com/digininja/DVWA) website. The dummy copy of the config file provided by DVWA is required to be copied into place with appropriate configurations. The command used for download was: git clone https://github.com/digininja/DVWA.g
2. Features: As mentioned in the GitHub repository, DVWA is a PHP/MySQL web application which is intentionally designed with vulnerabilities (Digininja). The key features it has such as it enables user to practice various web vulnerabilities with multiple difficulty level with a user friendly interface.

Transfer DVWA to VM

To transfer DVWA to the VM which involves SSH, a pre-established remote connection to the VM is needed and for this, an SSH client PuTTY was utilized.

Install DVWA

Command line (terminal) was used on the VM was used to direct to the directory and documentation of the DVWA to install was followed. The detailed steps are mentioned in the above section.

DVWA Environment Configuration

Operating system: Ubuntu Server 20.04.6 LTS, Network Configuration (Azure): Network Interface: dvwa-testing24_z1 Virtual Network /subnet: DVWA-Testing-vnet/default Public IP Address: 51.120.241.52 Private IP Address: 10.0.0.4 Network Security Group: Network Security Group (NSG) is an essential part of Azure's security architecture and is used to control traffic from and to Azure resources that are deployed inside of an Azure Virtual Network (VNet). The role of NSG is basically providing guidelines that are applied to Azure resources, It controls all the inbound and outbound traffic and utilizes these rules to allow or deny particular mentioned packets. These rules are monitored and configured similarly to usual firewall configurations (Thornton, 2019). The following rules govern inbound and outbound network traffic to and from the VM network interface:

1. Rule 1(SSH): Inbound traffic on port 22 with the TCP protocol is allowed from any source to reach the VM. This likely facilitates SSH access for remote management.
2. Rule 2 (AllowAnyCustom80Inbound): Inbound traffic on port 80 with protocol TCP for DVWA is allowed from any source to reach the VM The initial design is represented in figure 4.3
3. Rule 3 (AllowVnetInBound): Inbound traffic on any port and protocol from any resource within the same virtual network (DVWA-Testing-vnet) is allowed to reach the VM. This enables communication between resources within the private Azure vir-

tual network. figure 4.2 shows the Network Security Group rules that are applied to govern in Network Security Group.

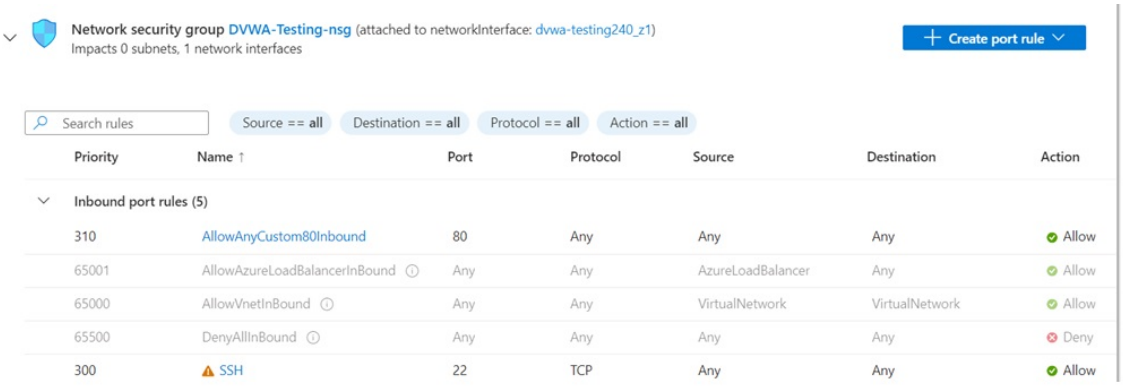

Figure 4.2: Network Security Group

4. Database Configuration: MySQL version: 8.0.36-0ubuntu0.20.04.1 (Ubuntu). Database Configuration: MySQL version : 8.0.36-0ubuntu0.20.04.1 (Ubuntu).

4.4 Pre-testing

This section of the chapter outlines the pretesting phase for the deployment of the Damn Vulnerable Web Application (DVWA). The objective of pre-testing is to explore and test the vulnerabilities within the DVWA environment under controlled conditions. Exploring and testing these vulnerabilities, allows users to acquire valuable insights into the intricacies of web application security, aiding the identification and mitigation of the vulnerabilities found. Appendix B provides thorough testing using both manual and automated tools.

4.5 Analysis

This analysis examines the security posture of the targeted web server deployed on Azure cloud based on various scan results. Several serious vulnerabilities were found during a security analysis of the targeted web server. Click-jacking attacks and improper content rendering can occur when security headers are missing. Exploitable vulnerabilities can be generated by outdated server software and accessible PHP information. Furthermore, the risk of DoS attacks, illegal data modifications, and information disclosure is increased when unnecessary HTTP methods are allowed on specific directories. To greatly strengthen the website's security posture, it is imperative to include missing headers, update the server, secure PHP information, and limit access to superfluous directories and HTTP methods.

4.6 Configuration of Security Measures

A Linux virtual machine (VM) was set up on Azure, hosting an Apache web server and MySQL to run a Damn Vulnerable Web Application (DVWA) website. This VM is deployed in a subnet within a virtual network (VNet). Controlling access to this subnet and its resources is managed by a Network Security Group (NSG), serving as Azure's primary security mechanism.

Administrators access the VMs through SSH on port 22, using the VM's assigned public IP address. The VM is assigned both a public and a private IP address within the subnet. Initially, security configurations for the deployment were minimal or nonexistent, permitting access to the website via the HTTP protocol, as all ports were open in the NSG.

The initial design is represented in figure 4.3

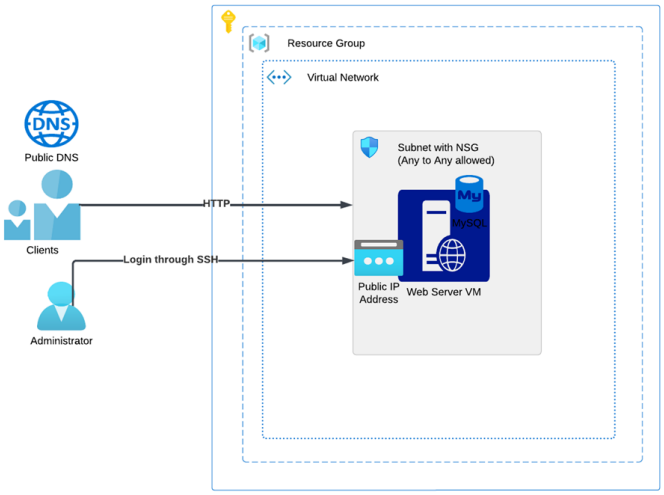

Figure 4.3: Initial insecure design

4.6.1 Network Access Control and Encryption

Multiple layers of protection can be implemented to safeguard the webserver and protect the DVWA website from potential attacks and vulnerabilities. As an initial step, configuring the Network Security Group (NSG) to block unwanted ports is crucial. In the figure below, the NSG is adjusted to allow inbound traffic only on ports 80 and 22.

Since the DVWA website currently lacks SSL encryption, it is accessible solely using the HTTP protocol. To enhance security, the next step involves securing the website with SSL traffic, this is shown in figure 4.4

Figure 4.4: NSG configuration for http and ssh

To secure the website with an SSL certificate, the cost-effective option of using a “Let’s Encrypt” free certificate was chosen. However, this certificate requires a domain name. Thus, a domain named kirticsoc.no was acquired from the domain provider domene.no, which also offers public DNS services alongside domain purchases.

The domain was meticulously configured on the webserver to direct the DVWA site to the domain name kirticsoc.no. Additionally, a free SSL certificate was generated for the domain name kirticsoc.no and applied within the Apache webserver’s configuration file.

Subsequently, public DNS changes were made to point the Azure webserver’s public IP address to kirticsoc.no. To ensure secure browsing, HTTP redirection was configured on the webserver to redirect all HTTP traffic to the HTTPS URL.

Following these changes, adjustments were made to the NSG settings to allow TCP port 443 and block port 80 which is shown in figure 4.5. This step further hardened the security posture, ensuring that only HTTPS traffic is allowed, thereby guaranteeing the website’s security with the SSL certificate, and restricting access to the HTTPS URL only.

Figure 4.5: NSG configuration for HTTPS and ssh

Accessing the site from a web browser using the domain name “kirticsoc.no” was attempted, and it functioned as expected and can be clearly seen in figure 4.6. figure 4.7 shows the SSL certificate showing the general details of Issuing details, Validity period, and hashing algorithm.

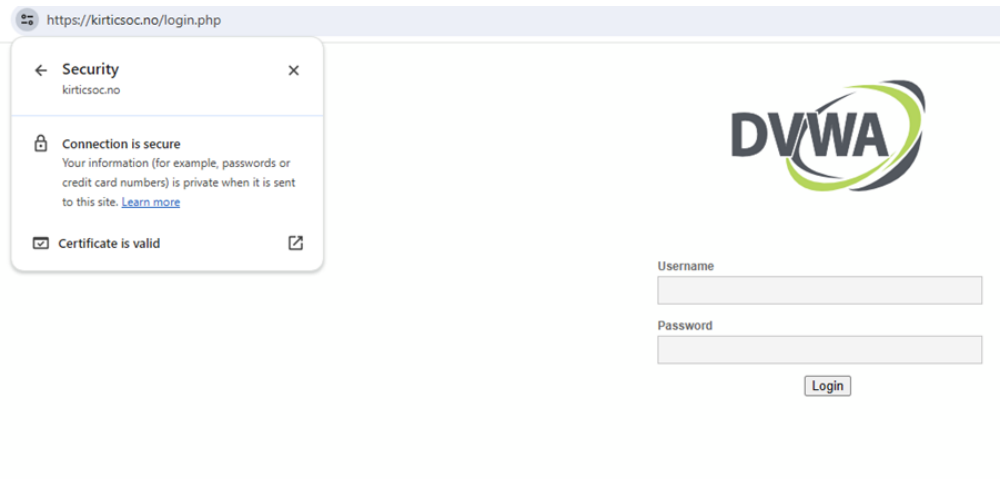

Figure 4.6: Secure DVWA website using valid SSL certificate

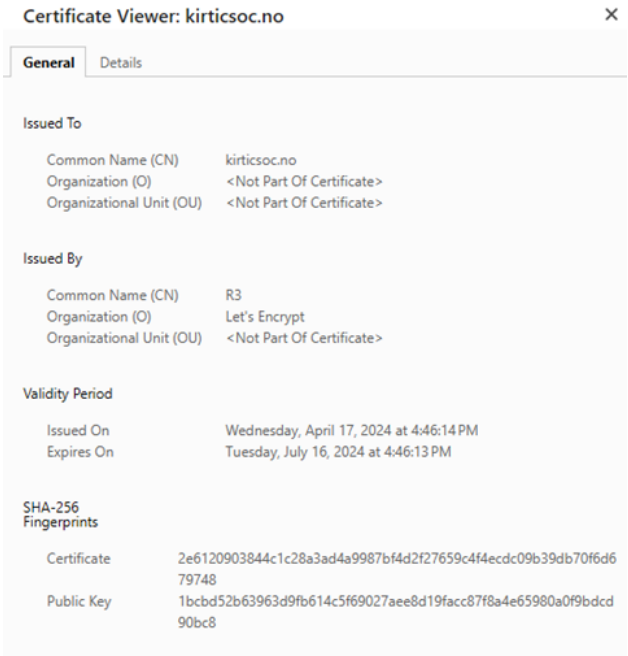

Figure 4.7: SSL certificate

4.6.2 Azure Firewall

Azure Firewall provides effective safeguards for Azure web applications. It centralizes threat screening and inspects both incoming and outgoing traffic. This ensures consistency in security requirements among all web servers and makes security administration easier. Azure Firewall detects and blocks threats including as malware, intrusion attempts, denial-of-service (DoS) attacks, and other unwanted traffic. By filtering traffic at the virtual network's perimeter, Azure Firewall keeps these dangers from reaching the web servers. Azure Firewall also provides built-in Web Application Firewall (WAF) features that protect the web server from common web application vulnerabilities, such as blocking malicious HTTP requests. It's considered a security best practice to ensure that the public IP of a web server is not directly accessible by clients. Exposing a production web server directly to clients can introduce security risks and is not recommended architectural practice.

To mitigate these risks, an Azure Firewall was deployed by creating an additional subnet dedicated specifically to the Azure Firewall. This architecture introduces an additional hop between clients and the web server, adding an extra layer of security. Clients accessing the web server must now pass through the Azure Firewall, which can enforce security policies, inspect traffic, and provide protection against various threats before allowing access to the web server. Figure 4.8 shows the working overview of the Azure Firewall in securing a web server.

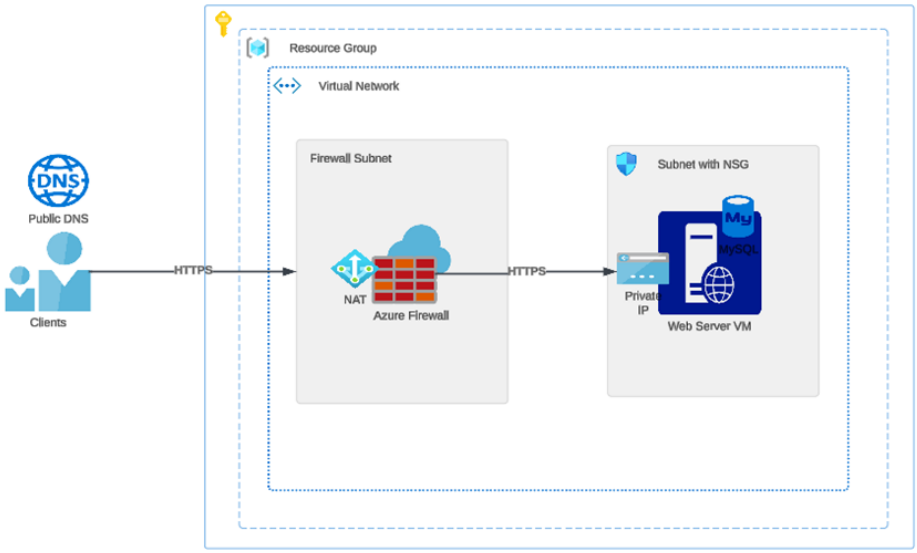

Figure 4.8: Securing Web server using Azure Firewall

Azure firewall secures webservers using the idea of Network Address Translation (NAT), in which traffic is routed to the Azure firewall's public IP address and then to the webserver's private IP address. A DNAT rule was established in Azure firewall to divert traffic from the Azure firewall's public IP address to the web server's private IP address, and another network rule was configured to allow only https traffic from the internet while blocking all other protocols. To minimize unauthorized attempts, network rules can be designed to allow just a certain range of client IP addresses, allowing communication only from authorized sources. Figure 4.9 and figure 4.10 shows the DNAT and network rules applied on the Azure firewall respectively.

Rule Collection Pr...	Rule collection na...	Rule name	Source	Port	Protocol	Destination	Translated Address or F...	Translated Port	Action
Rule Collection Group: DefaultDnatRuleCollectionGroup with priority 100.									
200	HTTPS-DNAT-Inbound	HttpsDNat	*	443	TCP	51.13.60.58	10.1.0.4	443	Dnat

Figure 4.9: DNAT rule

Rule Collection Priority	Rule collection name	Rule name	Source	Port	Protocol	Destination	Action
Rule Collection Group: DefaultNetworkRuleCollectionGroup with priority 200.							
200	Allow-HTTPS-Inbound	httpsInbound	*	443	TCP	10.1.0.4	Allow

Figure 4.10: Network rule

An application rule can also be enabled in Azure firewall which can do TLS inspection of the traffic. TLS inspection allow the firewall to inspect network packets and can control the potential threats. However, this feature is part of Standard or Premium tier of Azure firewall and this configuration task was done using basic Azure firewall tier. Figure 4.11 shows Application rule on the Azure Firewall.

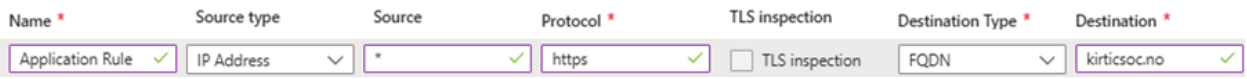

Figure 4.11: Application rule

A Threat intelligence policy can be enabled on Azure firewall to alert and/or block traffic from known malicious IP addresses and domains. The IP addresses and domains list are maintained by Microsoft cybersecurity teams. This feature is also part of standard or Premium tier (Vhorne & Coulter, 2023) Azure Firewall offers the option to activate a Threat Intelligence policy, which can both alert about and potentially block traffic originating from recognized malicious IP addresses and domains and is shown in Figure 4.12. These lists of IP addresses and domains are curated and updated by Microsoft's cybersecurity teams. It's important to note that this functionality is available exclusively in the Standard or Premium tiers of Azure Firewall (Microsoft, 2023b).

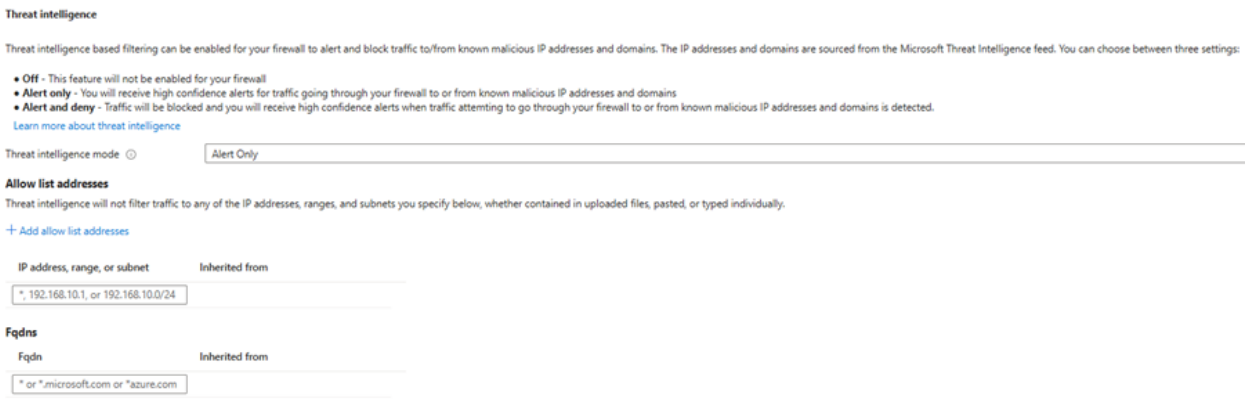

Figure 4.12: Azure firewall threat intelligence

An impactful attribute of Azure Firewall is its Intrusion Detection and Prevention System (IDPS), a functionality solely available in the premium tier. This system swiftly detects attacks by scrutinizing for distinct patterns, like byte sequences in traffic, or known malicious instruction sequences employed by malware. To delve into encrypted traffic, TLS inspection must be activated. Microsoft manages and routinely updates signature databases, which this feature relies on. IDPS can shield web applications from a spectrum of threats including malware, ransomware, SQL injection, cross-site scripting (XSS), and Denial of Service (DoS) attacks (Arghire, 2023). Figure 4.13 shows the IDPS mode of the Azure Firewall.

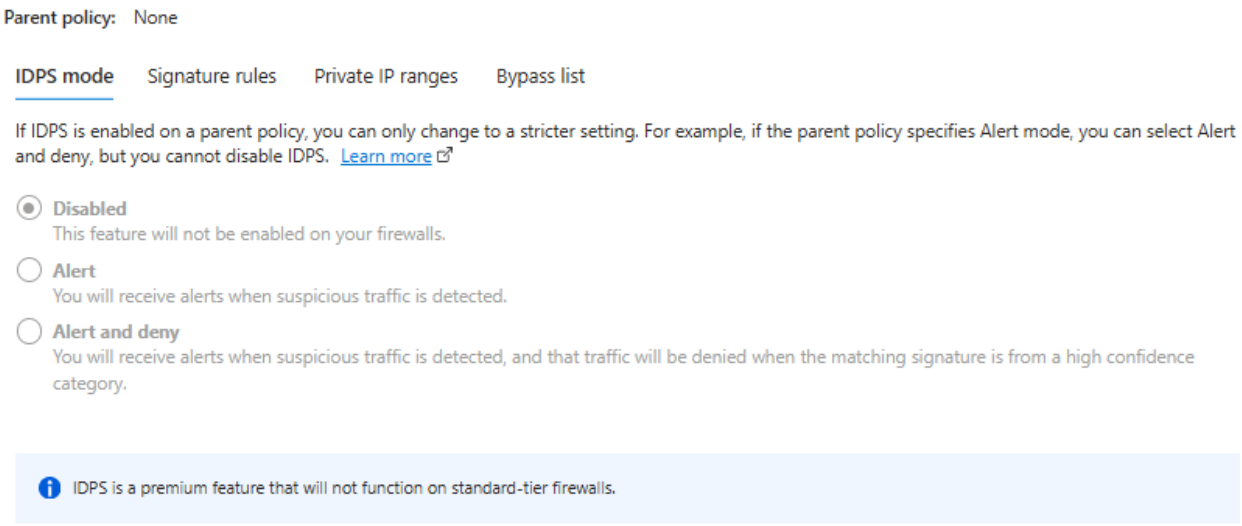

Figure 4.13: Azure firewall IDPS

4.6.3 Application Gateway

Another method to enhance the security of an Azure web server is by utilizing an Application Gateway, an Azure service that provides load balancing capabilities operating at layer 7 of the OSI model. While Application Gateway is typically utilized for load balancing across a pool of web servers, it can also serve as a primary defense mechanism for a standalone web server, preventing direct exposure to clients.

Application Gateway intercepts the web session from the client and establishes a separate session with the backend web server. This enables Application Gateway to act as a gateway, filtering traffic based on criteria such as source IP addresses, thereby helping to block malicious traffic.

Furthermore, Application Gateway supports SSL offloading, allowing it to decrypt and encrypt HTTPS traffic before forwarding it to the backend web server. This helps offload the processing burden of SSL encryption and decryption from the backend server, improving performance and scalability. Figure 4.14 is providing a demonstration of Application gateway securing a web server.

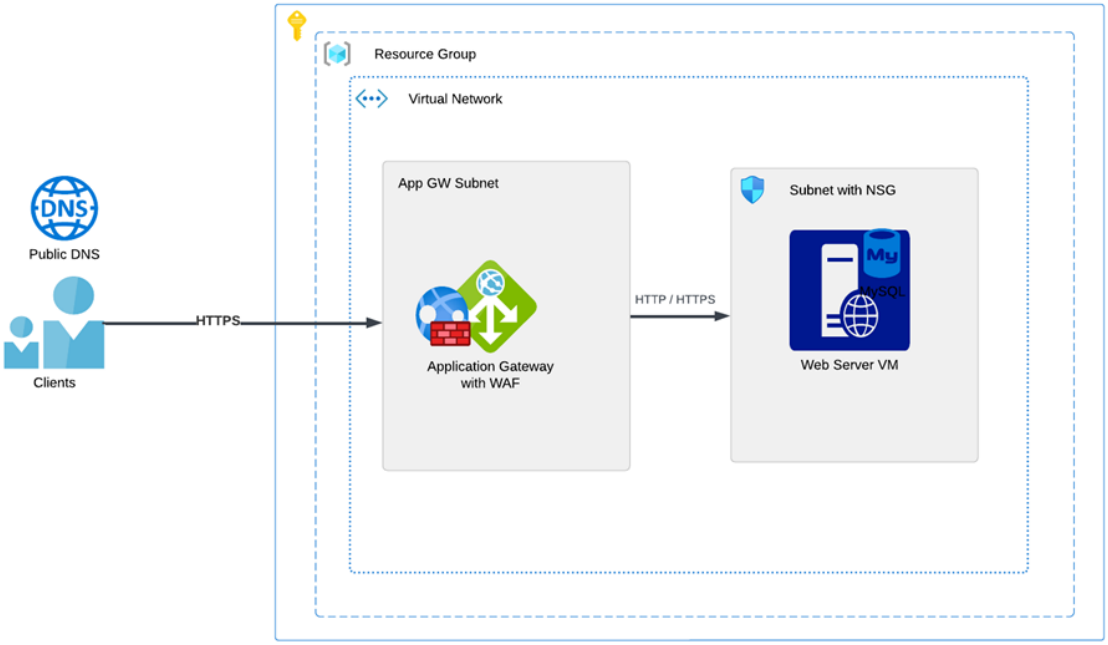

Figure 4.14: Securing Web server using Application Gateway

Application gateway v2 standard was implemented with WAF which possess the ability to modify the information within HTTP headers, thereby preventing the exposure of web server details to clients. This functionality, known as the “Rewrite Set,” allows for the concealment of information such as the web server version, host operating system version, and PHP version and is shown in the Figure 4.18 with Rewrite rules and a popup to show the list. By creating a rewritten set, details regarding versions of Ubuntu, Apache, and PHP were concealed, and instead, the value “No Info” is displayed. Further it can be seen in the Figure 4.15 and Figure 4.16.

```
HTTP/2 302
date: Tue, 30 Apr 2024 09:55:45 GMT
content-type: text/html; charset=UTF-8
server: No Info
x-powered-by: No Info
```

Figure 4.15: Server info is hidden

Rewrite sets	Rewrites	Rules Applied
HideServerInfo	1	1
		httpsrule

Figure 4.16: Hiding server info

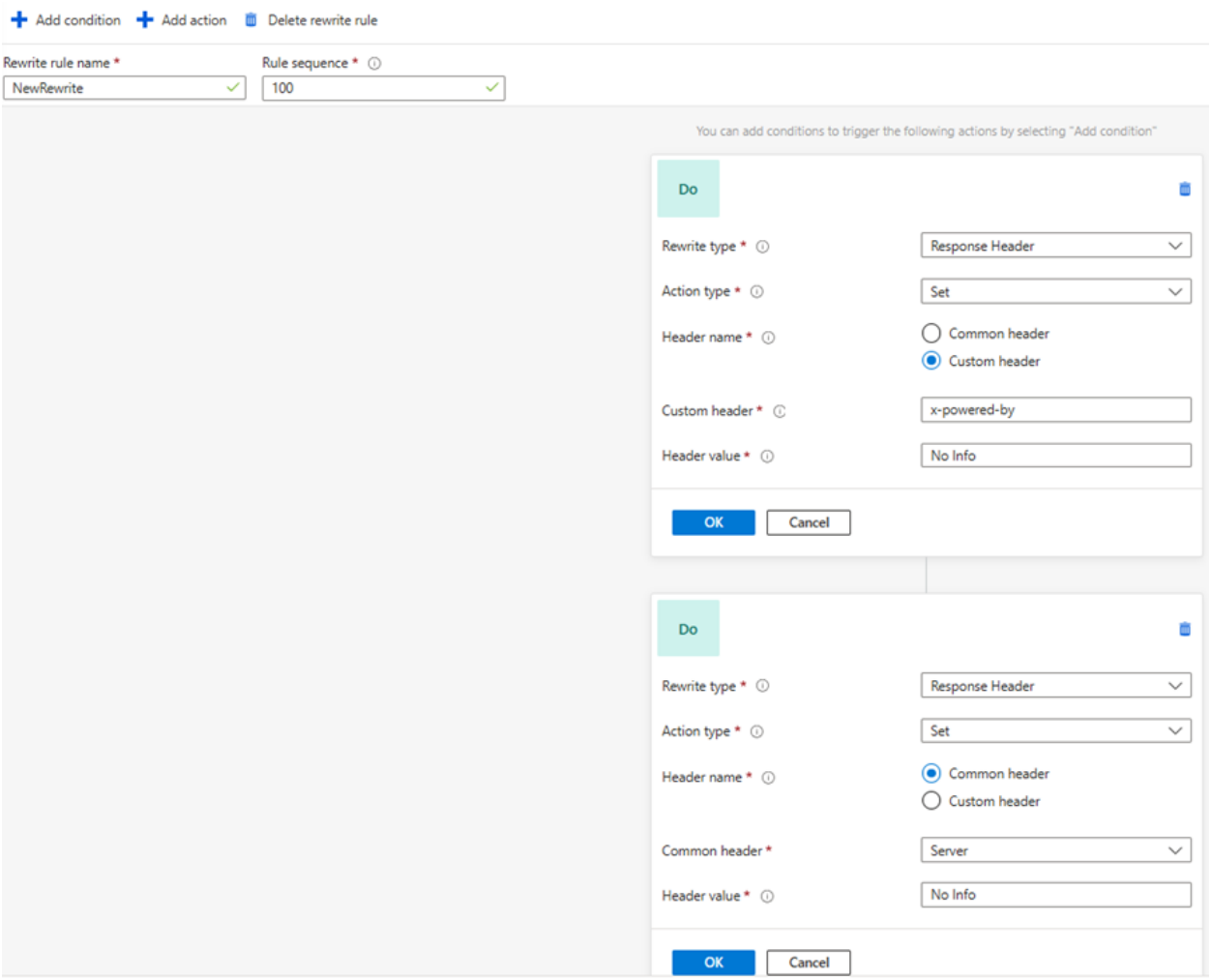

Figure 4.17: Azure App gateway Rewrite rules

Application Gateways include Layer 7 features, including an intelligent and powerful firewall known as the Web Application Firewall (WAF). To enable the WAF, switch from the “Standard V2” to the “WAF V2” pricing tier. The WAF may inspect incoming web traffic for various threats such as SQL injection, Cross-Site Scripting (XSS), remote file inclusion, XML injection, command injection, brute force attacks, and session hijacking. When the WAF is enabled, Managed WAF rule sets are automatically active. These rule sets are intended to protect the web application from the common threats identified in the OWASP Top Ten categories. The alert for any attack is generated and can be seen in the monitoring alerts in Azure Monitor. The rules sets are presented in the figure 4.18. The figure 4.19 shows an alert spike when command injection was executed on the targeted host. A SQL attack alert is shown in Figure 4.20.

Managed Rulesets Exclusions

A pre-configured rule set is enabled by default. This rule set protects your web application from common threats defined in the top-ten OWASP categories. The default rule set is managed by the Azure WAF service. Rules are updat

Policy mode is set to Detection. Detection mode monitors and logs all threat alerts if logging is enabled. Learn more

Assign Add exclusions Refresh | Enable Disable Change action

Filter for any field | Rule group : (All) | Status : (All) | Action : (All) | Group by Rule set

Rule Id	Description	Action	Status	Exclusions	Rule group	Rule set
OWASP_3.2 (186)						
200002	Failed to Parse Request Body.	Anomaly score	Enabled		General	OWASP_3.2
200003	Multipart Request Body Strict Validation.	Anomaly score	Enabled		General	OWASP_3.2
200004	Possible Multipart Unmatched Boundary.	Anomaly score	Enabled		General	OWASP_3.2
911100	Method is not allowed by policy	Anomaly score	Enabled		REQUEST-911-METHOD-ENFORCEMENT	OWASP_3.2
913100	Found User-Agent associated with security scanner	Anomaly score	Enabled		REQUEST-913-SCANNER-DETECTION	OWASP_3.2
913101	Found User-Agent associated with scripting/generic HTTP client	Anomaly score	Enabled		REQUEST-913-SCANNER-DETECTION	OWASP_3.2
913102	Found User-Agent associated with web crawler/bot	Anomaly score	Enabled		REQUEST-913-SCANNER-DETECTION	OWASP_3.2
913110	Found request header associated with security scanner	Anomaly score	Enabled		REQUEST-913-SCANNER-DETECTION	OWASP_3.2
913120	Found request filename/argument associated with security scanner	Anomaly score	Enabled		REQUEST-913-SCANNER-DETECTION	OWASP_3.2
920100	Invalid HTTP Request Line	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2
920120	Attempted multipart/form-data bypass	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2
920121	Attempted multipart/form-data bypass	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2
920160	Content-Length HTTP header is not numeric.	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2
920170	GET or HEAD Request with Body Content.	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2
920171	GET or HEAD Request with Transfer-Encoding.	Anomaly score	Enabled		REQUEST-920-PROTOCOL-ENFORCEMEI	OWASP_3.2

Figure 4.18: WAF Managed ruleset to protect web app against most known threats.

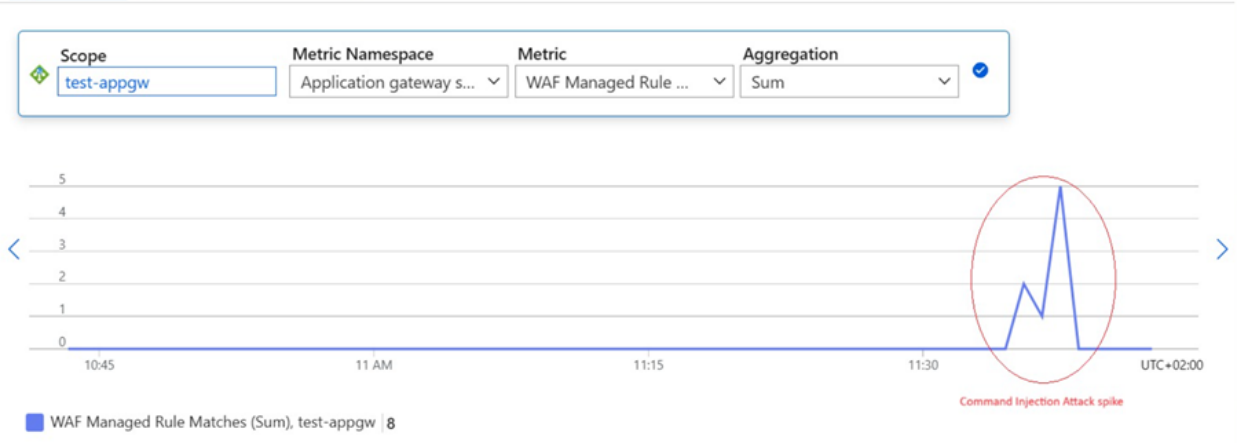

Figure 4.19: Appgateway alert spike during command injection

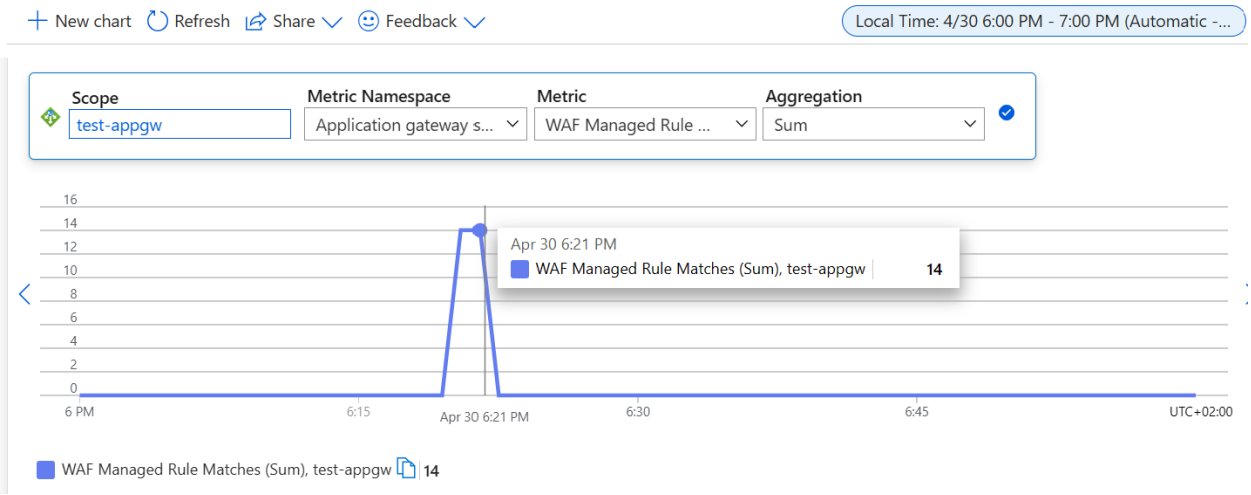

Figure 4.20: App Gateway alert in SQL injection

4.6.4 Zero Trust

To enhance security measures, combining an Application Gateway with Azure Firewall can be employed to architect a system based on the zero trust model, fortifying the protection of the web application. In order to improve security controls between individuals, systems, data, and assets that may change over time, zero trust is a cybersecurity strategy that moves from a location-centric to a more data-centric approach (Yeoh et al., 2023). The zero trust framework has been discussed in detail in subsubsection 3(Chapter 2). The zerotrust architecture is shown in the figure 4.21 with Application gateway and Azure firewall.

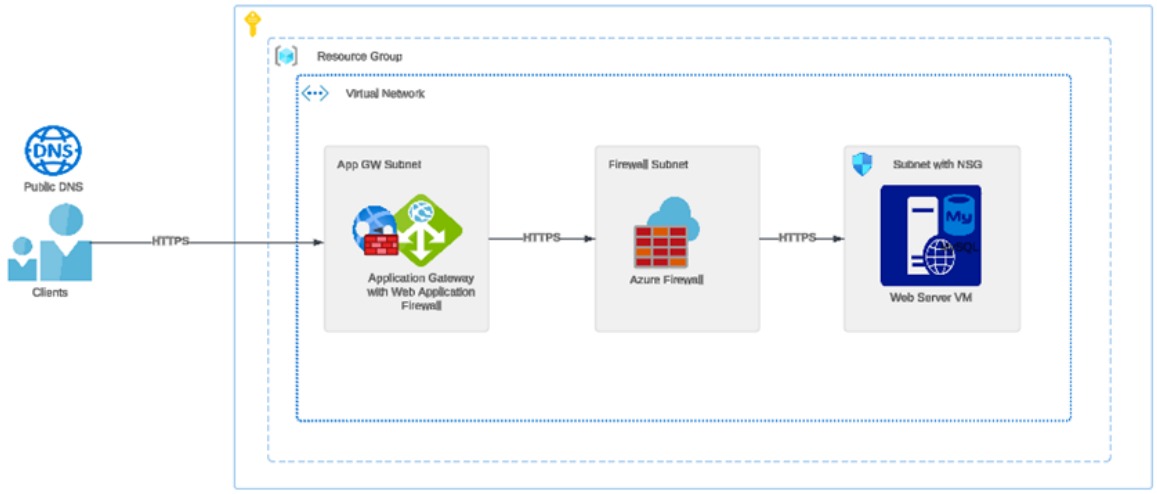

Figure 4.21: Zero trust architecture with Application gateway and Azure firewall

The traffic flow begins by passing through the Application Gateway, then continues through an Azure Firewall before reaching the web server, thus establishing a multi-layered defense strategy. In the event that an attacker manages to bypass one layer of defense, they must still contend with the subsequent layers, thereby bolstering the overall security

posture of the architecture. The Application Gateway provides basic filtering and Denial-of-Service (DoS) mitigation capabilities, while Azure Web Application Firewall (WAF) offers advanced threat protection against web application vulnerabilities. Positioned between these layers, the Azure Firewall delivers advanced threat protection features such as signature-based threat detection, which identifies and blocks traffic patterns indicative of malware, exploits, and Denial of Service (DoS) attacks. Additionally, Azure Firewall conducts stateful inspections to analyze the traffic flow and connection state, enabling the identification and blocking of suspicious activity patterns.

4.6.5 Encryption

As previously mentioned, the encryption of the web application is achieved through the utilization of an SSL certificate deployed on the web server. Additionally, the same certificate is applied to the Application Gateway, serving as the initial point of contact for web traffic. To enable this encryption, the SSL certificate is exported from the web server as a PFX file and subsequently imported into the web listener of the Application Gateway. Consequently, all communication between the client and the Application Gateway, as well as between the Application Gateway and the web server, is encrypted utilizing this SSL certificate, ensuring secure data transmission throughout the entire network pathway and is presented in figure 4.22.

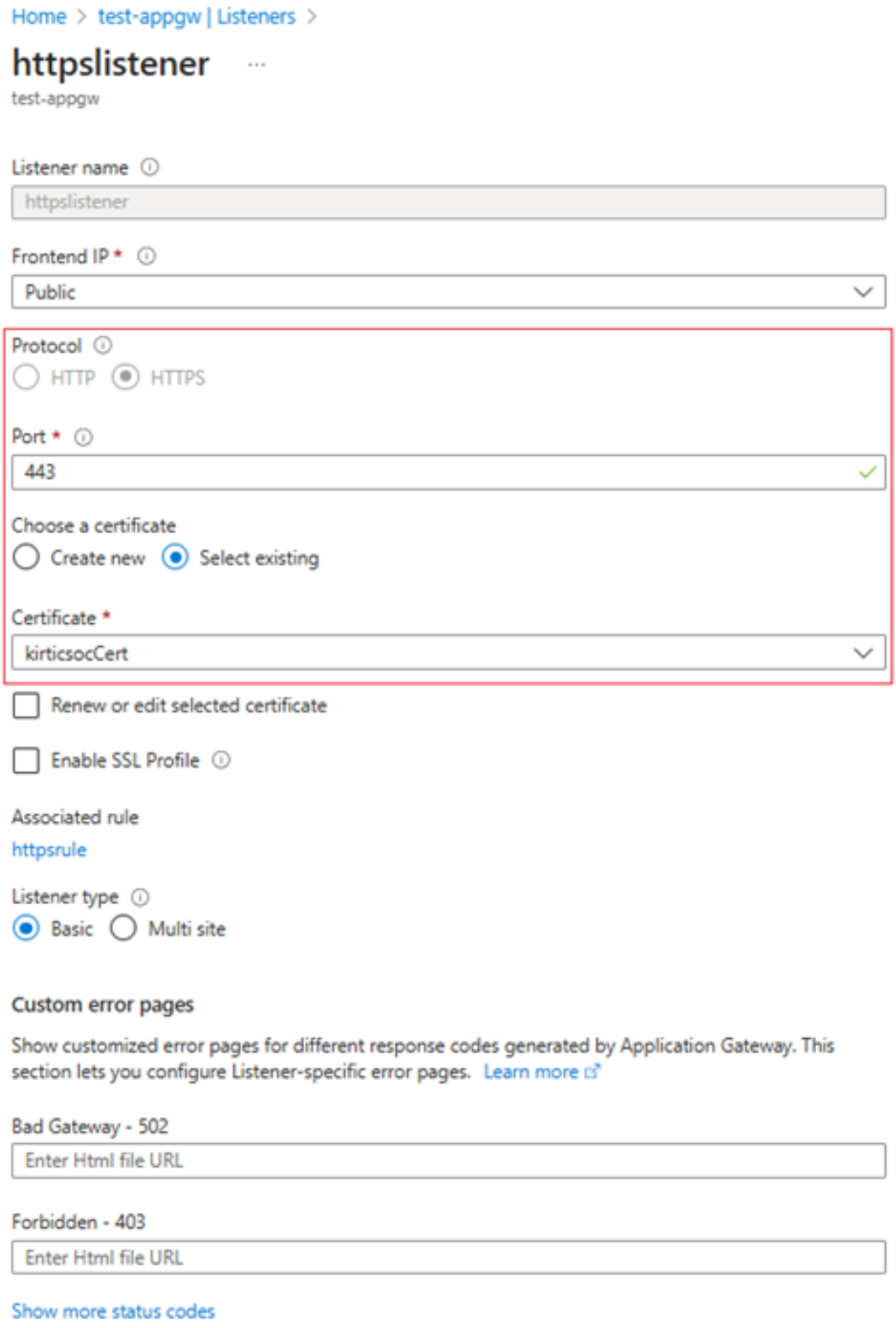

Figure 4.22: Application gateway listener configuration using SSL certificate

Although network traffic is encrypted, encrypted data at rest is also required for security reasons. The web server is deployed on an Azure virtual machine with an operating system disk; by default, all Azure VM disks are encrypted using a platform-managed key, which is an encryption key managed by Azure; however, if necessary, a customer-controlled key can be created as well. Figure 4.23 shows data on Azure-managed disks is protected transparently with 256-bit AES encryption, one of the strongest ciphers (Microsoft, 2024).

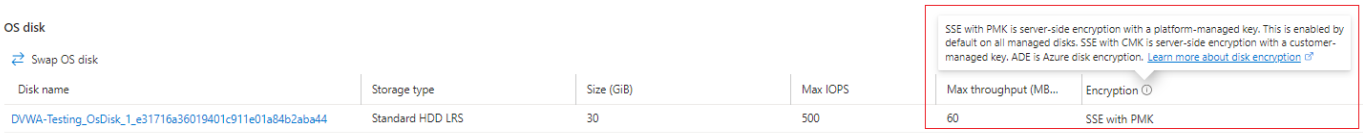

Figure 4.23: Azure VM disk encryption

4.6.6 Access Control

Control and monitoring of access to the Ubuntu web server can be achieved through Azure's role-based access control (RBAC) system. By navigating to the IAM settings of the Azure VM, administrators can add user accounts with the necessary roles. Roles such as "Virtual Machine Contributor" and "Virtual Machine Administrator" are well-suited for this task as shown in figure 4.24..

Figure 4.24: Adding role assigning using RBAC and IAM

Login to VM using Entra ID user ids, can be enabled when creating the VM, the option to "Login with Microsoft Entra ID" must be enabled, however it can be enabled later as well in the VM settings, this part is shown in figure 4.25. Enabling it uses SSH certificate-based authentication.

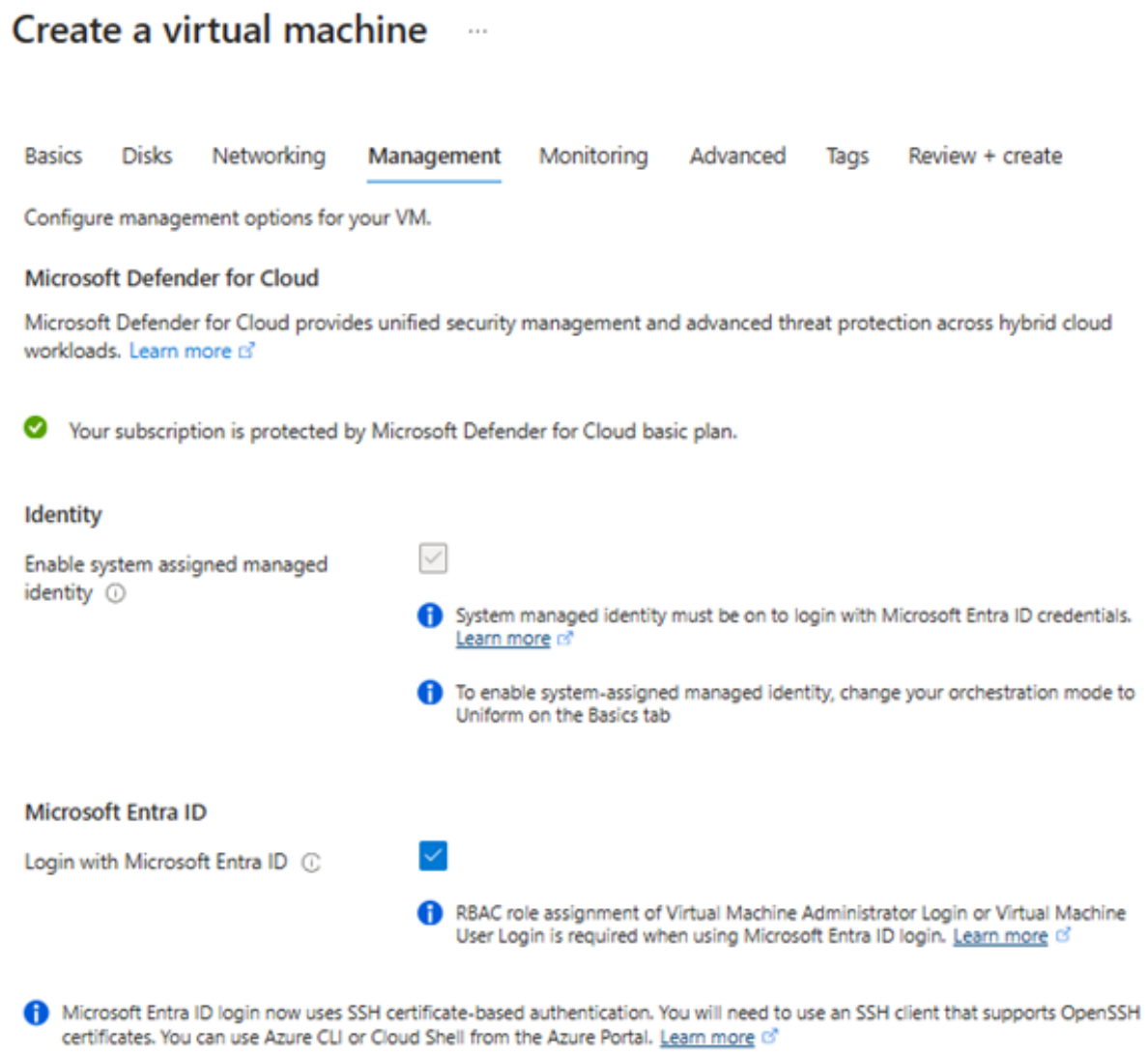

Figure 4.25: Enabling login to Azure VM using Entra ID

Users having IAM roles on VM can log in using their Entra ID credentials. To assign roles to others, the role assigner must be the Azure subscription's Owner or User Access Administrator. Entra ID and RBAC can be used to control and monitor access.

4.6.7 Compliance

Compliance with Azure workloads can be managed and controlled using the Microsoft Azure service called Azure policy. Azure policy helps in enforcing rules and regulations regarding resource configurations and compliance requirements. Azure Policy helps in defining policies that specify the desired state of Azure resources, such as virtual machines, storage accounts, networking configurations, etc. There should be one policy definition and one policy assignment. Policy definition defines the rules and conditions that a resource must adhere to within the Azure environment. Once a policy definition is created, it needs to be assigned to a specific scope of the Azure environment. This scope could be an Azure subscription, resource group, or individual resource. When a policy definition is assigned, it becomes active, and Azure Policy begins evaluating resources

against the defined rules. An example of policy assignment is shown in figure 4.26.

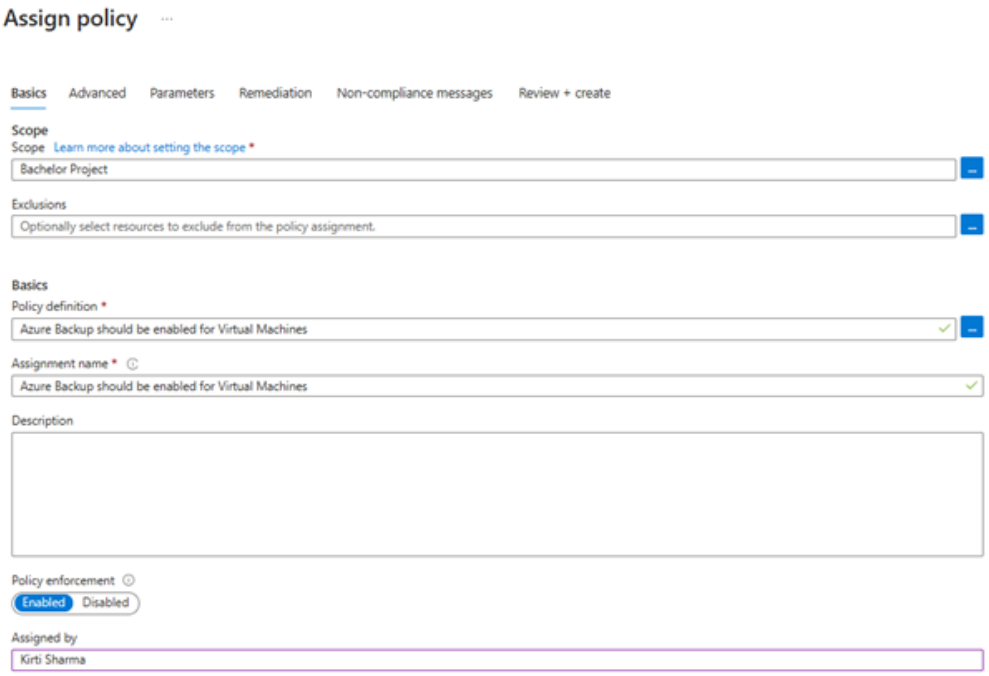

Figure 4.26: Policy assignment example

Some important Azure policies that can be applied to Azure web application workloads are –

- Minimum version of TLS protocol allowed for incoming connections to the web server.
- Enforce the use of endpoint protection solutions on VMs hosting web applications.
- Define backup policies to regularly back up critical data on the web server
- Implement IAM policies to control access to resources associated with the web application.
- Enforce HTTPS connections for web applications.
- Enforce strong VM password complexity.
- Enables Azure Security Center to continuously monitor your VM for vulnerabilities and threat
- Deny public network access to VMs

4.6.8 Vulnerability Management

The inclusion of scanning tools helps to identify vulnerabilities across Azure cloud infrastructure, including virtual machines, databases, storage accounts, networking configurations, etc. Azure offers a tool called Microsoft Defender for Cloud which has capability to continuously monitor and assess the security of Azure resources and generate alerts based on assessment, it also advice on how to fix the identified vulnerabilities. Figure 4.27 shows a working tab of Microsoft Defender.

Figure 4.27: Microsoft Defender for cloud

4.7 Post-configuration Testing

Purpose: This phase focuses on validating the effectiveness of the security configurations implemented during the “Configuration” phase. After the configuration stage, the aim is to confirm whether the security measures put in place actually work as intended. Testing procedures: Following configuration, a series of tests were undertaken to evaluate the security measures that had been installed. These tests focused on three main areas:

4.7.1 Network Security Group (NSG) functionality

Test: Confirmed that the NSG rules permit legal traffic while preventing illegitimate access. On the public IP address of the web server, a telnet test was run to confirm the Network Security Group’s (NSG) operation. Requests on port 80 were configured to be rejected by the NSG, but communication on port 443 was allowed. The preset settings are shown in the figure.

The domain name “kirticsoc.no,” which points to the public IP address of the Application Gateway, was then used to run telnet. As expected, port 443 was allowed for communication whereas port 80 was refused. It is important to note that the web server’s public IP address was only used to enable port 443 for demonstration purposes; normally, any traffic that does not originate from the public gateway is blocked. Figure 4.28 shows that the telnet test on port 80 was rejected while in figure 4.29 port 443 is open for communication.

Following figure shows the telnet test on the IP of the server.

```
└─$ telnet 51.120.241.52 80
Trying 51.120.241.52 ...
telnet: Unable to connect to remote host: Connection refused
```

Figure 4.28: Port 80 is rejected in telnet scan

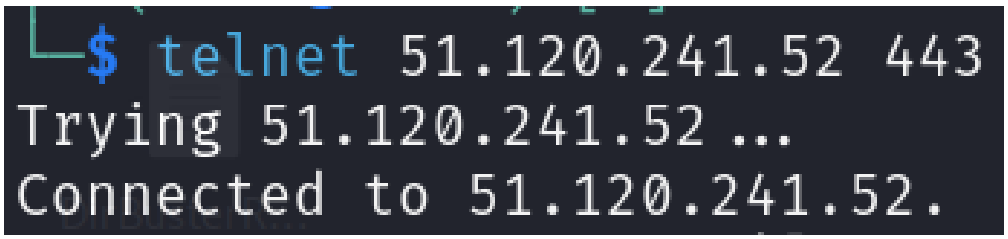

Figure 4.29: Port 443 is open for communication by NSG

4.7.2 Website Accessibility

Test: Successfully accessed the website using the intended domain name. The expected behavior of the Web Application Firewall (WAF) was observed: legitimate user traffic accessed the website without difficulty, while fraudulent requests were prevented. The domain “kirticsoc.no” was successfully accessed, as shown in figure 4.30.

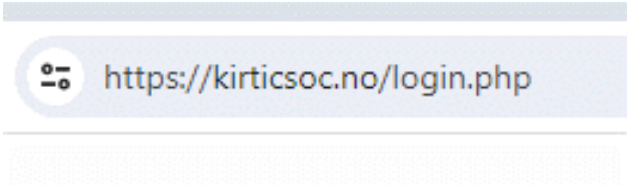

Figure 4.30: Website is accessible with domain name

4.7.3 Effectiveness of Additional Security Elements

The Azure WAF was configured and deployed to defend the web server against a variety of web-based threats. It serves as a reverse proxy, intercepting incoming requests and screening out harmful traffic before they reach the web server. The WAF has a number of security capabilities, including protection against SQL injection, cross-site scripting (XSS), and other typical web vulnerabilities. Various tests were executed to evaluate the efficiency of the WAF in preventing malicious attacks and Web Application Firewall (WAF) successfully defended against unauthorized access.

1. **Telnet Test:** Testing the Telnet Connection to the Web Server and initiating a Telnet Connection to the Web Server. The test can be seen in the Successfully accessed the website using the intended domain name. The expected behavior of the Web Application Firewall (WAF) was observed: legitimate user traffic accessed the website without difficulty, while fraudulent requests were prevented. The domain “kirticsoc.no” was successfully accessed, as shown in figure 4.31.

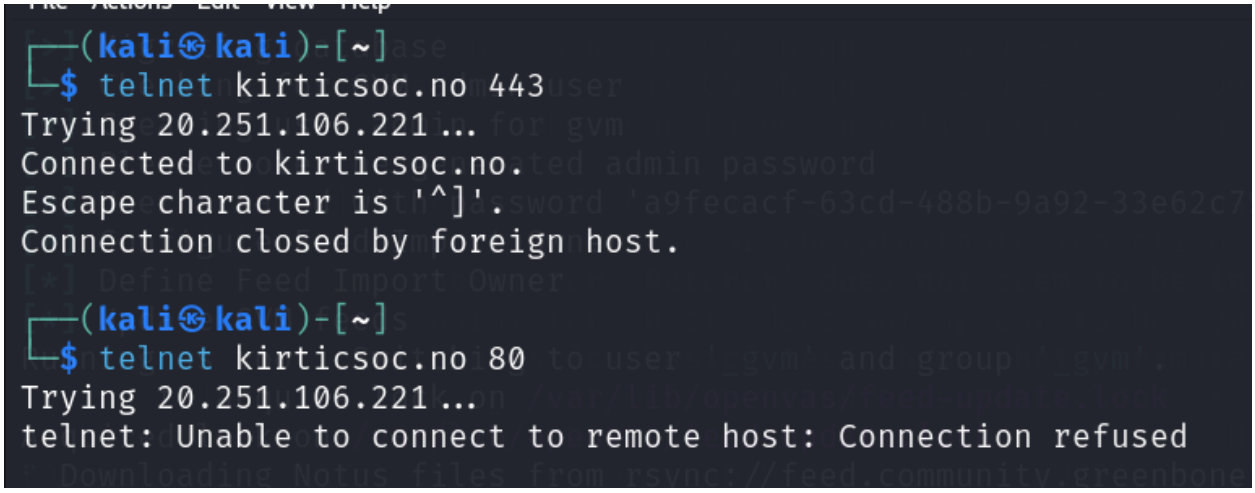

Figure 4.31: Result of Telnet scan

Result: The result shows that the WAF is actively protecting the web server on both HTTPS and HTTP. During the Telnet scan on port 443, which is commonly used for HTTPS traffic, the Telnet client successfully created an initial connection. Still, it can be observed that the initial connection does not instantly allow complete access. Since telnet is not the intended protocol for secure HTTPS communication, the WAF identified the Telnet connection as unauthorized and intervened, and immediately terminated the connection as a security measure. In this scenario, with the web server set to basic settings but the WAF potentially preventing telnet on port 80, refusing to establish a connection on port 80 (HTTP) indicates that the WAF is actively restricting telnet connections. Despite the web server's minimal configuration, the WAF's involvement demonstrates its function in imposing security controls. By prohibiting telnet access on port 80, the WAF improves the web server's overall security posture, protecting it from potential security vulnerabilities caused by illegal telnet connections.

2. **SQLmap scan:** To test the efficiency of WAF provided in preventing the web application vulnerabilities Sql scan was done on the web server.After a comprehensive SQLMap scan of the web application, no exploitable SQL injection vulnerabilities were found. During the scanning process, SQLMap thoroughly examined the application's parameters and endpoints for possible vulnerabilities. Despite employing a range of injection tactics, including boolean-based blind, error-based, time-based blind, and union-based injections, SQLMap discovered no parameters vulnerable to SQL injection attacks. The results are presented in the figure 4.32.

Figure 4.32: Result of SQl scan

The scan results indicate that none of the assessed parameters appeared to be injectable, and cautions were issued indicating that some settings did not seem to be

vulnerable. As an outcome, SQLMap's test confirms that the web application has no exploitable SQL injection vulnerabilities.

3. **Nikto Scan:** The purpose of this scan was to assess the security posture of the web server hosted at the Azure portal and determine the effectiveness of the Azure Web Application Firewall (WAF) in improving its security.

Figure 4.33: Nikto scan's output

Result: The Nikto scan in figure 4.33 identified multiple potential vulnerabilities and security concerns on the web server. These included missing security headers including X-Frame-Options, Strict-Transport-Security, and X-Content-Type-Options. Furthermore, cookie security concerns were discovered, with cookies produced without the secure flag, possibly exposing sensitive data to interception. It was expected that the scan could show list of some vulnerabilities as the server has a basic configuration, but during the scan, the Azure WAF exhibited its ability to mitigate potential vulnerabilities and protect the web server from exploitation. It intercepted and stopped malicious requests detected during the scan, preventing them from accessing the server's susceptible areas. In addition, the WAF addressed the security vulnerabilities raised by Nikto, such as missing security headers and cookie security concerns, by implementing suitable security policies and filtering rules.

4. **Commix scan:** To evaluate the effectiveness of the Web Application Firewall (WAF) in preventing command injection, the powerful tool "commix" was utilized. The scan involved testing the server's susceptibility to command injection by utilizing a variety of operating system commands as payloads. The following figure 4.34 shows the output of the scan.

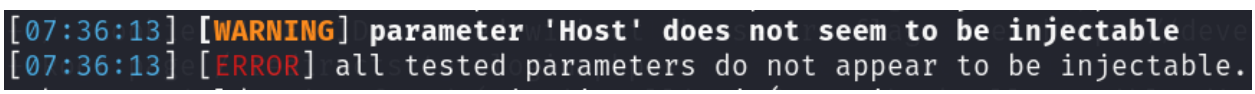

Figure 4.34: Commix scan output

When commix was run and injectable parameters were not detected, it is possible that the WAF prevented the attempted attacks. The WAF may have identified the commix payloads as malicious and blocked them from reaching the web server.

5. **Nessus scan:** The Nessus scan was employed to uncover potential vulnerabilities. However, as anticipated, it only detected the header vulnerability and some informational vulnerabilities. and did not identify any other critical vulnerabilities. Figure 4.35 displays the header misconfiguration and figure 4.36 shows that there were only informational level vulnerabilities in the scan result.

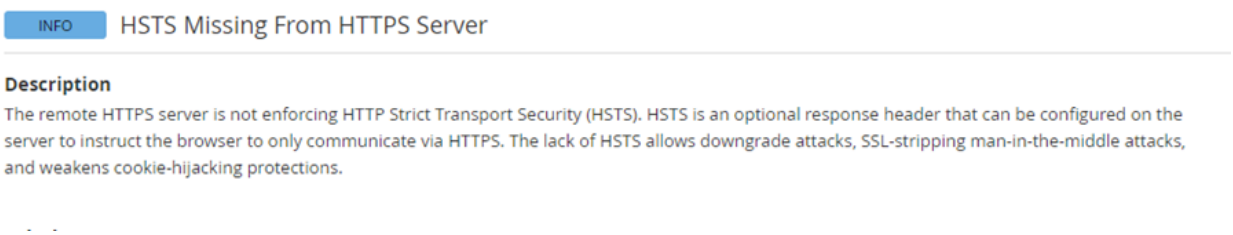

Figure 4.35: Nessus scan showing header misconfiguration

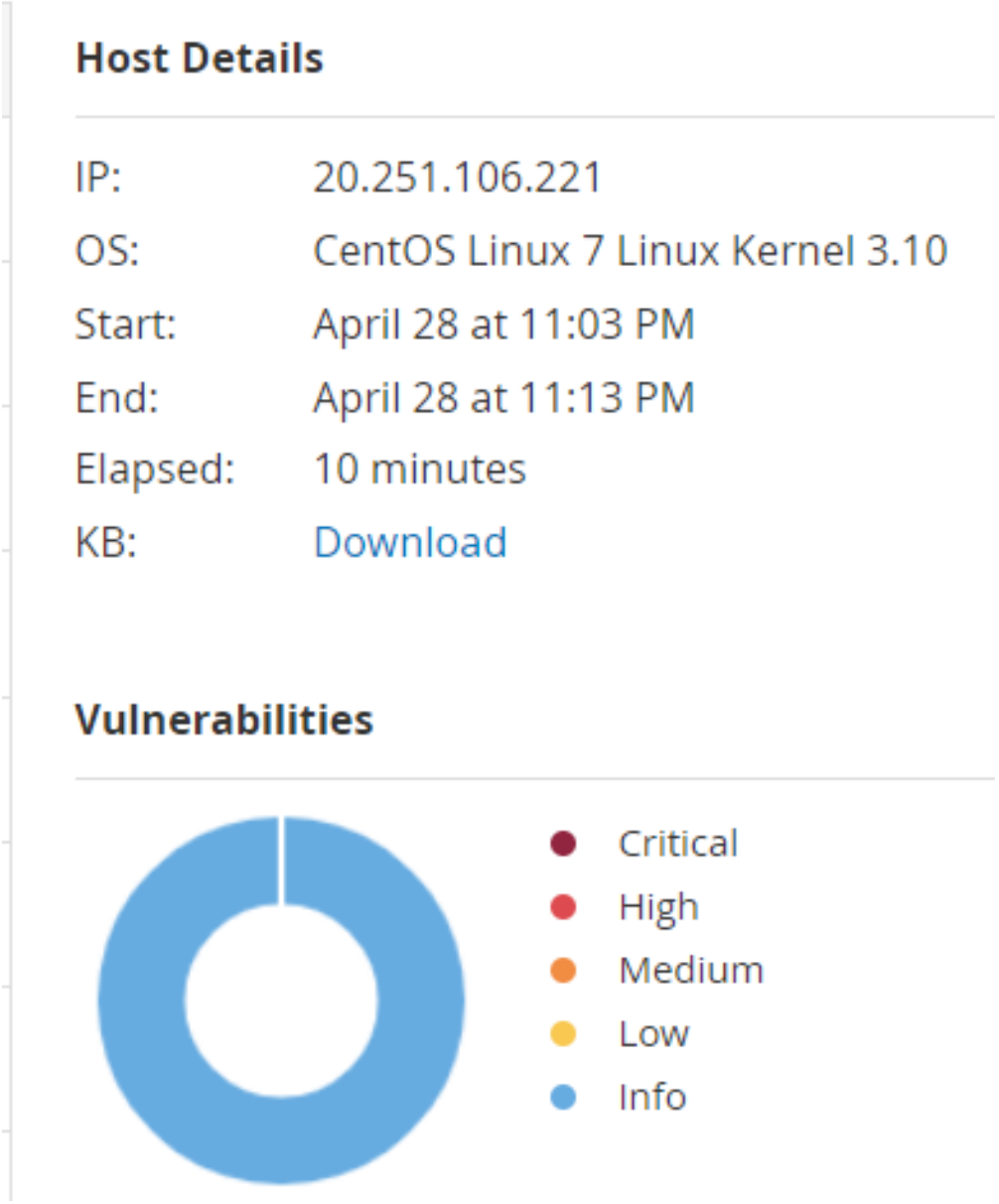

Figure 4.36: Nessus scan presenting only informational vulnerabilities

Manual testing of the server was done apart from utilizing the automated tools and various attacks were performed on the server.

1. **SQL injection:** A payload was injected in the input field of the server. The figure 4.37 shows the payload and there was no output apart from the login failed message. This was possible with the intervention of App Gateway to restrict any kind of attack on the website.

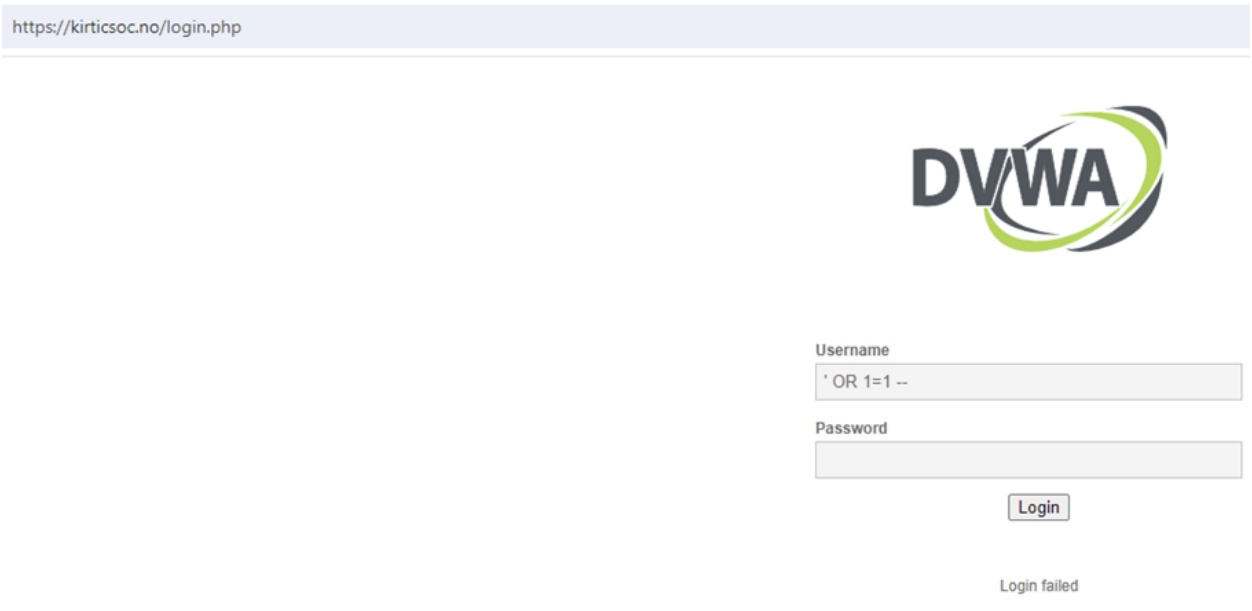

Figure 4.37: SQL injection payload

2. **Command Injection:** A variety of commands were injected in an attempt to evaluate the server's vulnerability to command injection; however, the App gateway intervened and effectively prevented them, preventing any effect on the website. By successfully blocking the execution of commands, the App gateway protected the server from potentially harmful activities. Figure 4.38 shows the payload with a login failed message.

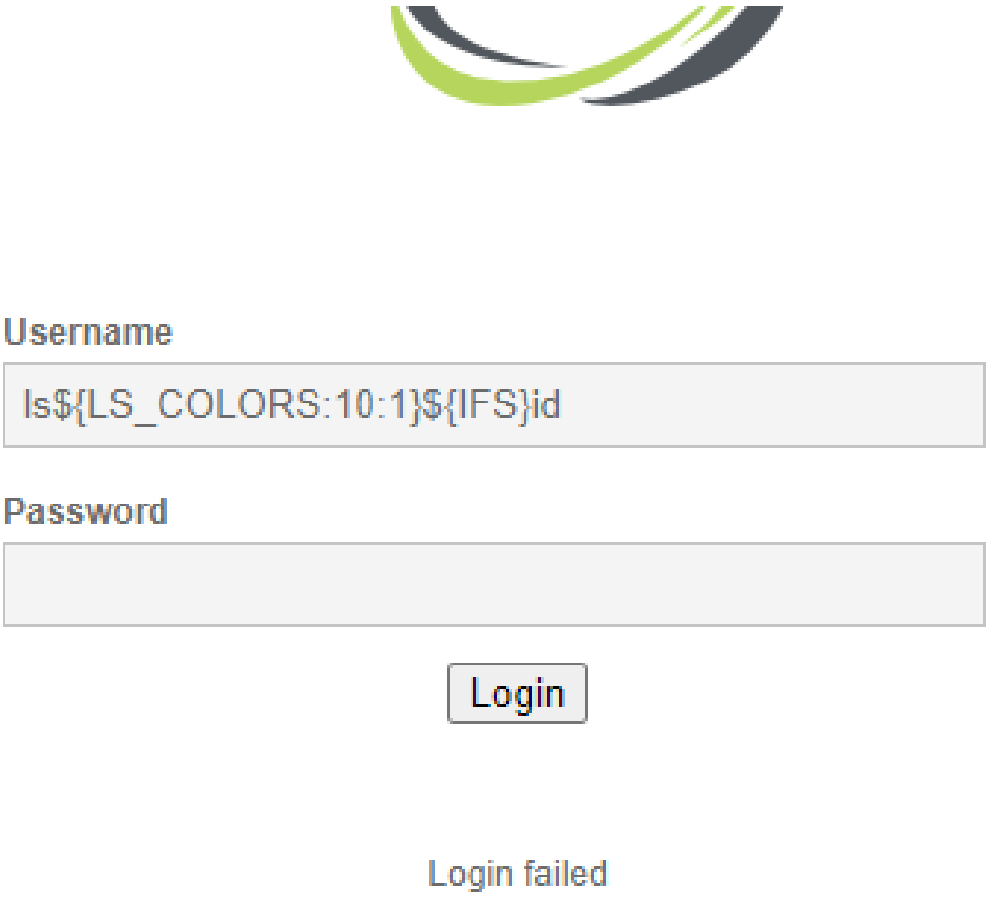

Figure 4.38: Command injection payload

3. **Cross-site scripting:** Several payloads were injected into the input field to assess the App gateway's efficiency in preventing cross-site scripting attacks on the site. As expected, the site responded with "login failed" once more, demonstrating that the App gateway successfully intercepted the malicious attempts. This can be seen in figure 4.39 as the output of cross-site scripting.

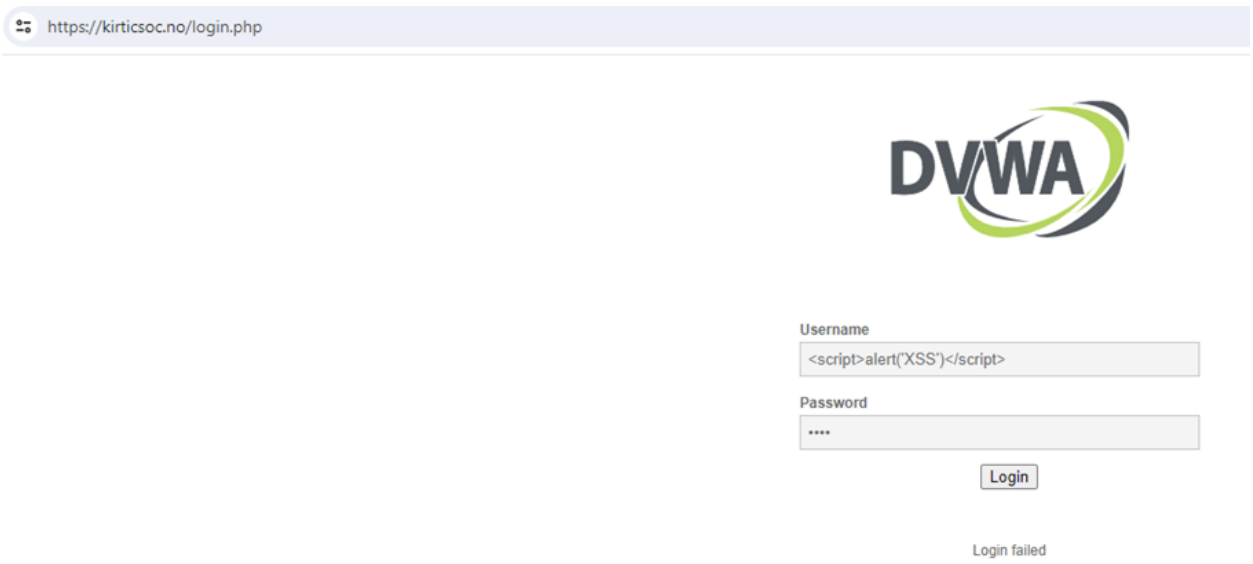

Figure 4.39: cross-site scripting payload with message login failed

4. **Directory traversal:** In order to assess the effectiveness of the App gateway in preventing unauthorized access to files and directories via directory traversal, a payload was injected into the URL for testing purposes. Figure 4.40 shows the payload and the output after testing.

Figure 4.40: Directory traversal output

4.8 Analysis

The purpose of the post-configuration testing step was to confirm that the security configurations put in place during the “Configuration” phase were effective. The security measures were assessed to make sure they worked as intended through a series of tests centered on three primary areas. The Network Security Group (NSG) test verified that it can allow genuine traffic while blocking unauthorized access. The NSG’s ability to control traffic was proven by Telnet tests on the designated domain name and the public IP address of the web server; port configurations matched expected behavior. Additionally, the Web Application Firewall’s (WAF) expected behavior was validated by the website’s successful access using the specified domain name, ensuring that legitimate user traffic could reach the site while fraudulent requests were blocked. Various thorough scans and manual testing were carried out in addition to NSG and WAF tests to assess the effectiveness of extra security components, such as defense against SQL injection, command injection, cross-site scripting, and directory traversal threats. The outcomes validated the strength of the security protocols put in place, with the App Gateway interfering to stop unwanted access and safeguard against different kinds of attacks, with the WAF successfully guarding against malicious payloads and unauthorized access.It is possible to fix header misconfiguration by implementing policies and paying close attention to details. Organizations can effectively address header misconfigurations by putting in place particular policies that regulate header configurations and making sure that these policies are consistently followed throughout the pertinent systems and components. This calls for a targeted approach to establishing, carrying out, and enforcing of policies in addition to continuous maintenance and monitoring to stop misconfigurations in the future. To provide RBAC, proper policies and sanctioning of the roles should be done. In the post configuration testing it was found that various roles should be assigned according the needs and to simplify the management, further it should follow the rule of least privileges. The compliance area needs to assign policies and this should be done carefully and keeping in the mind the requirement of the management, project and accesses. Wi

4.9 Summary

This chapter offers a pragmatic approach to addressing web application security within the Azure environment, moving beyond theoretical concepts that were discussed in Chapter 2 (Chapter 2) and bridges the gap between theory and practice in AzureWeb Application security. Through the deployment and testing of Damn Vulnerable Web Application (DVWA), complex security principles are broken down into actionable steps. Pre-testing serves to identify vulnerabilities, while post-configuration testing validates the of implemented security measures such as Network Security Groups (NSGs) and Web Application Firewalls (WAFs), ensuring their ability to thwart unauthorized access and fraudulent requests. Furthermore, thorough scans and manual testing validate the effectiveness of supplementary security features against a spectrum of potential attacks.

Role-Based Access Control (RBAC) is emphasized during the chapter, which extends beyond technical specifics. RBAC makes ensuring that policies are assigned correctly for compliance, matching project requirements and management demands with strong security safeguards. All things considered, this chapter offers a useful and thorough method for bolstering web application security on the Azure platform.

CHAPTER 5

Discussions

This chapter delves into the findings from Chapter 4 Implementation and Testing and the implications for real-world scenarios, while also identifying limitations and suggesting areas for further research.

This chapter delves and analyzes the outcomes of the security testing conducted in Chapter 4. It also explores how real-world web applications are affected by vulnerabilities that are found in DVWA and how proper security measures in Azure Portal protects them. The chapter also discusses about the limitations of the present-day structure and suggests the areas that need more research and for further improvement. The section 5.1 explains the vulnerabilities that were discovered in DVWA during testing such as Cross-Site Scripting and SQL injection as threats that are present in real world scenarios. Further it also highlights the necessity of security testing by including case studies from Chapter 2 that actually show the consequences of following proper security measures. In the section 5.2 the effectiveness of the security configurations that were implemented during deployment in Chapter 4. The successful implementation of Azure security tools such as Web Application Firewalls (WAF) and Network Security Group (NSG) illustrates their strong ability to mitigate such types of challenges. Section 5.3 elaborates system's resilience against the vulnerabilities and the ability of preventive measures in the defending aginst them. Section 5.4 acknowledges the limitations that are related and relevant with using Azure student and basic subscriptions. Section 5.5 sheds light on the areas for improvement as there is always room for improvement even after any successful security measures. It identifies potential areas where the improvement is needed which can further strengthen the security frameworks. Section 5.6 explains how web applications can be secured even with limited access to the resources and discusses about the access control policy. At the end section 5.7 provides a wide range of mitigation ways and details about the importance of Azure features.

5.1 Vulnerabilities in DVWA and real-life threats

The testing verified several vulnerabilities in the DVWA application, such as SQL injection (SQLi), cross-site scripting (XSS), Authorization Bypass, HTTP redirect, etc. Security researchers uncovered eight major cross-site scripting (XSS) problems in Azure HD Insight, a massive data processing service hosted on Azure. These weaknesses might have enabled attackers to insert and execute malicious scripts in users' browsers, posing serious threats to data integrity and privacy (Arghire, 2023). Although the issues were reported to Microsoft and remedied, their existence emphasizes the significance of proactive safety precautions and not considering the security of third-party services lightly (Constantin, 2023). A new vulnerability, known as "LeakyCLI," has been discovered in major cloud service providers' command line interfaces (CLIs), including Google Cloud (Gcloud CLI), AWS (AWSCLI), and Azure (Azure CLI). The vulnerability, CVE-2023-36052, has a severity rating of 8.6 (High), indicating that it compromises sensitive information via environment variables.. It's critical to understand that DVWA is purposely insecure for teaching purposes, emphasizing the importance of thorough vulnerability testing before implementing real-world web applications (Eswar, 2024). Relevance is demonstrated with case studies discussed in Section 2.11 (Chapter 2), such as the EmojiDeploy event, which is closely related to the vulnerability management topics covered in Chapter 2's section on case studies. It highlights how important it is to have a strong procedure in place for consistently locating, ranking, and fixing vulnerabilities. This incident serves as a reminder of the possible repercussions of disobeying security precautions, as attackers can take advantage of these weaknesses to obtain unapproved access and control. The findings of Cheah's research have a significant impact on the discussions around security monitoring, secure coding techniques, and access control. Organizations can actively evaluate their Azure deployments and spot any vulnerabilities by utilizing this list as a guide. Limiting public access to storage accounts and requiring multi-factor authentication (MFA) for privileged users, for instance, can greatly reduce the dangers related to compromised credentials. Furthermore, setting thorough log alerts in Azure Monitor offers insightful information about possible security events, facilitating quicker detection and reaction. The report discussed by Alspach in 2022 described a significant security breach which is shown in the Section 2.11 (Chapter 2) also shows that when seen from the shared responsibility lens, there is always a room for attack. Despite the architecture and security protections that Azure provides, it is likely that attackers took advantage of flaws or misconfigurations in the affected database. This event emphasizes how crucial it is for businesses to be in charge of protecting their own workloads and data on cloud platforms. To improve their overall security posture, organizations need to do more than just rely on Azure's security protections. They should also put in place other security controls including encryption, access control lists, and regular security assessments.

5.2 Effectiveness of Implemented Security Configurations

Several important insights regarding the effectiveness of security configurations established during the “Configuration” step can be determined by analyzing them. Tests concentrating on Web Application Firewalls (WAFs) and Network Security Groups (NSGs) in particular show a proactive approach to mitigating potential vulnerabilities. The NSG functioning test verifies that it can allow authorized traffic while blocking unauthorized access, in line with what is expected. In the same way, successful access to the website using the assigned domain name demonstrates the desired behavior of the WAF, which is to permit traffic from authorized users while thwarting fraudulent requests.

5.3 Withstanding known threats

The testing demonstrates the system’s resistance to known threats and includes scans for vulnerabilities related to directory traversal, SQL injection, command injection, and cross-site scripting. The findings of the SQLMap scan demonstrate that there are no exploitable SQL injection vulnerabilities, which suggests that strong prevention mechanisms are in place. Moreover, the efficiency of the WAF’s defense against malicious payloads and unauthorized access highlights how well it protects the web server from potential threats. It was also should be noted that the Azure firewall has three version and is recommended according to the needs. It is recommended that small and medium-sized businesses with throughput requirements of 250 Mbps use Azure Firewall Basic. For businesses seeking a Layer 3–Layer 7 firewall that requires autoscaling to manage periods of peak traffic of up to 30 Gbps, Azure Firewall Standard is advised. Enterprise features such web categories, DNS proxy, custom DNS, and threat intelligence are supported. For the protection of really sensitive applications (like payment processing), Azure Firewall Premium is advised. TLS inspection and malware are examples of sophisticated threat security features that it provides (Vhorne, 2024). ’

5.4 Challenges and Limitations with Azure student and basic Subscription

Microsoft’s Azure for Students program grants students free access to Azure cloud services and tools. Eligible students can sign up for an Azure for Students subscription and receive a 100 USD credit for 12 months. this credit allows students to explore Azure services like virtual machines, databases, and other tools. This helps students to get a valuable opportunity to explore cloud computing resources and get practical skills with the industry needs. There are various challenges with the Azure for Students program. One big problem is that students with more than one course or using many resources with a subscription may use all of the credit allotment within the twelve-month period. Moreover, students risk experiencing a shortfall in hands-on experience upon credit exhaus-

tion, potentially undermining their learning outcomes. In the insight of basic subscription of Azure, it also has many limitations that the basic subscriptions limit access to specific Azure services, potentially hindering the utilization of essential functionalities crucial for projects (Azure, 2024). It also has limitations on compute resources which can significantly impact the performance and scalability. Further, it has no or minimal technical support from Microsoft, so troubleshooting generally becomes a challenge. In the end, because of resource constraints and possible outages, minimal subscriptions are better suited for development and testing settings rather than production settings. When bigger organizations and businesses are looking for a future proof work market they should mind these limitations as for large organizations migrating a large scale cloud migration with basic subscription is not a foolproof and effective longterm solution. Limitations to certain resources hinder the security, efficiency and reliability of the businesses. For the positive continuity of the business, organizations should ensure to get access to the full range of Azure services, such as to achieve the required compute resources, and strong foolproof security framework. As the main goals of any organization is to get the work continuity, peak performance, and a strong foolproof security.

5.5 Areas for Improvement

There may be still room for improvement even while the security measures in place are successful in reducing known vulnerabilities. Examples of potential vulnerabilities that require attention include the Nikto scan's discovery of missing security headers and concerns about cookie security. The overall security posture of the system could be strengthened even more by addressing these problems with configuration improvements and policy changes. Besides Azure basic subscription limitations, secure measures are crucial. Frequent vulnerability scans and security assessments can identify and address emerging threats. Employee training ensures continued attention to details and system updates to patch the vulnerabilities. Despite this, when focused to a high budget for Azure cloud, it still has challenges like complexity in implementing and administrating a complex security stack consisting of various tools and services can be challenging. Misconceptions concerning cloud security stem from a misperception of the shared responsibility approach. While some businesses believe that Cloud Service Providers (CSPs) are completely responsible for data security after migration, the reality is that customers still have duties such as data protection, access management, and configuration settings. This misperception leads to either excessive dependence on CSPs or unfounded skepticism. Effective cooperation necessitates both sides' understanding and adaptation to CSP solutions and updates, which presents issues for businesses with limited IT resources (Marget, 2024).

5.6 Azure Best Practices in the free tier or low budget

It is important to secure web applications even with limited resources. Microsoft Azure provides a free tier for the Azure active directory(Azure Ad) currently known as Entra ID, as a basic service for IAM. This offers to implement basic access controls, like Role Based Access Control (RBAC) which restricts and limits the access to the resources that are based on the user roles. This facility helps to reduce the risk of unauthorized access. Further, the IAM with a low budget also involves implementing multi-factor authentication for an additional layer of security for logins. The free tier helps and shows students or users with a low budget can also leverage free tier Azure services and implement best security practices. Apart from the benefits in this free tier, Azure has some disadvantages also like it provides limited access to the services and security. Azure is further also susceptible to many potential threats like Access token abuse and leakage, lateral displacement from a compromised workload, compromised third-party partners with privileged access, Credential theft, Reconnaissance with search, Data capture using blob searching insider Threats with Authorized Permissions. Further security misconfiguration even from user or business ends can also lead towards improper access restrictions and networking regulations resulting in unintentional data disclosure on the internet, insufficient authentication methods, and inadequate encryption techniques for data in transit and at rest. Thanks to Microsoft Defender, plays a key role in preventing it from attacks and threats by providing security alerts which gets triggered in a various kind of scenarios like malicious data upload, compromised, misconfigured unusual authentication of tokens, and data ex-filtration (Dcurwin, 2023).

The discussion emphasizes the significance of secure coding techniques, testing for vulnerabilities, and layered security measures like HTTPS and WAF in addressing cybersecurity issues in Azure web apps. While acknowledging the experiment's shortcomings, it also proposes potential areas for future research to improve the security posture of cloud-based web services.

5.7 Mitigatating Web Application Vulnerabilities

Azure tools has been discussed in Chapter 2 and Chapter 4 to mitigate the web application vulnerabilities. If considered again, it can seen that Azure Firewall, uses signatures bases Intrusion detection and prevention system. It also provides threat intelligence bases fileteing which means it can enable real time alerts and restrict from /to traffic and malicious IP and domains (De Canaga, 2023). The web application security framework provides all-around defense against frequent flaws and assaults, such as cross-site scripting, SQL injection, and other online-based exploits. Features like programmable request size limitations, lists of exclusions for excluding particular request parameters, and the capacity to design unique rules based on application requirements are also included. SQL injection vulerabilties can also be mitigated by utilizing Azure SQL Databases's built in security took, like Advanced Threat Protection and transparent Data Encryption. Input

validation should also be used to mitigate injection vulnerabilities (Pietschmann, 2024). To mitigate the Broken Authentication can be prevented by the implementation of Azure Active Directory for proper identity and access management. Utilizing Multi Factor Authentication (MFA) critical accounts and roles can be saved. Security Misconfiguration can be prevented by including Azure security Center, which can identify and prevent the misconfigurations. To prevent insufficient logging and monitoring, Azure Monitor and Azure Security Center can provide a great and valauble overview in the security posture of the web application (Rboucher, 2024). The Web Application Gateway has a list of OWASP vulnerabilities that can be easily mitigated by deploying it with proper policies. By creating proper and legitimate policices the security posture can be enhanced which can provide a foolproof security.

5.8 Summary

In summary, this chapter explored the limits of basic and free tier of Azure subscriptions for organizations moving towards the cloud. Even though Azure gives these invaluable practical insight of experiencing cloud, it is restricted to limited resources and functionality which prevents the small businesses and learners being from fully learners. Organizations should understand their business needs and plan the strategy according to them to access to a greater selection of the resources and services. In order to achieve business continuity, peak performance, and strong security towards their cloud-based operations, this is essential.

CHAPTER 6

Conclusion

In conclusion, this research explored the critical factors for securing web application deployments in the Azure cloud environment. The research goal was to find the cybersecurity challenges in the web applications that are deployed on Microsoft Azure. The research found inherent weaknesses in cloud settings, underlining the need for enterprises to prioritize secure coding methods, strong access controls, and regular monitoring, despite Azure's security measures. The suggested security plan addresses a variety of issues, including access control, encryption, network security, vulnerability management, and cloud-specific procedures, to effectively mitigate these threats (Chkadmin, 2022).

Considering the improvements brought about by HTTPS and WAF, post-configuration testing indicated the persistence of previously disclosed vulnerabilities in DVWA, implying that the security measures used may not entirely address the identified problems. Furthermore, the efficiency of the App Gateway and WAF in mitigating real-world threats is uncertain due to the experiment's limited scope. To close these gaps, specialized testing centered on the functionality of these security measures is required. The proposed security plan offers a comprehensive approach to mitigating various threats, including access control, encryption, network security, vulnerability management, and cloud-specific procedures. The research's testing environment using DVWA has limitations. DVWA is a pre-configured vulnerable application designed for educational purposes and might not reflect the full spectrum of vulnerabilities present in a real-world, complex web application (Shinde & Waghere, 2017). Additionally, the testing likely focused on identifying basic vulnerabilities. Although the testing was done on the web applications but real-world penetration testing often involves a wider range of tools and techniques to discover more sophisticated vulnerabilities specific to the application under test.

The research did, however, identify drawbacks, such as the limited scope of DVWA testing, which necessitated additional security measure evaluations. Notably, the persistence

of vulnerabilities after deployment underscored the need for continued awareness and repair efforts. Future work could include improving the security framework, doing focused testing of the App Gateway and WAF in real-life scenario, and investigating advanced threat detection methodologies. Continuous monitoring, security awareness training, and compliance assessments will strengthen the security posture. Finally, adopting a preventive and structured security approach, rather than relying just on cloud provider features, is critical. This approach ensures a safe and secure environment for Azure-based web apps, which is consistent with the concept of shared responsibility in cloud security.

References

Aarness, A. (2023). Retrieved February 9, 2024, from https://www.crowdstrike.com/cybersecurity-101/endpoint-security/endpoint-detection-and-response-edr/

Abdullayeva, F. (2023). Cyber resilience and cyber security issues of intelligent cloud computing systems. *Results in Control and Optimization*, *12*, 100268. https://doi.org/10.1016/j.rico.2023.100268

Academy, E. (2023, August). What is the damn vulnerable web application (dvwa) and why is it recommended for practicing web application security testing? https://eitca.org/cybersecurity/eitc-is-wapt-web-applications-penetration-testing/spidering/spidering-and-dvwa/examination-review-spidering-and-dvwa/what-is-the-damn-vulnerable-web-application-dvwa-and-why-is-it-recommended-for-practicing-web-application-security-testing/

Agarwal, U. A. (2021, August). What is azure firewall? introduction and importance. Retrieved February 9, 2024, from https://k21academy.com/microsoft-azure/az-500/azure-firewall/

Ahmad, I. (2017). Cloud computing - a comprehensive definiton. *Journal of Computing and Management Studies*, *1*.

Ahmad, S., Mehfuz, S., & Beg, J. (2022). Assessment on potential security threats and introducing novel data security model in cloud environment. *Materials Today: Proceedings*, *62*, 4909–4915. https://doi.org/https://doi.org/10.1016/j.matpr.2022.03.536

Alazmi, S., & De Leon, D. C. (2022). A systematic literature review on the characteristics and effectiveness of web application vulnerability scanners. *IEEE Access*, *10*, 33200–33219. https://doi.org/10.1109/ACCESS.2022.3161522

Alhomdy, S., Thabit, F., Abdulrazzak, F. H., Haldorai, A., & Jagtap, S. (2021). The role of cloud computing technology: A savior to fight the lockdown in covid 19 crisis, the benefits, characteristics and applications. *International Journal of Intelligent Networks*, *2*, 166–174. https://doi.org/10.1016/j.ijin.2021.08.001

Almubaddel, M., & Elmogy, A. M. (2016). Cloud computing antecedents, challenges, and directions. *Proceedings of the International Conference on Internet of Things and Cloud Computing*, 1–5. https://doi.org/10.1145/2896387.2896401

Al-Sayyed, R. M. H., Hijawi, W. A., Bashiti, A. M., AlJarah, I., Obeid, N., & Al-Adwan, O. Y. A. (2019). An investigation of microsoft azure and amazon web services from users' perspectives. *International Journal of Emerging Technologies in Learning (iJET)*, *14*(10), 217. https://doi.org/10.3991/ijet.v14i10.9902

Alsolami, E. (2018). Security threats and legal issues related to cloud based solutions. *IJCSNS International Journal Of Computer Science And Network Security*, *18*(5).

Alspach, K. (2022). Microsoft azure vulnerabilities pose new cloud security risk - protocol. Retrieved February 9, 2024, from https://www.protocol.com/enterprise/microsoft-azure-vulnerabilities-cloud-security

Anaparthy, S. (2023). Step-by-step guide: Creating a virtual machine in azure from scratch. *Medium*. Retrieved February 9, 2024, from https://medium.com/@srijaanaparthy/step-by-step-guide-creating-a-virtual-machine-in-azure-from-scratch-fbacccc57635

Arghire, I. (2023, September). Azure hdinsight vulnerabilities: Data access, session hijacking, payload delivery. https://www.securityweek.com/azure-hdinsight-flaws-allowed-data-access-session-hijacking-payload-delivery/

Azure, M. (2024). Create your azure free account today. https://azure.microsoft.com/en-us/free/search

BasuMallick, C. (2022). Retrieved February 9, 2024, from https://www.spiceworks.com/it-security/vulnerability-management/articles/owasp-top-ten-vulnerabilities/

Belal, M. M., & Sundaram, D. M. (2022). Comprehensive review on intelligent security defences in cloud: Taxonomy, security issues, ml/dl techniques, challenges and future trends. *Journal of King Saud University-Computer and Information Sciences*, *34*(10), 9102–9131.

Benali, A., & Asri, B. E. (2021). Towards rigorous selection and configuration of cloud services: Research methodology. *arXiv preprint arXiv:2104.00819*.

BigCommerce. (2021). *Saas vs. paas vs. iaas: Examples and how to tell them apart*. Retrieved January 29, 2024, from https://www.bigcommerce.com/articles/ecommerce/saas-vs-paas-vs-iaas/

Brook, C. (2023). What is azure security? Retrieved February 9, 2024, from https://www.digitalguardian.com/blog/what-azure-security

Burgers, W., Verdult, R., & Van Eekelen, M. (2013). Prevent session hijacking by binding the session to the cryptographic network credentials. *Secure IT Systems: 18th Nordic Conference, NordSec 2013, Ilulissat, Greenland, October 18-21, 2013, Proceedings 18*, 33–50.

Butt, U., Mehmood, M., Shah, S., Amin, R., Waqas Shaukat, M., Raza, S., Suh, D., & Piran, M. (2020). A review of machine learning algorithms for cloud computing security [Publisher Copyright: © 2020 by the authors. Licensee MDPI, Basel, Switzerland.]. *Electronics (Switzerland)*, *9*(9), 1–25. https://doi.org/10.3390/electronics9091379

Bydrec, I. (n.d.). What is php used for and it's importance to web development. https://blog.bydrec.com/what-is-php-used-for#:~:text=You%20can%20use%20PHP%20to,dynamic%20website%2C%20including%20eCommerce%20sites.

Caraballo, J. (2024). Strengthening security: 5 lessons to learn from microsoft's recent azure and exchange server challenges. https://www.imprivata.com/uk/node/105432

Chang, V., Walters, R. J., & Wills, G. B. (2016). Cloud computing and frameworks for organisational cloud adoption. In *Web-based services: Concepts, methodologies, tools, and applications* (pp. 683–707). IGI Global.

Cheah, E. S. (2020, November). 20 common security vulnerabilities and misconfiguration in azure. Retrieved February 9, 2024, from https://engsooncheah.medium.com/20-common-security-vulnerabilities-and-misconfiguration-in-azure-5c19dd9289c2

Chkadmin. (2022, May). Microsoft azure security best practices.

Ciupa, I., Meyer, B., Oriol, M., & Pretschner, A. (2008). Finding faults: Manual testing vs. random+ testing vs. user reports. *2008 19th International Symposium on Software Reliability Engineering (ISSRE)*, 157–166. https://doi.org/10.1109/ISSRE.2008.18

Clements, J. (2023, January). *Owasp top 10 - 10 server-side request forgery*. Retrieved February 9, 2024, from https://foresite.com/blog/owasp-top-10-10-server-side-request-forgery/

Cloudflare. (2024). Owasp top ten. Retrieved February 9, 2024, from https://www.cloudflare.com/learning/security/threats/owasp-top-10/

Cobbins, R. (2022, September). An introduction to azure cloud security tools. Retrieved February 9, 2024, from https://sonraisecurity.com/blog/an-introduction-to-azure-cloud-security-features/

A comprehensive guide to azure security tools and features. (2024). Retrieved February 9, 2024, from https://www.sentra.io/blog/azure-security-tools

Constantin, L. (2023). Severe Azure HDInsight Flaws Highlight Dangers of Cross-Site Scripting. *CSO Online*. https://www.csoonline.com/article/652273/severe-azure-hdinsight-flaws-highlight-dangers-of-cross-site-scripting.html

Copeland, M., Soh, J., Puca, A., Manning, M., & Gollob, D. (2015, January). Microsoft azure and cloud computing. https://doi.org/10.1007/978-1-4842-1043-7_1

Crespo, I., Campazas, A., Guerrero-Higueras, Á. M., Del Castillo, V. R., Álvarez-Aparicio, C., & Fernández-Llamas, C. (2023). Sql injection attack detection in network flow data. *Computers and Security*, *127*, 103093. https://doi.org/10.1016/j.cose.2023.103093

(CSA), C. S. A. (2022, June). 1 threat to cloud computing: Iam | insufficient identity, credential access, and key management. Retrieved February 9, 2024, from https://cloudsecurityalliance.org/blog/2022/06/25/1-threat-to-cloud-computing-insufficient-identity-credential-access-and-key-management

Dahan, A. (2022, January). Enabling zero trust with azure network security services. Retrieved February 9, 2024, from https://azure.microsoft.com/en-us/blog/enabling-zero-trust-with-azure-network-security-services/

Davies, V. (2021, November). The history of cloud computing. Retrieved February 9, 2024, from https://cybermagazine.com/cloud-security/history-cloud-computing

Dcurwin. (2023, June). List of security threats and security alerts - microsoft defender for cloud. https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-storage-threats-alerts

De Canaga, F. (2023, September). Web application firewall, an essential element of web security. https://www.ubikasec.com/en/posts/web-application-firewall-waf-an-essential-element-of-web-security/

Deepa, G., & Thilagam, P. S. (2016). Securing web applications from injection and logic vulnerabilities: Approaches and challenges. *Information and Software Technology*, *74*, 160–180. https://doi.org/10.1016/j.infsof.2016.02.005

Diaby, T., & Rad, B. B. (2017). Cloud computing: A review of the concepts and deployment models. *International Journal of Information Technology and Computer Science*, *9*(6), 50–58. https://doi.org/10.5815/ijitcs.2017.06.07

Eswar. (2024). Leakycli: New vulnerability exposes credentials in aws, azure and google cloud. *Cyber Security News*. https://cybersecuritynews.com/leakycli-vulnerability-cloud-credentials/

Ferda, N. (2023, December). Top 11 challenges to cloud security. Retrieved February 9, 2024, from https://clarusway.com/top-11-cloud-security-challenges/

Fiser, D., & Olivera, A. (2023, March). Exploring Potential Security Challenges in Microsoft Azure. Retrieved September 21, 2023, from https://www.trendmicro.com/vinfo/us/security/news/cybercrime-and-digital-threats/exploring-potential-security-challenges-in-microsoft-azure

Foote, K. D. (2021, December). A brief history of cloud computing. https://www.dataversity.net/brief-history-cloud-computing/

Foundation, O. (n.d.). *Command injection*. Retrieved February 9, 2024, from https://owasp.org/www-community/attacks/Command%5C_Injection

George, A. S., & Sagayarajan, S. (2023). Securing cloud application infrastructure: Understanding the penetration testing challenges of iaas, paas, and saas environments. *Partners Universal International Research Journal*, *2*(1), 24–34. https://www.puirj.com/index.php/research/article/view/84/68

Goedtel, M. (2023, March). Cloud monitoring strategy - cloud adoption framework. Retrieved February 9, 2024, from https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/strategy/monitoring-strategy

Gong, C., Liu, J., Zhang, Q., Chen, H., & Gong, Z. (2010). The characteristics of cloud computing. *2010 39th International Conference on Parallel Processing Workshops*, 275–279. https://doi.org/10.1109/ICPPW.2010.45

Grimshaw, M. (2024, January). Governance, security, and compliance for azure arc-enabled servers. Retrieved February 9, 2024, from https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/hybrid/arc-enabled-servers/eslz-security-governance-and-compliance

Hashizume, K., Rosado, D. G., Fernández-Medina, E., & Fernández, E. B. (2013). An analysis of security issues for cloud computing. *Journal of Internet Services and Applications*, *4*(1), 5. https://doi.org/10.1186/1869-0238-4-5

Hassan, M. M., Nipa, S., Akter, M., Haque, R., Deepa, F., Rahman, M. M., Siddiqui, M. A., & Sharif, M. H. (2018). Broken authentication and session management vulnerability: A case study of web application. *International Journal of Simulation: Systems, Science and Technology*, *19*. https://doi.org/10.5013/IJSSST.a.19.02.06

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/doi.org/10.2307/25148625

Hoseini, S. A., Bouhafs, F., & den Hartog, F. (2022). A practical implementation of physical layer security in wireless networks. *2022 IEEE 19th Annual Consumer Communications and Networking Conference (CCNC)*, 1–4. https://doi.org/10.1109/CCNC49033.2022.9700672

Humayun, M., Niazi, M., Jhanjhi, N., Alshayeb, M., & Mahmood, S. (2020). Cyber security threats and vulnerabilities: A systematic mapping study. *Arabian Journal for Science and Engineering*, *45*(4), 3171–3189. https://doi.org/10.1007/s13369-019-04319-2

Hyun-Jin & Im. (2023). A comprehensive guide to microsoft security copilot. *ProServeIT Blog*. Retrieved February 9, 2024, from https://www.proserveit.com/blog/microsoft-security-copilot-comprehensive-guide

IBM. (2023). What are the benefits of cloud computing? | ibm. Retrieved February 9, 2024, from https://www.ibm.com/topics/cloud-computing-benefits

IG, S. (2023, August). Top 11 cloud security challenges in 2023. https://nioyatech.com/top-11-cloud-security-challenges/

Ipacs, D. (2023, April). *The history of cloud computing: Tracing its evolution and impact*. Retrieved April 29, 2024, from https://bluebirdinternational.com/history-of-cloud-computing/

Jabiyev, B., Mirzaei, O., Kharraz, A., & Kirda, E. (2021). Preventing server-side request forgery attacks. *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, 1626–1635. https://doi.org/10.1145/3412841.3442036

Jadeja, Y., & Modi, K. (2012). Cloud computing - concepts, architecture and challenges. *2012 International Conference on Computing, Electronics and Electrical Technologies (ICCEET)*, 877–880. https://doi.org/10.1109/ICCEET.2012.6203873

Johnson, J. (2019, November). Common web application vulnerabilities – authorization bypass. Retrieved April 29, 2024, from https://www.triaxiomsecurity.com/common-web-application-vulnerabilities-authorization-bypass/

Jones, D. N., Padilla, E., Curtis, S. R., & Kiekintveld, C. (2021). Network discovery and scanning strategies and the dark triad. *Computers in Human Behavior*, *122*, 106799. https://doi.org/10.1016/j.chb.2021.106799

Journey, S. (2024, January). Owasp top 10 server-side request forgery explained. Retrieved February 9, 2024, from https://www.securityjourney.com/post/owasp-top-10-server-side-request-forgery-explained

Katzan, H. (2011). On the privacy of cloud computing. *International Journal of Management and Information Systems (IJMIS)*, *14*. https://doi.org/10.19030/ijmis.v14i2.824

Kaur, M., & Singh, H. (2015). A review of cloud computing security issues. *International Journal of Advances in Engineering and Technology*, *8*(3), 397.

Kazim, M., & Zhu, S. Y. (2015). A survey on top security threats in cloud computing. *International Journal of Advanced Computer Science and Applications*, *6*(3), 109–113.

Kelleher, A. (2022, November). Understanding azure dns private resolver. Retrieved February 9, 2024, from https://medium.com/azure-architects/understanding-azure-dns-private-resolver-7f4045a3b2fa

Kelly, P. (2023, September). Understanding owasp top 10 v2021:a09: Security logging and monitoring failures — gratitech. Retrieved February 9, 2024, from https://www.gratitech.com/blog/understanding-owasp-top-10-v2021a09-security-logging-and-monitoring-failures

Kenner, L. (2022). The CSA's Pandemic 11. Retrieved February 9, 2024, from https://www.uptycs.com/blog/the-csas-pandemic-11-top-cloud-security-threats-and-what-to-do-about-them

Kingthorin, O. F. (2024). Blind sql injection. *OWASP*. Retrieved February 9, 2024, from https://owasp.org/www-community/attacks/Blind%5C_SQL%5C_Injection

Kiprin, B. (2021, April). Retrieved February 9, 2024, from https://crashtest-security.com/sql-injections/

Kukutla, T. R. (2023). A comprehensive guide to sql injection prevention.

Kumar, A., & Taterh, S. (2016). Analysis of various levels of penetration by sql injection technique through dvwa'. *Journal of Advanced Computing and Communication Technologies*, *4*(2), 28–32.

Kumar, S. N., & Vajpayee, A. (2016). A survey on secure cloud: Security and privacy in cloud computing. *American Journal of Systems and Software*, *4*(1), 14–26. https://doi.org/10.12691/ajss-4-1-2

Kushida, K. E., Murray, J., & Zysman, J. (2015). Cloud computing: From scarcity to abundance. *Journal of Industry, Competition and Trade*, *15*(1), 5–19. https://doi.org/10.1007/s10842-014-0188-y

Lanfear, T. (2023, April). Network security concepts and requirements in azure. Retrieved February 9, 2024, from https://learn.microsoft.com/en-us/azure/security/fundamentals/network-overview

Lawrence, C., Tuunanen, T., & Myers, M. D. (2010). Extending design science research methodology for a multicultural world. *Human Benefit through the Diffusion of Information Systems Design Science Research: IFIP WG 8.2/8.6 International Working Conference, Perth, Australia, March 30–April 1, 2010. Proceedings*, 108–121.

Lebeau, F., Legeard, B., Peureux, F., & Vernotte, A. (2013). Model-based vulnerability testing for web applications. *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation Workshops*, 445–452. https://doi.org/10.1109/ICSTW.2013.58

Li, Q., Li, W., Wang, J., & Cheng, M. (2019). A sql injection detection method based on adaptive deep forest. *IEEE Access*, *7*, 145385–145394. https://doi.org/10.1109/ACCESS.2019.2944951

Likaj, X., Khodayari, S., & Pellegrino, G. (2021). Where we stand (or fall): An analysis of csrf defenses in web frameworks. *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, 370–385. https://doi.org/10.1145/3471621.3471846

Linskens, A. (2023). What is owasp? *Sonatype Blog*. Retrieved February 9, 2024, from https://blog.sonatype.com/what-is-owasp

Loureiro, S. (2021). Security misconfigurations and how to prevent them. *Network Security*, *2021*(5), 13–16. https://doi.org/10.1016/S1353-4858(21)00053-2

Ltwagnon. (2023, June). Owasp mitigation strategies part 1: Injection attacks. https://community.f5.com/kb/technicalarticles/owasp-mitigation-strategies-part-1-injection-attacks/281381

Maayan, G. D. (2021, April). How the cloud has evolved over the past 10 years. https://www.dataversity.net/how-the-cloud-has-evolved-over-the-past-10-years/

Machado, G. S., Hausheer, D., & Stiller, B. (2009). Considerations on the interoperability of and between cloud computing standards. *27th open grid forum (OGF27), G2C-Net workshop: from grid to cloud networks*.

Marget, A. (2024, March). The shared responsibility model: Importance and best practices for cloud security. Retrieved April 25, 2024, from https://www.unitrends.com/blog/shared-responsibility-model

Marinos, A., & Briscoe, G. (2009). Community cloud computing. In M. G. Jaatun, G. Zhao, & C. Rong (Eds.), *Cloud computing* (pp. 472–484). Springer. https://doi.org/10.1007/978-3-642-10665-1_43

Marston, S., Li, Z., Bandyopadhyay, S., Zhang, J., & Ghalsasi, A. (2011). Cloud computing — the business perspective. *Decision Support Systems*, *51*(1), 176–189. https://doi.org/doi.org/10.1016/j.dss.2010.12.006

Matthews, A. (2024, January). Understanding the essentials of identity and access management (IAM) | Microsoft Entra Identity Platform. Retrieved February 9, 2024, from https://devblogs.microsoft.com/identity/iam-essentials/

Meier, J. (2006). Web application security engineering. *IEEE Security and Privacy*, *4*(4), 16–24. https://doi.org/10.1109/MSP.2006.109

Mell, P., & Grance, T. (n.d.). The nist definition of cloud computing.

Mello, J., John. (2022, July). 11 top cloud security threats. Retrieved February 9, 2024, from https://www.csoonline.com/article/555213/top-cloud-security-threats.html

Michael Roza, J. M. B., Gestin, A., & Hargrave, V. (2022, September). Retrieved February 9, 2024, from https://cloudsecurityalliance.org/blog/2022/09/17/top-threat-4-to-cloud-computing-lack-of-cloud-security-architecture-and-strategy

Microsoft. (2023a). *Microsoft azure security fundamentals overview*. Retrieved February 9, 2024, from https://learn.microsoft.com/en-us/azure/security/fundamentals/overview

Microsoft. (2023b, August). Azure firewall threat intelligence based filtering. https://learn.microsoft.com/en-us/azure/firewall/threat-intel

Microsoft. (2024, January). Server-side encryption of azure managed disks - azure virtual machines. https://learn.microsoft.com/en-us/azure/virtual-machines/disk-encryption

Milani, B. A., & Navimipour, N. J. (2016). A comprehensive review of the data replication techniques in the cloud environments: Major trends and future directions. *Journal of Network and Computer Applications*, *64*, 229–238. https://doi.org/https://doi.org/10.1016/j.jnca.2016.02.005

Miyachi, C. (2018). What is 'cloud'? it is time to update the nist definition? *IEEE Cloud Computing*, *5*(3), 6–11. https://doi.org/10.1109/MCC.2018.032591611

Mohammed, C. M., & Zeebaree, S. R. M. (2021). Sufficient comparison among cloud computing services: Iaas, paas, and saas: A review. https://doi.org/10.5281/ZENODO.4450129

Mullarkey, M. T., & Hevner, A. R. (2019). An elaborated action design research process model. *European Journal of Information Systems*, *28*(1), 6–20.

Nadeem, M. (2016). Cloud computing: Security issues and challenges. *Journal of Wireless Communications*, *1*, 10–15. https://doi.org/10.21174/jowc.v1i1.73

New Microsoft Azure vulnerability uncovered — EmojiDeploy for RCE attacks. (2023, January). Retrieved February 9, 2024, from https://thehackernews.com/2023/01/new-microsoft-azure-vulnerability.html

Nicho, M., & Hendy, M. (2013). Dimensions of security threats in cloud computing: A case study. *Review of Business Information Systems (RBIS)*, *17*(4), 159–170. https://doi.org/https://doi.org/10.19030/rbis.v17i4.8238

Nikto, C. (2024). Nikto. *Cybersecurity Infrastructure Security Agency (CISA)*. https://www.cisa.gov/resources-tools/services/

Odun-Ayo, I., Ananya, M., Agono, F., & Goddy-Worlu, R. (2018). Cloud computing architecture: A critical analysis. *2018 18th International Conference on Computational Science and Applications (ICCSA)*, 1–7. https://doi.org/10.1109/ICCSA.2018.8439638

OWASP. (2024). Identification and authentication failures.

OWASP Foundation. (2014). Owasp testing guide - v4.2. https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion

OWASP Foundation. (2024). OWASP Web Security Testing Guide - v4.2.

Parast, F. K., Sindhav, C., Nikam, S., Yekta, H. I., Kent, K. B., & Hakak, S. (2022). Cloud computing security: A survey of service-based models. *Computers and Security*, *114*, 102580. https://doi.org/doi.org/10.1016/j.cose.2021.102580

Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A design science research methodology for information systems research. *J. Manage. Inf. Syst.*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Pietschmann, C. (2024, April). Top 10 web application security risks in microsoft azure and ways to mitigate them. https://build5nines.com/top-10-web-application-security-risks-in-microsoft-azure-and-ways-to-mitigate-them/

PortSwigger. (2024). What is sql injection? tutorial and examples. *Web Security Academy*. Retrieved February 9, 2024, from https://portswigger.net/web-security/sql-injection

Poston, H. (2020). Mapping the owasp top ten to blockchain. *Procedia Computer Science*, *177*, 613–617.

Radoff, J. (2023, June). The cloud native game development canon. Retrieved February 9, 2024, from https://meditations.metavert.io/p/the-cloud-native-game-development

Rafique, S., Humayun, M., Hamid, B., Abbas, A., Akhtar, M., & Iqbal, K. (2015). Web application security vulnerabilities detection approaches: A systematic mapping study. *2015 IEEE/ACIS 16th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 1–6. https://doi.org/10.1109/SNPD.2015.7176244

Rashid, A., & Chaturvedi, A. (2019). Cloud computing characteristics and services: A brief review. *International Journal of Computer Sciences and Engineering*, *7*, 421–426. https://doi.org/10.26438/ijcse/v7i2.421426

Rboucher. (2024, February). Azure monitor overview - azure monitor. https://learn.microsoft.com/en-us/azure/azure-monitor/overview

Rodríguez, G. E., Torres, J. G., Flores, P., & Benavides, D. E. (2020). Cross-site scripting (xss) attacks and mitigation: A survey. *Computer Networks*, *166*, 106960. https://doi.org/https://doi.org/10.1016/j.comnet.2019.106960

Rosencrance, L. (2023, March). Cloud security alliance (csa). Retrieved February 9, 2024, from https://www.techtarget.com/searchsecurity/definition/Cloud-Security-Alliance-CSA

Sadqi, Y., & Maleh, Y. (2020). A systematic review and taxonomy of web applications threats. *Information Security Journal*, *31*(1), 1–27. https://doi.org/10.1080/19393555.2020.1853855

Security, S. (2022, June). What are the owasp top 10 vulnerabilities (and how to mitigate them)? Retrieved February 9, 2024, from https://cheapsslsecurity.com/blog/what-are-the-owasp-top-10-vulnerabilities-and-how-to-mitigate-them/

SecurityScorecard. (2021). 41 common web application vulnerabilities explained. Retrieved February 9, 2024, from https://securityscorecard.com/blog/common-web-application-vulnerabilities-explained/

Sengupta, S. (2022, November). Software and data integrity failures - examples and prevention. Retrieved February 9, 2024, from https://crashtest-security.com/owasp-software-data-integrity-failures/

Setting up damn vulnerable web app (dvwa) on ubuntu in azure. (n.d.). Retrieved February 9, 2024, from https://technet239.rssing.com/chan-4753999/article27209.html

Shahid, W. B., Aslam, B., Abbas, H., Afzal, H., & Khalid, S. B. (2022). A deep learning assisted personalized deception system for countering web application attacks. *Journal of Information Security and Applications*, *67*, 103169. https://doi.org/10.1016/j.jisa.2022.103169

Sharma, R., & Trivedi, R. (2014). Literature review: Cloud computing –security issues, solution and technologies. *International Journal of Engineering Research*, *3*, 221–225. https://doi.org/10.17950/ijer/v3s4/408

Shinde, G., & Waghere, S. S. (2017). Analysis of sql injection using dvwa tool. *RICE*, 107–110.

Singh, A., & Chatterjee, K. (2017). Cloud security issues and challenges. *J. Netw. Comput. Appl.*, *79*(100), 88–115. https://doi.org/10.1016/j.jnca.2016.11.027

Stanoevska-Slabeva, K., & Wozniak, T. (2010, January). Grid and cloud computing: A business perspective on technology and applications. In *Grid and cloud computing: A business perspective on technology and applications* (pp. 1–274). Springer. https://doi.org/10.1007/978-3-642-05193-7

Subramanian, N., & Jeyaraj, A. (2018). Recent security challenges in cloud computing. *Computers and Electrical Engineering*, *71*, 28–42.

Sulthan, N. N. (n.d.). Php allow_url_include is enabled. https://beaglesecurity.com/blog/vulnerability/allow-url-fopen-is-enabled.html

Sumo Logic, I. (2023, March). File inclusion - definition and overview. https://www.sumologic.com/glossary/file-inclusion/

Sunyaev, A. (2020). Cloud computing. In A. Sunyaev (Ed.), *Internet computing: Principles of distributed systems and emerging internet-based technologies* (pp. 195–236). Springer International Publishing. https://doi.org/10.1007/978-3-030-34957-8_7

Surbiryala, J., & Rong, C. (2019). Cloud computing: History and overview. *2019 IEEE Cloud Summit*, 1–7. https://doi.org/10.1109/CloudSummit47114.2019.00007

Susanto, H., Almunawar, M. N., & Kang, C. (2012). A review of cloud computing evolution individual and business perspective. *Available at SSRN 2161693*.

Tabrizchi, H., & Kuchaki Rafsanjani, M. (2020). A survey on security challenges in cloud computing: Issues, threats, and solutions. *J. Supercomput.*, *76*(12), 9493–9532. https://doi.org/10.1007/s11227-020-03213-1

Takabi, D., & Ghasemigol, M. (2019, February). Introduction to the cloud and fundamental security and privacy issues of the cloud. https://doi.org/10.1002/9781119053385.ch1

TerryLanfear. (2024, January). Azure security features that help with identity management. https://learn.microsoft.com/en-us/azure/security/fundamentals/identity-management-overview

Thornton, T. (2019, January). Azure network security groups: 10 suggestions for best practice! https://www.kainos.com/insights/blogs/azure-network-security-groups-10-suggestions-for-best-practice

Thu, M., Trang, P., & Minh, P. (2023). An introduction to cloud computing platform. *Engineering and Technology Journal*, *08*. https://doi.org/10.47191/etj/v8i5.05

Torkura, K. A., Sukmana, M. I. H., Cheng, F., & Meinel, C. (2021). Continuous auditing and threat detection in multi-cloud infrastructure. *Computers and Security*, *102*, 102124. https://doi.org/https://doi.org/10.1016/j.cose.2020.102124

Troscianecki, L. (2023, September). What is azure security? fundamentals and key concepts. Retrieved February 9, 2024, from https://www.veeam.com/blog/what-is-azure-security-fundamentals-key-concepts.html

Uzoma, B., & Okhuoya, B. (2022, December). *A research on cloud computing*.

Valezquez, A. (2022, September). Owasp top 10: 6 vulnerable and outdated components. Retrieved February 9, 2024, from https://foresite.com/blog/owasp-top-10-vulnerable-and-outdated-components/

Varun, K., Rajkumar, N., & Kumar, N. (2014). Survey on security threats in cloud computing. *Int. J. Appl. Eng. Res*, *9*, 10495–10500.

Vellela, S., Balamanigandan, A., & Professor. (2022). Strategic survey on security and privacy methods of cloud computing environment. *Journal of Next Generation Technology*, *2*(1), 2583–2604. https://jnxtgentech.com/mail/documents/Strategic%20Survey%20on%20Security%20and%20Privacy%20Methods%20of%20Cloud%20Computing%20Environment.pdf

Venable, J., Pries-Heje, J., & Baskerville, R. (2012a). A comprehensive framework for evaluation in design science research. In K. Peffers, M. Rothenberger, & B. Kuechler (Eds.), *Design science research in information systems. advances in theory and practice* (pp. 423–438). Springer Berlin Heidelberg.

Venable, J., Pries-Heje, J., & Baskerville, R. (2012b). A comprehensive framework for evaluation in design science research. *Design Science Research in Information Systems. Advances in Theory and Practice: 7th International Conference, DESRIST 2012, Las Vegas, NV, USA, May 14-15, 2012. Proceedings 7*, 423–438.

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). Feds: A framework for evaluation in design science research. *European journal of information systems*, *25*, 77–89.

Vhorne. (2024, April). Choose the right azure firewall version to meet your needs. https://learn.microsoft.com/en-us/azure/firewall/choose-firewall-sku

Vhorne & Coulter, D. (2023, August). Azure firewall threat intelligence based filtering.

Vordel, M. O. (2011, January). A security checklist for SaaS, PaaS and IaaS cloud models. https://www.csoonline.com/article/528248/saas-paas-and-iaas-a-security-checklist-for-cloud-models.html

Wang, S., Gong, Y., Chen, G., Sun, Q., & Yang, F. (2013). Service vulnerability scanning based on service-oriented architecture in web service environments. *Journal of Systems Architecture*, *59*(9), 731–739. https://doi.org/https://doi.org/10.1016/j.sysarc.2013.01.002

Wang, Y., & Yang, J. (2017). Ethical hacking and network defense: Choose your best network vulnerability scanning tool. *2017 31st International Conference on Advanced Information Networking and Applications Workshops (WAINA)*, 110–113. https://doi.org/10.1109/WAINA.2017.39

Weichselbaum, L., Spagnuolo, M., Lekies, S., & Janc, A. (2016). Csp is dead, long live csp! on the insecurity of whitelists and the future of content security policy. *Proceedings of the 23rd ACM Conference on Computer and Communications Security*. https://doi.org/10.1145/2976749.2978363

Wen, S.-F., & Katt, B. (2023). A quantitative security evaluation and analysis model for web applications based on owasp application security verification standard. *Computers and Security*, *135*. https://doi.org/10.1016/j.cose.2023.103532

What Is Microsoft Azure Cloud? (n.d.). Retrieved February 9, 2024, from https://www.perforce.com/blog/vcs/what-microsoft-azure-cloud

Wibowo, R. M., & Sulaksono, A. (2021). Web vulnerability through cross site scripting (xss) detection with owasp security shepherd. *Indonesian Journal of Information Systems*, *3*(2), 149–159. https://doi.org/10.24002/ijis.v3i2.4192

Yadwadkar, N. J., Hariharan, B., Gonzalez, J. E., Smith, B., & Katz, R. H. (2017). Selecting the best vm across multiple public clouds: A data-driven performance modeling approach. *Proceedings of the 2017 Symposium on Cloud Computing*, 452–465. https://doi.org/10.1145/3127479.3131614

Yeoh, W., Liu, M., Shore, M., & Jiang, F. (2023). Zero trust cybersecurity: Critical success factors and a maturity assessment framework. *Computers and Security*, *133*, 103412. https://doi.org/10.1016/j.cose.2023.103412

Zech, P., Felderer, M., & Breu, R. (2017). Knowledge-based security testing of web applications by logic programming. *International Journal on Software Tools for Technology Transfer*, *21*(2), 221–246. https://doi.org/10.1007/s10009-017-0472-3

Zhang, S., Zhang, X., & Ou, X. (2014). After we knew it: Empirical study and modeling of cost-effectiveness of exploiting prevalent known vulnerabilities across iaas cloud. *Proceedings of the 9th ACM symposium on Information, computer and communications security*, 317–328.

APPENDIX A

Deployment on Azure

This appendix details the deployment process followed for creating an instance and the installation of DVWA within the Azure clouds.

A.1 Login process:

The first step of the deployment process is to log in to the Azure portal. To access the Microsoft Azure site, enter https://portal.azure.com in a web browser. Input the login credentials and after successful login.

A.2 Resource Group Creation

To create a resource group in Azure, Go to the Azure services section and select the Resource Groups icon. Click on the Add and create the Resource groups shown in the figure A.1

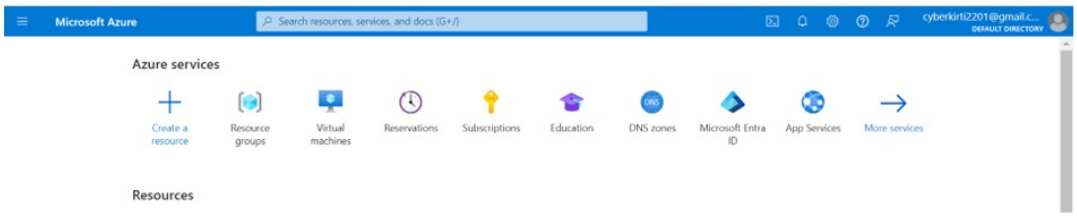

Figure A.1: Resource Group Creation

Provide and update the required information such as the name of the subscription, Resource Group, and Region. The following figures demonstrate setting up a resource group named “Bachelor Project DVWA Azure” within the “Bachelor Project” subscription located in the “(Europe) Norway East” region. Figure A.2 shows the creation of a resource group.

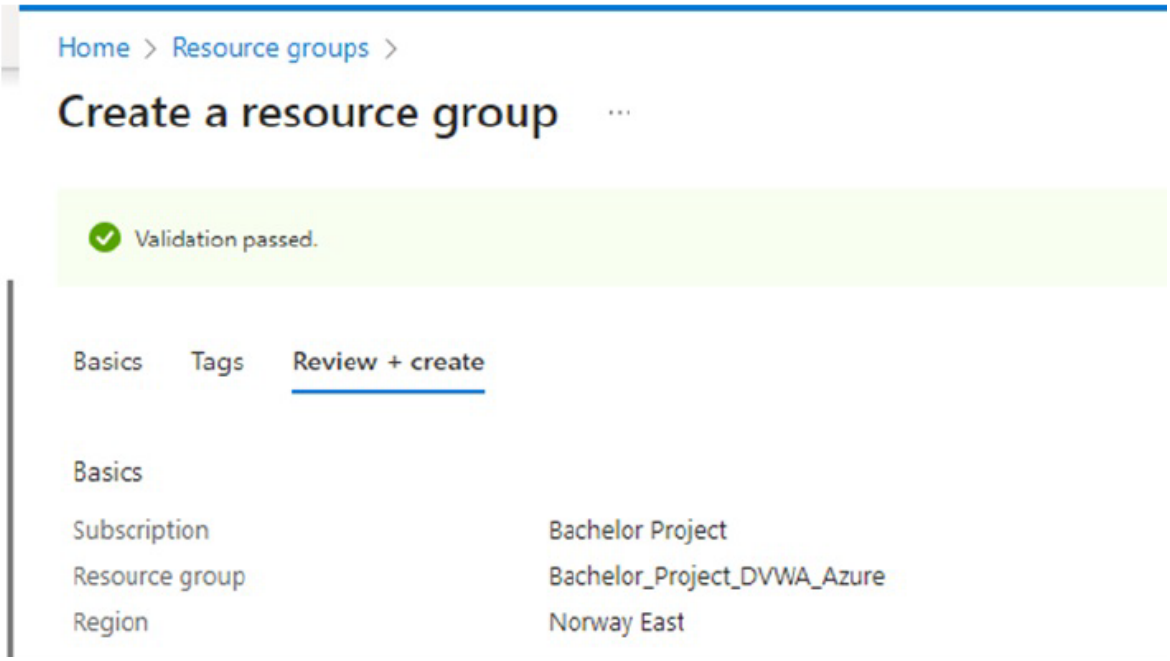

Figure A.2: Creating a resource Group

A.3 VM Creation

The third step in the deployment of DVWA is to create a VM and for that, Navigate to Virtual Machine and click the “Create” button as shown in figure A.3.

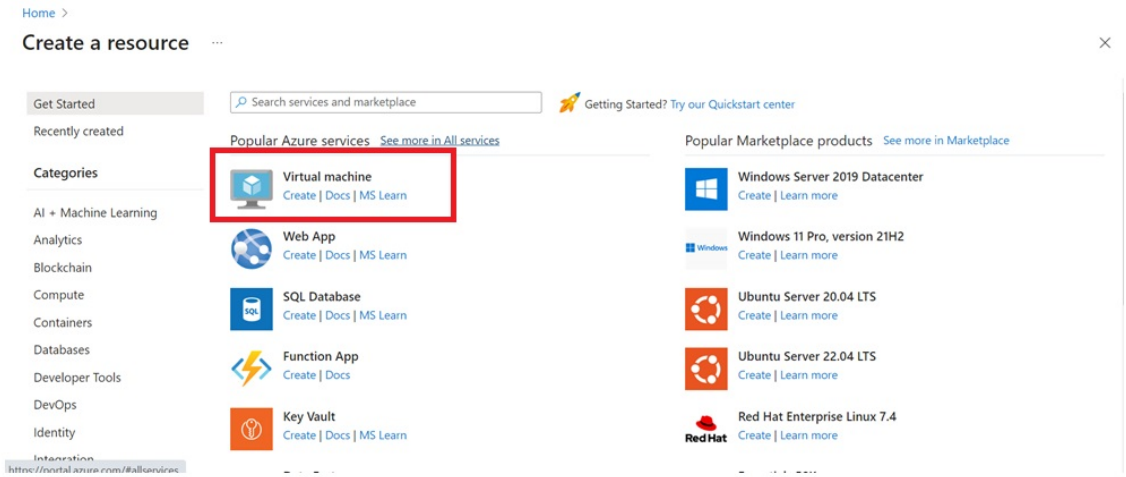

Figure A.3: Process of Creating a VM

Setting up the VM's structure is the first step in the process. First, the suitable subscription is chosen as "Bachelor Project". Next, an existing or newly formed resource group is selected which is "Bachelor Project DVWA Azure". The virtual machine is given the name as "DVWA-testing". The region or data center that is closest to the location is (Europe) Norway East. Options for availability are chosen according to preferences. Lastly, the virtual machine's operating system image is selected as Ubuntu Server 20.04 LTS – x64 Gen2. To configure a VM in Azure, it is important to take care of the requirements that fit the demands of the workload and requirements of performance, availability, and security. This is crucial as it influences the performance of parameters including runtime, latency, cost, security, and availability. These configurations are shown in figure A.4 Selecting the configuration that best aligns with the user's objectives, including price and performance goals, is essential (Yadwadkar et al., 2017).

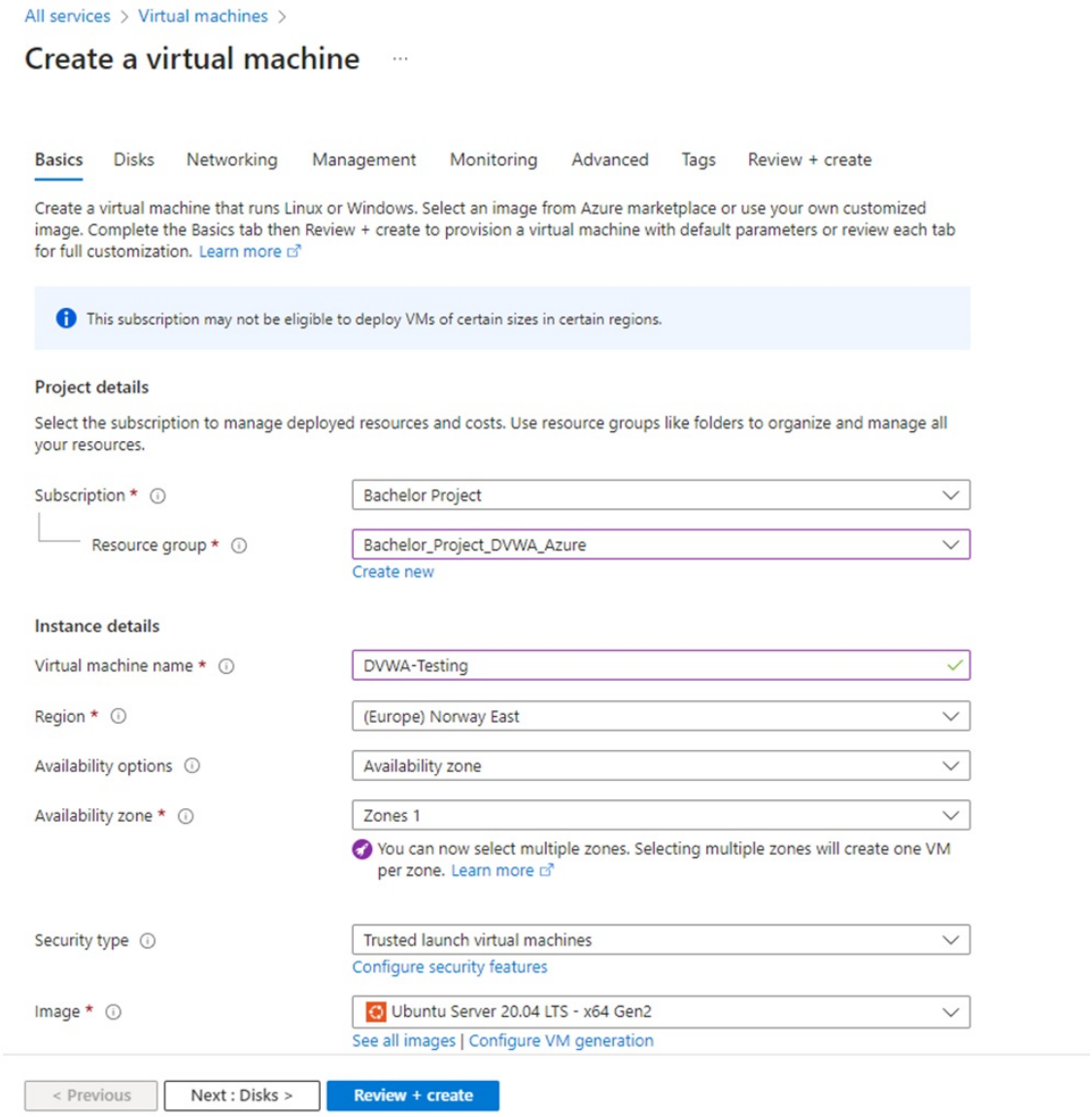

Figure A.4: Creating a VM

The next step in the VM process is setting up the authentication and security of the VM. When setting up authentication, provide the necessary credentials and choose between using a password or an SSH public key. After that, define public incoming ports and create inbound port rules to allow remote access over SSH or RDP as shown in figure A.5and guarantee connectivity from outside sources (Anaparthy, 2023).

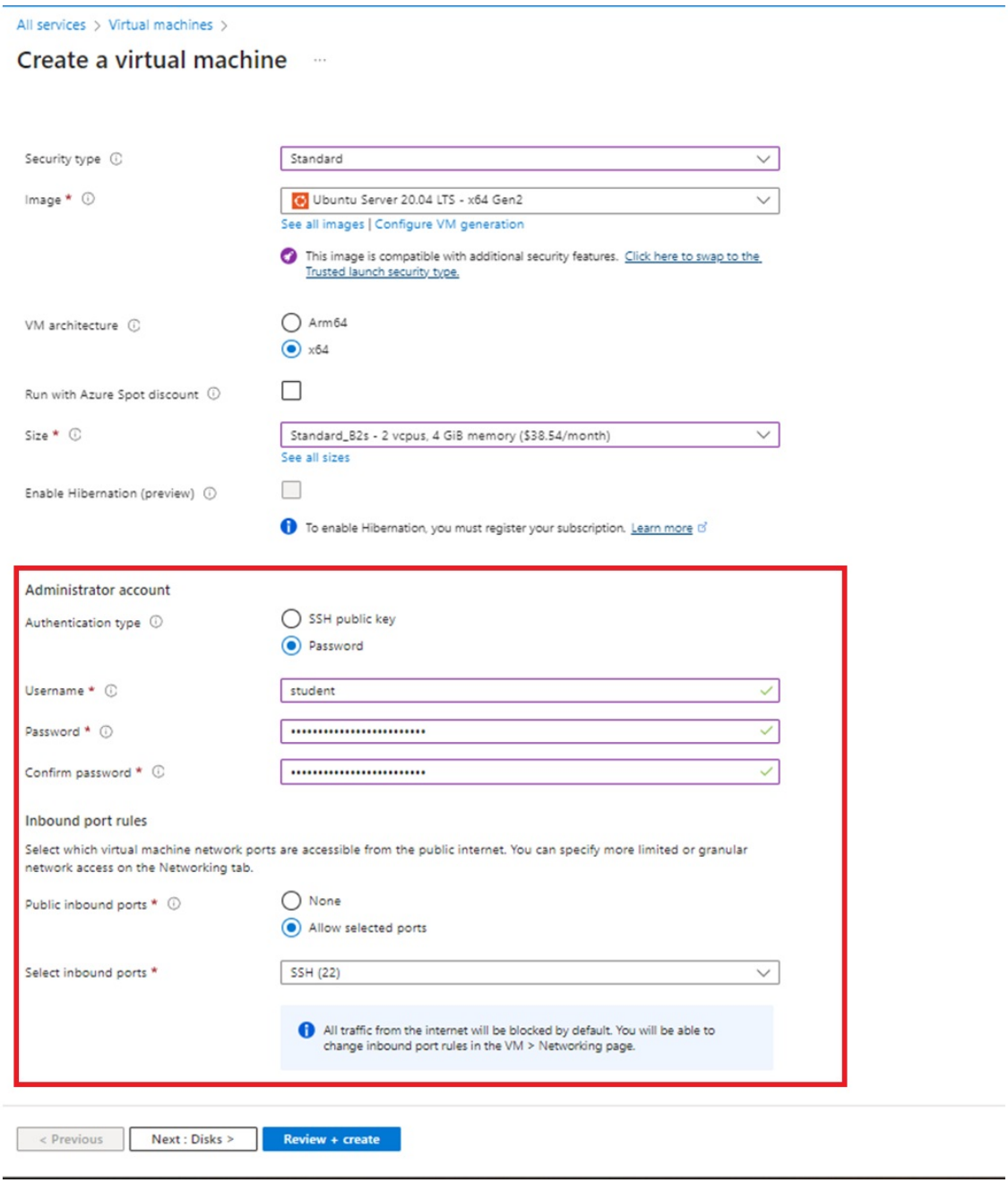

Figure A.5: Account details

After the authentication process, disks and storage need to be configured. The OS disk size is adjusted according to the needs of the project. Following the setup of disk and storage configurations, the next step involves configuring the network settings. Within the “Networking” section, choose an already existing virtual network and subnet or create new one as per the need. If required assign a public IP address and create network security groups to efficiently control the traffic flow. The networking In the figure below, the virtual network created is “ DVWA-Testing-vnet” and a new subnet named “default” with an address range of 10.0.0.0/24 is being created within the virtual network. This subnet helps segment the virtual network and organize IP addresses as well as improves the network performance and speed. Figure A.6 shows the network configurations of the VM and figure A.7 shows that a VM is created successfully.

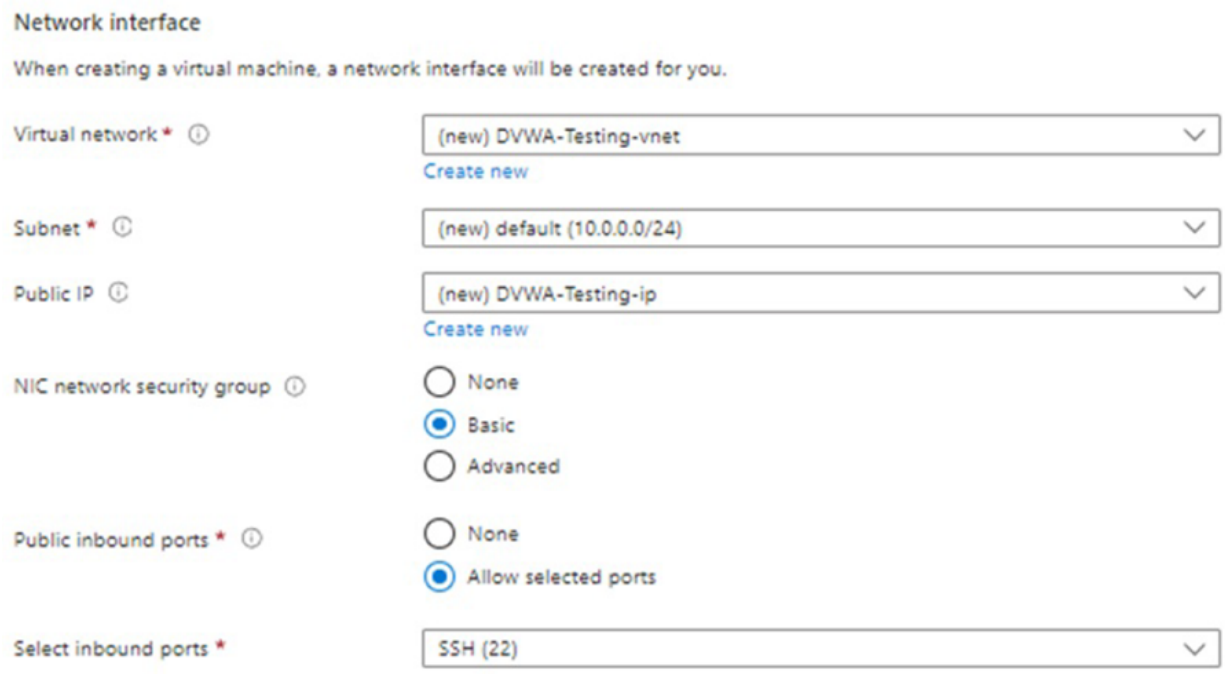

Figure A.6: Network Configurations

The following figure A.8 shows the successful validation and deployment of the VM.

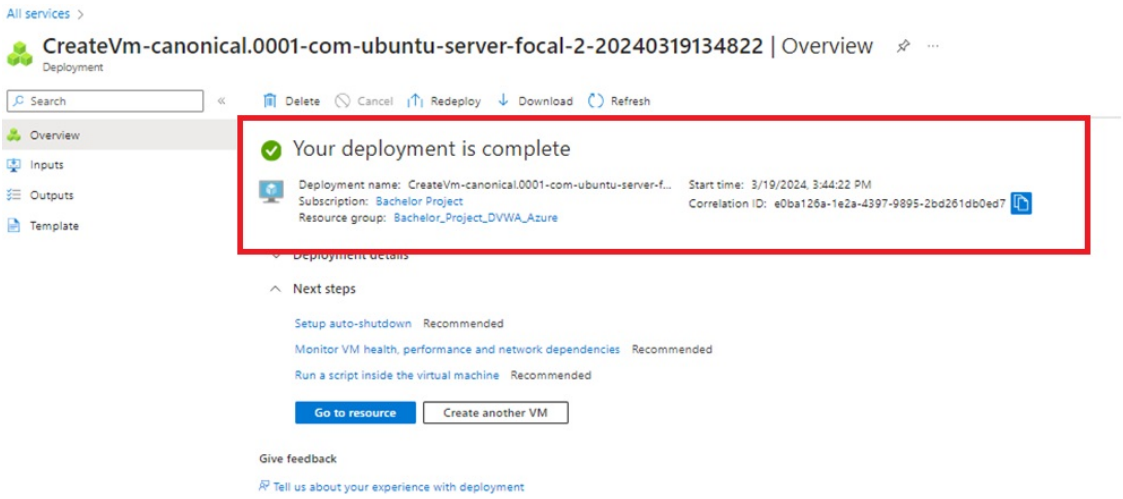

Figure A.7: VM deployment is successful

A.4 Deployment of DVWA on Azure VM

To begin the installation, the DVWA GitHub repository needs to be cloned into the var/www/html directory. This directory serves as the storage location for the Localhost files (“Setting up Damn Vulnerable Web App (DVWA) on Ubuntu in Azure”, n.d.). To clone the repository the command here used is:

sudo git clone https://github.com/digininja/DVWA

After cloning and downloading DVWA in the required directory, some configurations need to be done such as providing permissions. To give the permission, the following command is used “chmod -R 644 dvwa/”.

To configure the web application, the directory of the web application config must be accessed, and the config file should be copied into a new file. The file is copied to preserve the default values in case any mistakes are made during configuration. By retaining the original file, default values can be easily retrieved as they remain intact in the original file. Here, changes were made to the username and password. The configuration file is shown in figure A.8 The command used for this is: cd dvwa/config sudo cp config.inc.php.dist config.inc.php

To open the file in editor following command is used: sudo nano config.inc.php

```
<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled

# Database variables
#   WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
#   Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
#   See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_user' ]     = 'student';
$_DVWA[ 'db_password' ] = 'password';
$_DVWA[ 'db_port']      = '3306';

# ReCAPTCHA settings
#   Used for the 'Insecure CAPTCHA' module
#   You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA[ 'recaptcha_public_key' ]  = '';
$_DVWA[ 'recaptcha_private_key' ] = '';

# Default security level
#   Default value for the security level with each session.
#   The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or impossible'.
$_DVWA[ 'default_security_level' ] = 'impossible';

# Default locale
#   Default locale for the help page shown with each session.
#   The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$_DVWA[ 'default_locale' ] = 'en';
```

Figure A.8: DVWA File editing for configuration

After this procedure, the DVWA is done for configuration. The next step is to configure the MySQL database which is shown in figure A.9. To start the Mysql service with the following command : sudo service mysql start Check if the service is running using the “systemctl status” with the following command: systemctl status MySQL

```
student@DVWA-Testing:/var/www/html/DVWA/config$ service mysql start
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ===
Authentication is required to start 'mysql.service'.
Authenticating as: Ubuntu (student)
Password:
==== AUTHENTICATION COMPLETE ===
student@DVWA-Testing:/var/www/html/DVWA/config$
```

Figure A.9: mysql configuration

After this login to the MySQL database shown in figure A.10 with the command sudo mysql -u root -p

```
Enter password:
ERROR 1698 (28000): Access denied for user 'root'@'localhost'
student@DVWA-Testing:/var/www/html/DVWA/config$ mysql -u root -p
Enter password:
ERROR 1698 (28000): Access denied for user 'root'@'localhost'
student@DVWA-Testing:/var/www/html/DVWA/config$ ^C
student@DVWA-Testing:/var/www/html/DVWA/config$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.36-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure A.10: mysql database login

Now, a new user will be created with the username and password set in our DVWA application configuration file. Figure A.11, shows that the username was 'student' and the password was 'password.' The server being used is Localhost (127.0.0.1) . The following command will be utilized: create user 'student'@'127.0.0.1' identified by 'password';

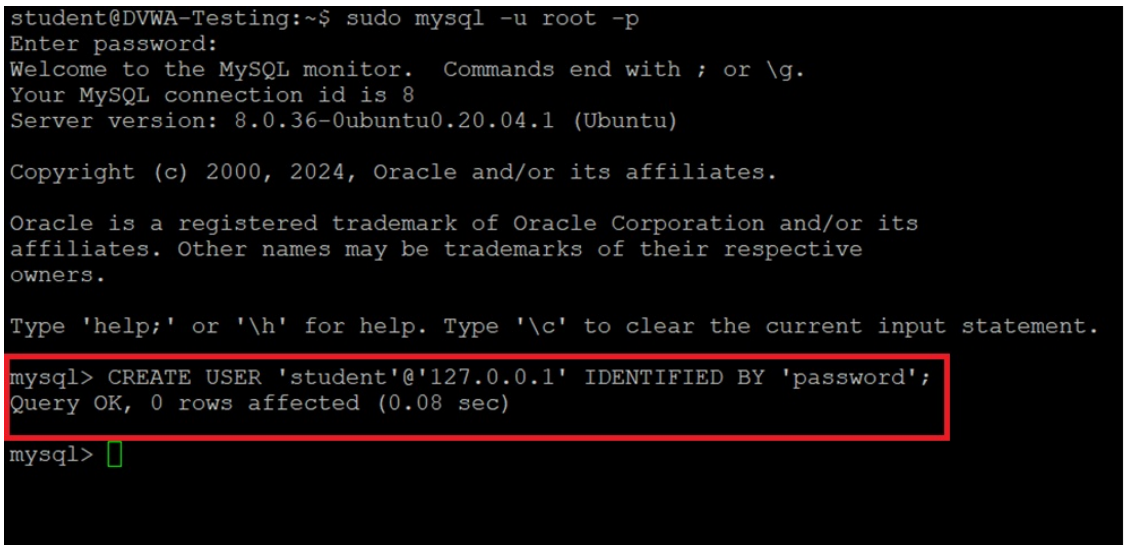

Figure A.11: mysql database create user

The next step is to give privileges so that it can use database and this is in figure A.12 and the command used is: GRANT ALL PRIVILEGES ON dvwa.* TO 'student'@'127.0.0.1' IDENTIFIED BY 'password'

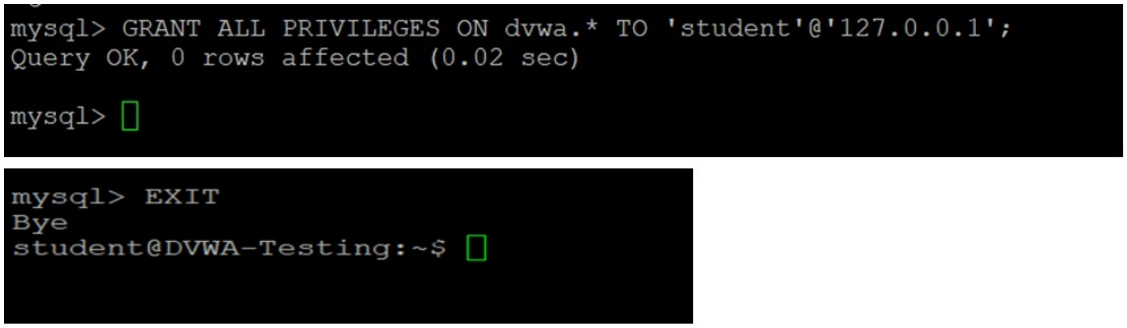

Figure A.12: Granting Privileges

Now, at this point, the process of configuring both the DVWA and database is complete and to exit and close the database, type exit. The next further step in the DVWA deployment process is to configure and install PHP. PHP functions as a scripting language mainly used for web development, which means that it is interpreted by other programs during runtime and does not require compilation. It mainly functions server-side but can also be client-side. As a web application, DVWA uses PHP to manage form submissions, generate dynamic web pages, input validation, and user authentication (Bydrec, n.d.). To install PHP following commands were used “sudo apt update” “sudo apt install php7.4”

After PHP the configuration of the server is needed and to configure the server following command is used. cd /etc/php/7.4/apache2 In the /etc/php/7.4/apache2 when the com-

mand is executed there is a “php.ini” file that is visible and in that file, the editing is done to configure the localhost. Figure A.13 shows the configuration of the server. The command used to edit the file is “sudo nano php.ini”

```
student@DVWA-Testing:/etc/php/7.4/apache2$ sudo nano /etc/php/7.4/apache2/php.ini
student@DVWA-Testing:/etc/php/7.4/apache2$
```

Figure A.13: Editing the file

Move down to where these two lines are located as shown in the figure A.14: “allow‘_url_fopen” and “allow‘_url_include” and set them On. The allow_url_include function helps the user to include a remote file using a URL instead of a local file (Sulthan, n.d.).

```
;;;;;;;;;;;;;;;;;;
; Fopen wrappers ;
;;;;;;;;;;;;;;;;;;

; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
; http://php.net/allow-url-fopen
allow_url_fopen = On

; Whether to allow include/require to open URLs (like http:// or ftp://) as files.
; http://php.net/allow-url-include
allow_url_include = On

; Define the anonymous ftp password (your email address). PHP's default setting
; for this is empty.
; http://php.net/from
;from="john@doe.com"

; Define the User-Agent string. PHP's default setting for this is empty.
; http://php.net/user-agent
;user_agent="PHP"
```

Figure A.14: Configuring PHP

After the execution of the above commands, the server Apache needs to be started by the following command and the figure A.15 also shows the service is restrating. command: sudo service apache2 start

```
student@DVWA-Testing:/etc/php/7.4/apache2$ service apache2 start
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ===
Authentication is required to start 'apache2.service'.
Authenticating as: Ubuntu (student)
Password:
==== AUTHENTICATION COMPLETE ===
student@DVWA-Testing:/etc/php/7.4/apache2$
```

Figure A.15: Starting Apache Server

After starting the server, required firewall rules were applied to run the web application in Azure as shown in the figure A.16 such as for inbound port rules, port 80,443 was configured to TCP.

Figure A.16: Setting up firewall rules

After all these procedures and configurations, the DVWA was launched and was checked by entering the URL HTTP://51.120.241/DVWA/login.php as shown in the figure A.17.

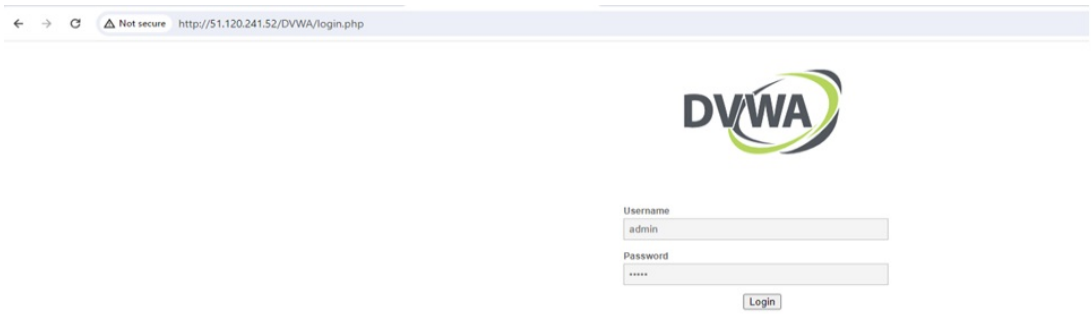

Figure A.17: DVWA Login Page

APPENDIX B

Post Deployment Testing

This appendix details the post-deployment testing of DVWA on the Azure platform before any security features are put in places

B.0.1 Scope

1. Explore vulnerabilities in the OWASP Top 10 (2021) at low risk.
2. Focus on common vulnerabilities that are aligned with OWASP such as SQL injection, Cross-Site Scripting (XSS), Broken Authentication, etc.
3. Manual testing techniques will be used to enhance the concept understanding.
4. To ensure learning is retained, findings will be documented.

The safe and controlled testing environment permits the exploration of the following vulnerabilities in a broader way:

During the pre-testing phase, data regarding the target was gathered methodically to spot any potential weaknesses. The following stages can be applied to analyze the process:

B.0.2 Vulnerability Scanning

While vulnerability scanning is most commonly associated with the scanning of Internet-connected systems, it additionally applies to system audits conducted on private networks that are not linked with the Web in order to evaluate the risk of malicious software or malevolent workers within an organization (S. Wang et al., 2013).

In the Vulnerability scanning, the network was scanned using Nikto and Nmap to get information about the host. Nmap, an acronym for “Network Mapper,” is an open-source,

publicly accessible program for network analysis and security assessments. Beyond its main function, it is an invaluable tool for system and network administrators, supporting tasks like scheduling service upgrades, keeping an eye on host or service availability, and doing network inventories. Nmap excels at identifying available hosts, the services they provide (including application name and version), and the operating systems (and versions) they run by utilizing novel techniques with raw IP packets(Jones et al., 2021).

Nikto

Nikto is a freely available web server vulnerability scanner that checks web servers for a variety of threats, including malicious files and applications. Nitko looks for web server software that is out of date. Additionally, it looks for mistakes in server settings and any vulnerabilities they might have introduced (Nikto, 2024). figure B.1 shows the output of the Nikto and Nmap scan:

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 51.120.241.52
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          51.120.241.52
+ Target Hostname:    51.120.241.52
+ Target Port:        80
+ Start Time:         2024-04-11 05:42:59 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.41 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.
 See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 2aa6, size: 61406cf506d72, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi
?name=CVE-2003-1418
+ Apache/2.4.41 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ OPTIONS: Allowed HTTP Methods: POST, OPTIONS, HEAD, GET .
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. See: CWE-552
+ 8102 requests: 0 error(s) and 7 item(s) reported on remote host
+ End Time:           2024-04-11 05:46:12 (GMT-4) (193 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Figure B.1: Output of Nikto scan

Explanation of the findings in the Nikto output which was performed on an Apache/2.4.41 server on Ubuntu provides a list of weaknesses identified:

1. Missing X-Frame-Options Header: This frame's anti-clickjacking header is missing. This header prevents the website from being rendered within an iframe or frame from another origin, hence assisting in the prevention of clickjacking attacks. This header instructs the browser as to whether the content is suitable for clickjacking's usage of frames for display.
2. Missing X-Content-Type-Options Header: There is no set value for the X-Content-Type-Options header. This might make it possible for the user agent to render the website's content differently than the MIME type, which could result in security flaws.
3. Outdated Server Version: Attackers may be able to take advantage of known vulnerabilities in the Apache/2.4.41 version. It is advised to update to the most recent version.
4. PHP Information Revealed: The existence of phpinfo() output suggests that comprehensive PHP configuration data is available to the public, potentially giving attackers valuable insights to take advantage of further vulnerabilities.

Nmap

The results of the network scan by Nmap show that it was conducted on the DVWA host. The scan reveals two key findings: First, the scan confirms that a host machine exists at the given IP address and is currently operational. Second, the scan identified an open port on this machine. Port 80, commonly used for HTTP traffic, is found to be accessible. Additionally, the scan shown in the figure B.2 was able to determine that a web server service (HTTP) is running on this open port. This suggests that the machine at a certain IP might be hosting a website or some other web-based application.

Figure B.2: Output of Nmap scan

Directory Bruteforcing

Directory Bruteforcing assists in locating possibly hidden directories on websites that hold private data. It operates utilizing automated technologies to attempt a variety of options or by methodically speculating on directory names. This may uncover application vulnerabilities or concealed content that should not made available to the general public (Shahid et al., 2022). To access different folders and files, DirBuster examined the target IP address and port. It discovered shown in figure B.3 as other directories that returned a 403 Forbidden answer, suggesting that although they may exist, access is limited. It also discovered the /phpinfo.php file, which contains information on the PHP settings on the server. Hackers may use this information to take advantage of PHP environment vulnerabilities.

```
DirBuster 1.0-RC1 - Report
http://www.owasp.org/index.php/Category:OWASP_DirBuster_Project
Report produced on Sun Apr 14 09:37:44 EDT 2024
————————————————————————————————

http://51.120.241.52:80
————————————————————————————————
Directories found during testing:

Dirs found with a 403 response:

/icons/
/javascript/
/icons/small/
/javascript/jquery/
/server-status/

Dirs found with a 200 response:

/


————————————————————————————————
Files found during testing:

Files found with a 200 responce:

/phpinfo.php


————————————————————————————————

```

Figure B.3: Output of DirBuster

Nessus Scan

The Nessus scan revealed enough information vulnerabilities in the VM. Allowing unsafe HTTP methods on unnecessary directories, such PUT, DELETE, CONNECT, TRACE, and HEAD, increases the danger of an attacker taking advantage of your server. Since PUT and DELETE may make it easier for unauthorized people to alter or remove data without authorization if security precautions aren't taken, the TRACE method is capable of offering information about internal servers. Furthermore, using specific techniques to make excessive requests could lead to DoS attacks, which would jeopardize server availability. Nessus scan has revealed many informational level vulnerabilities or weaknesses, that need to be done to secure the website.

The scan shows the Web servers permitting insecure HTTP methods such as PUT, DELETE, CONNECT, TRACE, and HEAD on unnecessary directories risk exploitation by attackers. The TRACE method can reveal internal server information, while PUT and DELETE may facilitate unauthorized data modification or deletion if security measures are lacking. Additionally, sending excessive requests with certain methods could result in DoS attacks, compromising server availability. Figure B.4 shows that the HTTP Method was allowed.

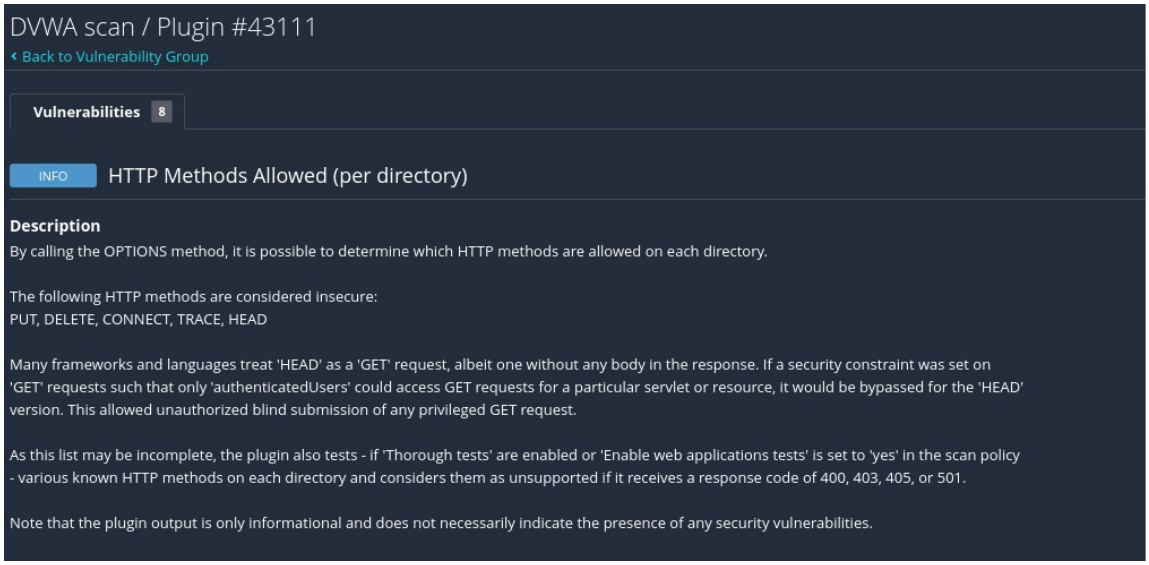

Figure B.4: HTTP Method allowed

Further, the scan reveals in figure B.5 that the VM has a vulnerability that refers to a missing or weak Content-Security-Policy header mainly for the framing. The CSP is a type of security that) aids in the detection and mitigation of some attack types, such as data injection and cross-site scripting (XSS) attacks. These assaults are used for a variety of purposes, such as malware delivery and data theft (Weichselbaum et al., 2016).

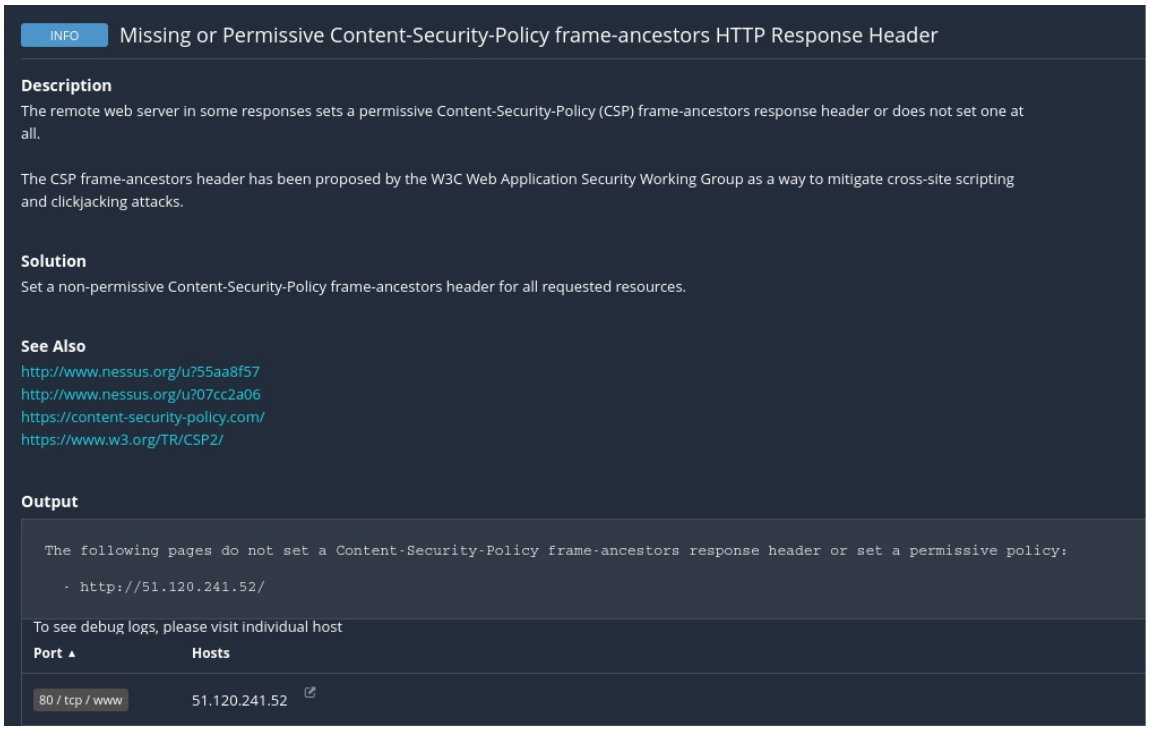

Figure B.5: Missing CSP frame

A SYN scan in figure B.6 conducted in Nessus reveals the status of open and closed ports. This disclosed information can be exploited by attackers to pinpoint vulnerabilities within a company's or user's network for potential exploitation.

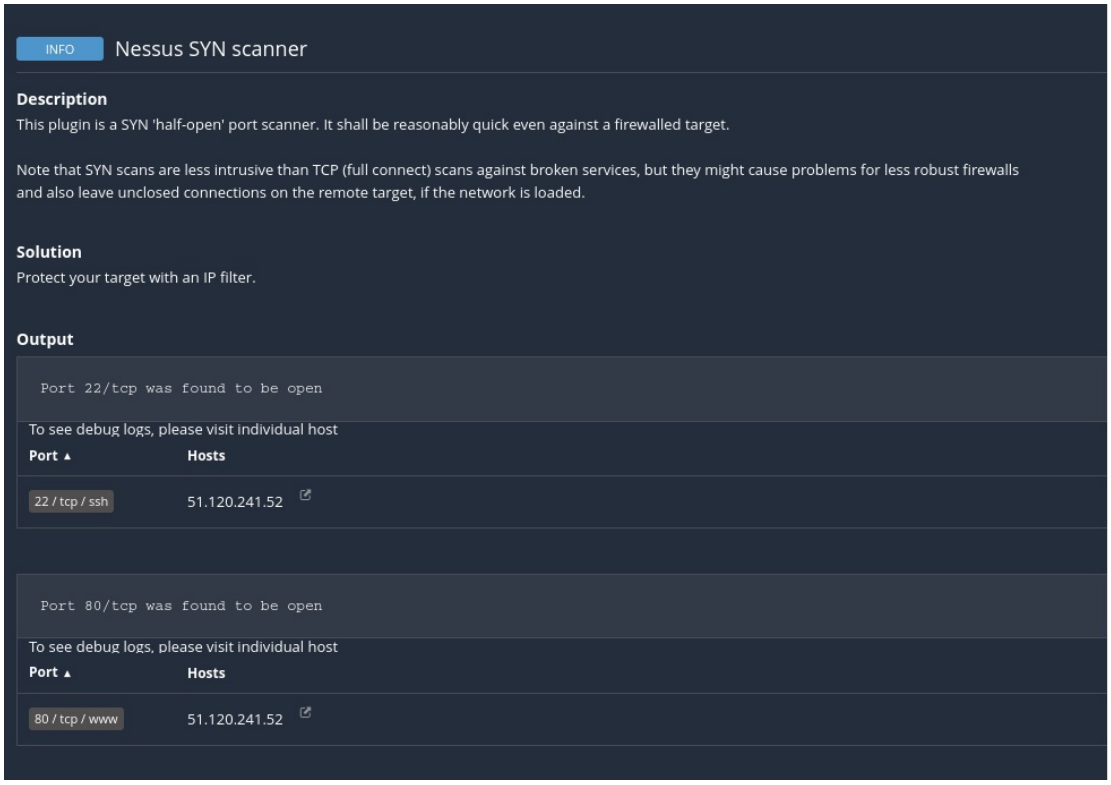

Figure B.6: Nessus SYN scan

B.0.3 Manual Testing of DVWA

This section displays the vulnerabilities identified during the manual testing of DVWA. A variety of methods were used to exploit DVWA and uncover these vulnerabilities. The research on the subject of web application security on the Azure cloud platform is closely related to the manual testing of DVWA. Although DVWA isn't directly implemented on Azure, it still has several benefits. Even though DVWA isn't directly hosted on Azure, manual testing of DVWA is essential for this study on the security of web apps installed on Azure. DVWA is a powerful tool that highlights web app vulnerabilities and provides information about their nature and consequences. It serves as a secure environment for assessing the practicality of security measures outside of the scope of automated tools. Furthermore, manual testing helps to create strong security measures by offering insights into attacker tactics. Through experimentation in the controlled setting of DVWA, researchers may efficiently improve security frameworks. This is in line with Peffer's DSR methodology, which allows for iterative development depending on test results.

Command Injection

1. Description: Command injection is a type of vulnerability that is related to web application security in which an attacker executes arbitrary commands on the server that is hosting an application that mainly results in the complete compromise of the application with the data (Ltwagnon, 2023). When an application provides sensitive user-supplied data—such as forms, cookies, HTTP headers, etc.—to a system

shell, command injection attacks may be conceivable. The operating system commands that the attacker supplies are often run in this attack using the privileges of the susceptible program. Inadequate input validation makes command injection attacks viable.

2. Proof of Concept: Upon inspecting the source code for the low-security leveling figure B.7, It becomes evident that the code lacks validation for whether the target aligns with an IP address. Special character filtering is absent. In Unix/Linux systems, the semicolon (;) facilitates command separation, potentially leading to vulnerabilities (Foundation, n.d.).

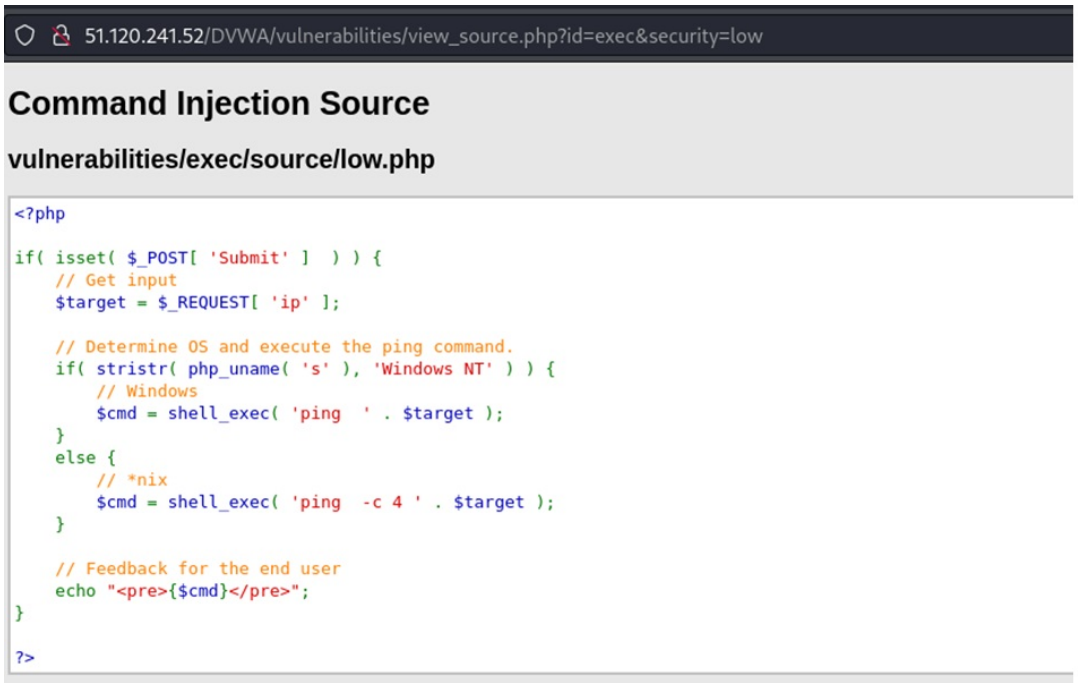

Figure B.7: Source code for Common Injection

When the payload is entered the results were displayed in figure B.8 as the details of /etc/passwd/

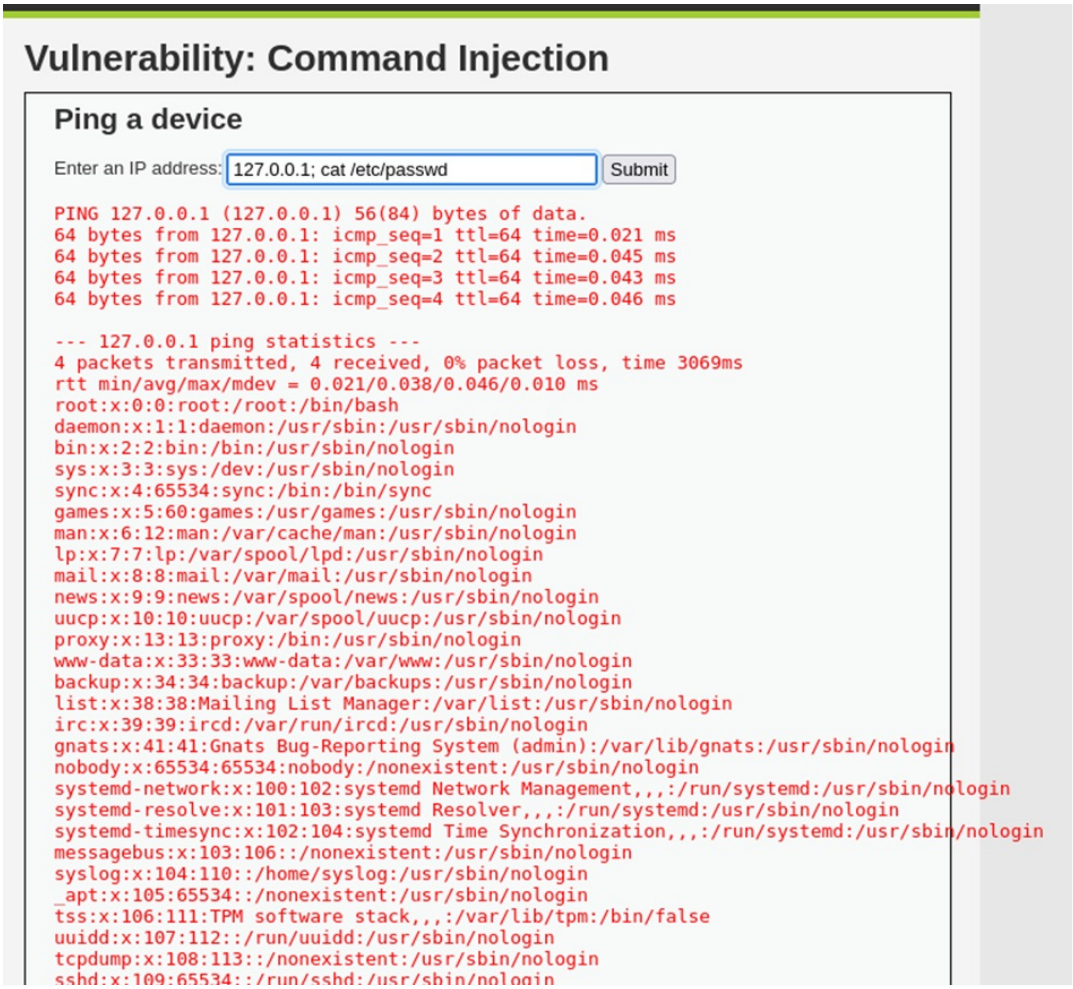

Figure B.8: Command injection payload

Medium Level: In this level, it can be seen in figure B.9 that a blacklist has been implemented to prohibit the use of some characters. When adding the payload in figure B.10 of 127.0.0.1 |cat/etc/passwd, the output can be seen.

Command Injection Source

vulnerabilities/exec/source/medium.php

```php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&&' => '',
        ';'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping  ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Figure B.9: Command injection source code medium level

Vulnerability: Command Injection

Ping a device

Enter an IP address: 127.0.0.1 | cat /etc/passwd Submit

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:106::/nonexistent:/usr/sbin/nologin
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
uuidd:x:107:112::/run/uuidd:/usr/sbin/nologin
tcpdump:x:108:113::/nonexistent:/usr/sbin/nologin
sshd:x:109:65534::/run/sshd:/usr/sbin/nologin
landscape:x:110:115::/var/lib/landscape:/usr/sbin/nologin
pollinate:x:111:1::/var/cache/pollinate:/bin/false
fwupd-refresh:x:112:116:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
_chrony:x:113:121:Chrony daemon,,,:/var/lib/chrony:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
student:x:1000:1000:Ubuntu:/home/student:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
mysql:x:114:123:MySQL Server,,,:/nonexistent:/bin/false
```

Figure B.10: Command injection payload at medium level

Brute Force

1. Description: A brute force attack, to put it briefly, is a password experiment that employs a variety of potential ASCII characters, either separately or in combinations, and several different numeric/alphanumeric password combinations to unlock a blocked device (Sadqi & Maleh, 2020).
2. Proof of Concept: Low level: The first step of the attack is to inspect the form field. When inspecting in figure B.11 it is found that the login form is using the “GET” method.

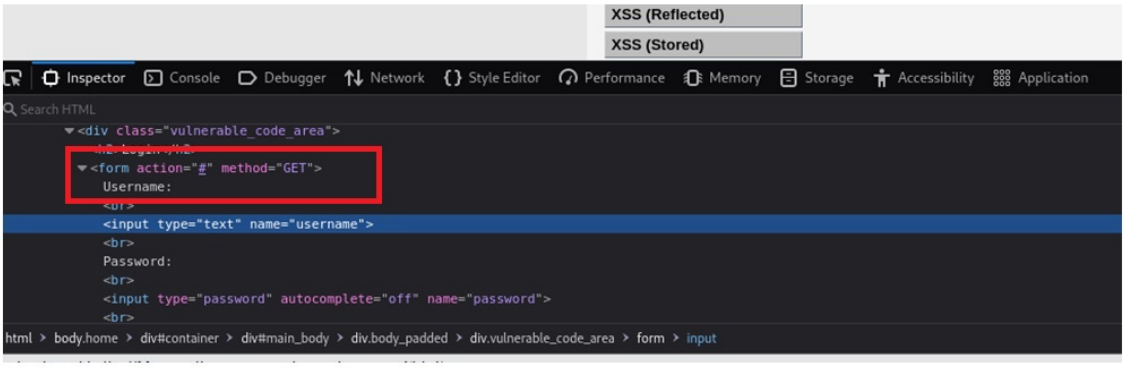

Figure B.11: GET method is used in the attack

The command “hydra 51.120.241.52 -l admin -P /home/kali/Desktop/password.txt http-get-form” demonstrates how Hydra can be used to brute-force a login form on a web server in figure B.12 and the output was a list of admin and password credentials.

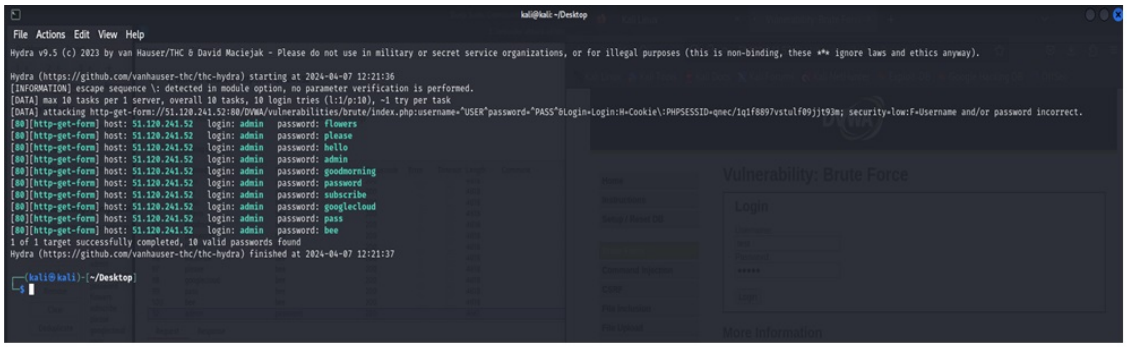

Figure B.12: Output of hydra showing credentials

In the BurpSuite proxy figure B.13 , intercept the request, and send it to the intruder to check if the login page is visible and identify the parameters to be changed or

attacked.

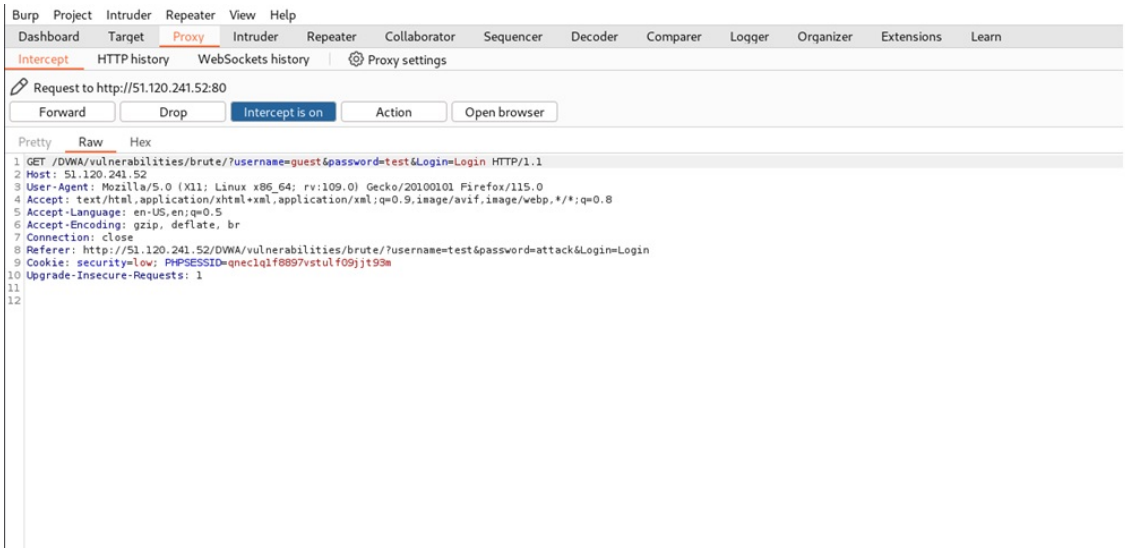

Figure B.13: Burpsuite intercepting a request

In the intruder section figure B.14 shows the payloads. Set the payload, and choose the type of attack. And after the attack, the login credentials can be found that are shown in figure B.15.

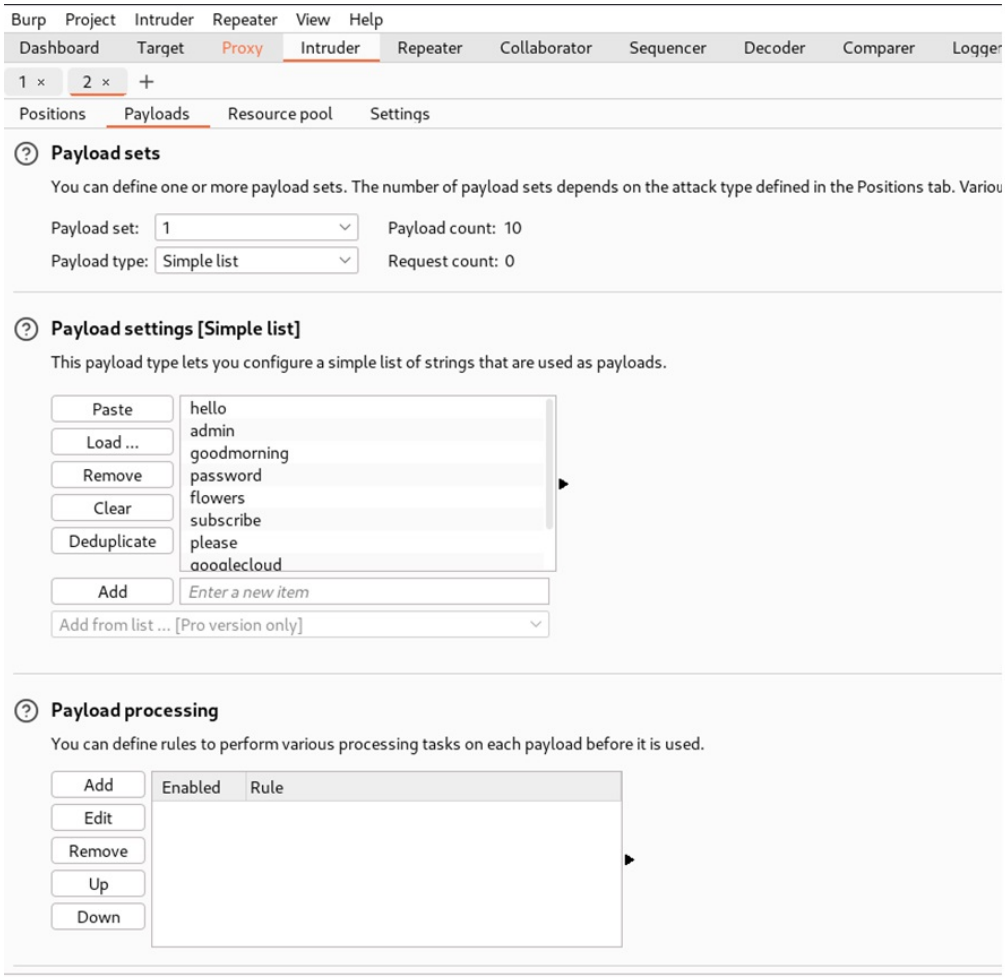

Figure B.14: Intruder with payloads

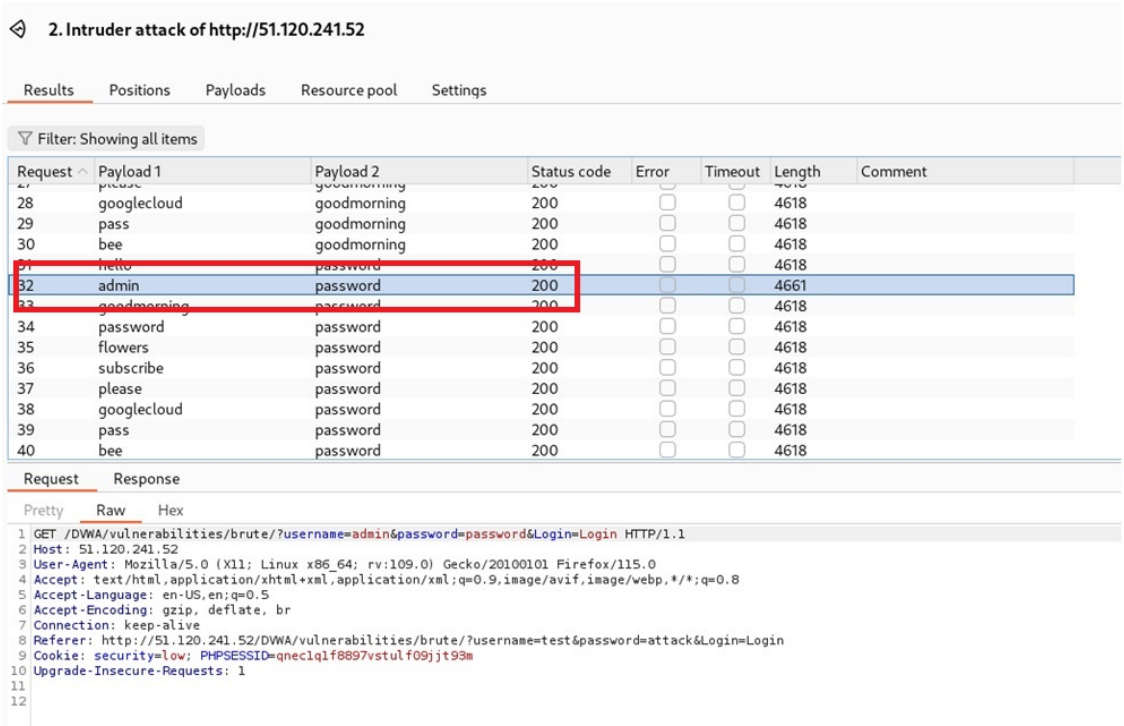

Figure B.15: Credentials are visible

Medium Level: Intercept the request in BurpSuite as shown in figure B.16, send it to the Intruder visible in figure B.17, and then set payloads, which eventually leads to the retrieval of the credentials.

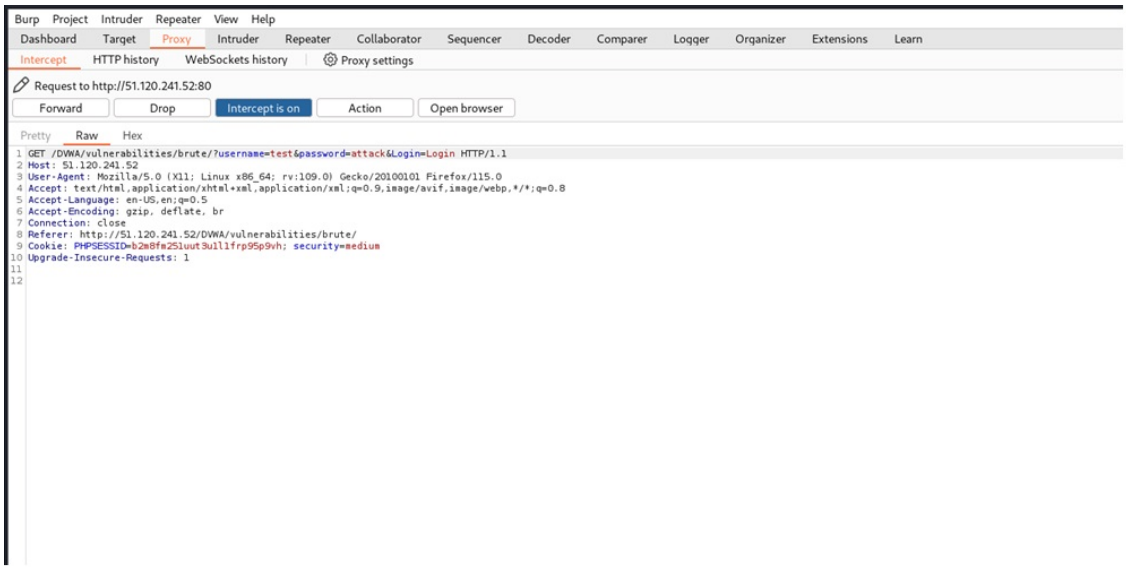

Figure B.16: Request intercepted in BurpSuite

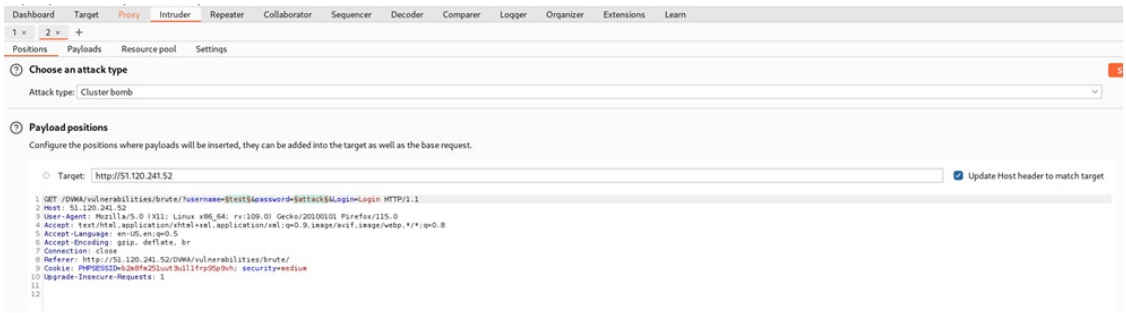

Figure B.17: Request in Intruder in BurpSuite

This action was taken to identify and extract the credentials by executing a brute-force attack on the input field in the figure B.18.

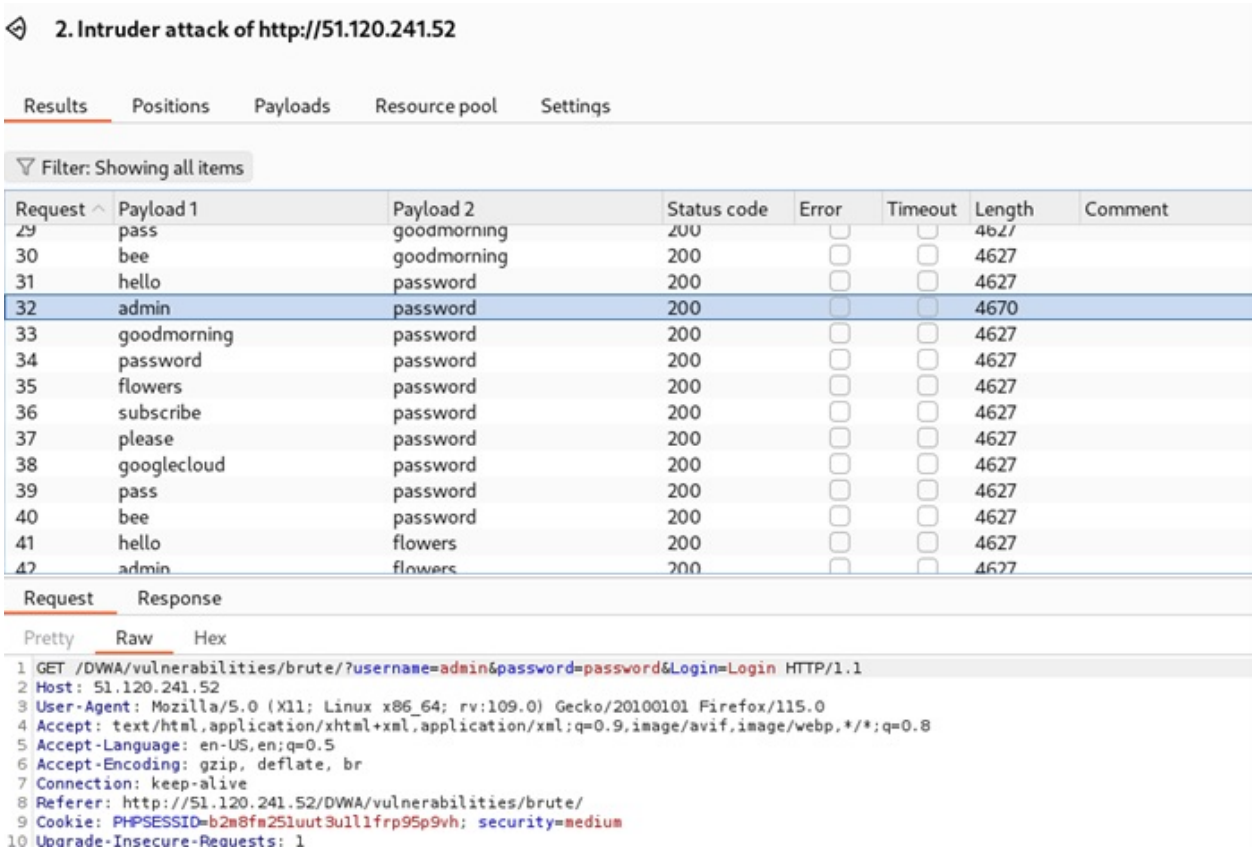

Figure B.18: Credentials in intruder

Cross-Site request Forgery (CSRF)

1. Description: In a cross-site request forgery (CSRF) attack, an attacker deceives the victim's web browser into submitting an authorized HTTP request to a web application that is vulnerable, with the goal of executing a state-changing operation without the victim's knowledge or agreement. CSRF attacks can target things like altering user

account credentials or privileges, executing arbitrary code remotely, or transferring money illicitly (Likaj et al., 2021).

2. Proof of Concept: Low Level: As the code lacks sufficient CSRF protection, attackers can take advantage of it by creating malicious URLs that carry out unauthorized actions on behalf of users that are signed in. Attackers can generate URLs with parameters for password changes and trick authenticated users into clicking on them if there is no way to verify the origin of the request. When the code is clicked, it changes the password without requiring further authentication or user permission, which poses a serious security risk and can be seen in figure B.19 .

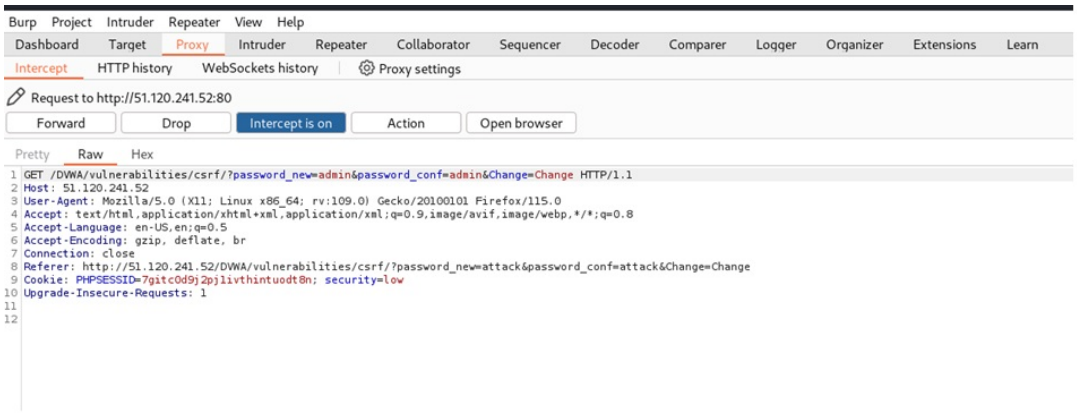

Figure B.19: BurpSuite request intercepting

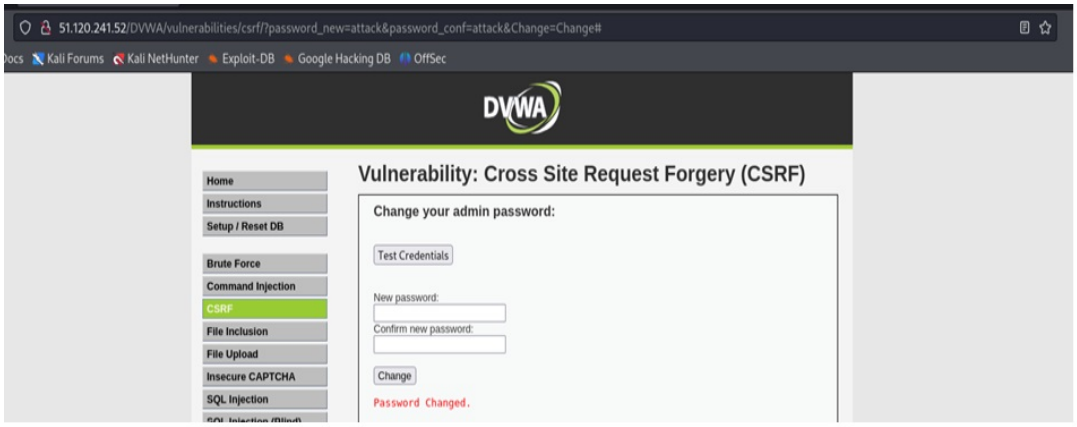

Figure B.20: Credentials are visible

Figure B.20 credentials are visible in the request and are not encoded, or encrypted. The credentials can be changed in the URL. Medium level: A Cross-Site Request Forgery (CSRF) vulnerability exists in the code that has been provided. This type of vulnerability arises when a website trusts a request that seems to come from the same site but is actually coming from a hostile source. In an effort to stop this, the code verifies that the request came from the same server by looking at the HTTP

Referer header in figure B.21 . This safeguard, however, is ineffective because an attacker can readily modify the Referer header. The attacker may deceive the server into accepting the malicious request by capturing the legitimate request using a tool like Burp and adding the HTTP Referer header that corresponds to the expected value. This gives them the ability to carry out unauthorized actions on the user's behalf, like changing their password without the user's knowledge.

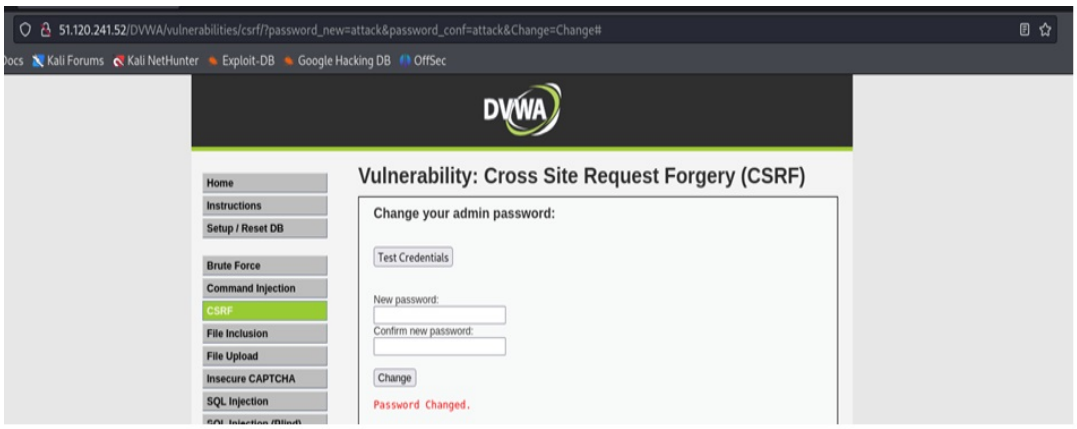

Figure B.21: Credentials are visible

In this level, a HTML file as in figure B.22 with a script was created and that uses XMLHttpRequest to send a request to the vulnerable server, with the Referer serving as the target URL. However, browser constraints on changing this header make this technique impractical.

Sending the request straight from the target server is the goal instead. Exploiting the combination of a Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) vulnerability is one simple way. This reduces the focus to CSRF by placing the payload inside a reflected XSS.

The payload will be sent to the server via a file upload vulnerability, which will make it easier for the malicious link to be sent.

To execute this set the DVWA security level low and the file will be uploaded. figure B.23 and figure B.24 shows the uploading of the file and successful result.

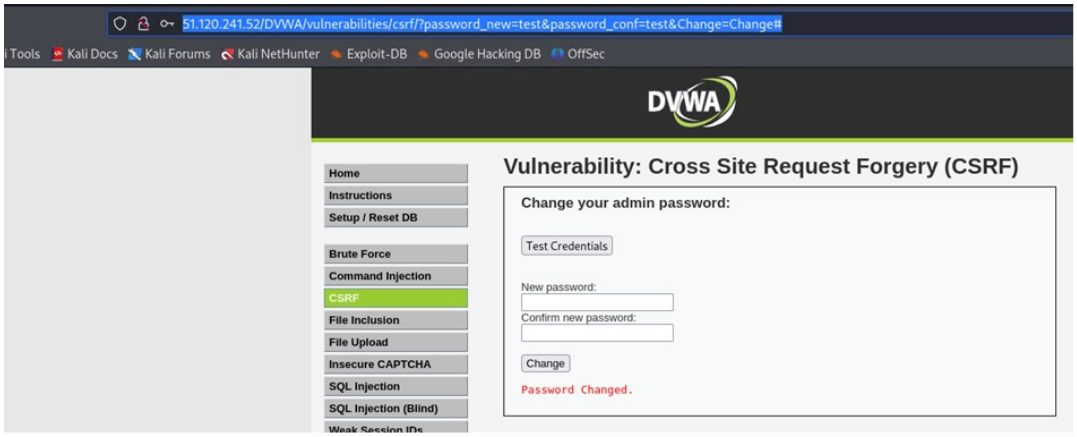

Figure B.22: HTML file with a script

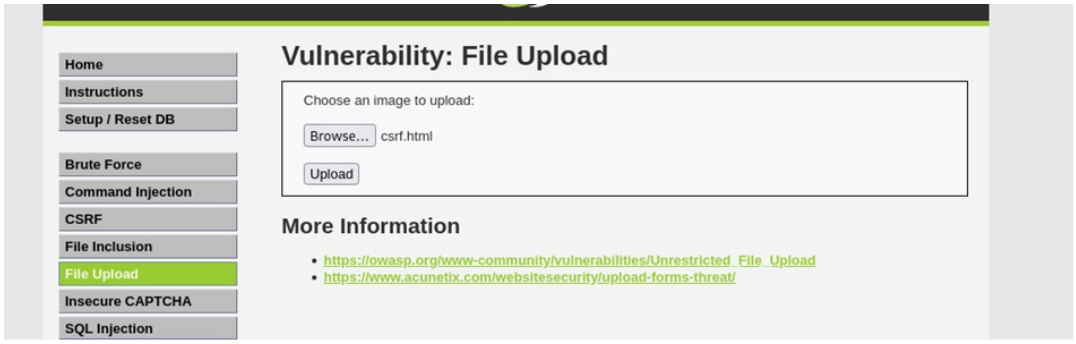

Figure B.23: CSRF file.html file uploading

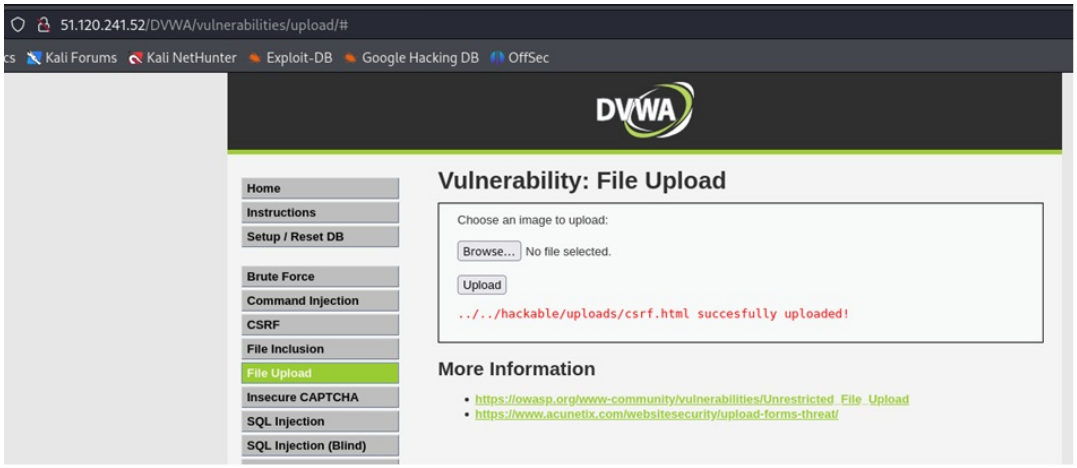

Figure B.24: Successfully uploaded the file

File Inclusion

1. Description: The file inclusion vulnerability enables the attacker to access and include a file, generally by taking advantage of “dynamic file inclusion” to the targeted application. This vulnerability happens because of the insufficient validation of user-supplied input (OWASP Foundation, 2014). File inclusion vulnerabilities are predominantly present in web applications employing a scripting runtime. These vulnerabilities grant attackers access to sensitive files on web servers or enable them to execute malicious files by exploiting included functionality on servers (Sumo Logic, 2023).

2. Proof of Concept: Low Level: In the file Inclusion tab, navigate to the URL as shown in the figure B.25 and update it from include.php to ?page=../../../../../../etc/passwd as shown in figure B.26. Figure B.2 shows that file inclusion is successful.

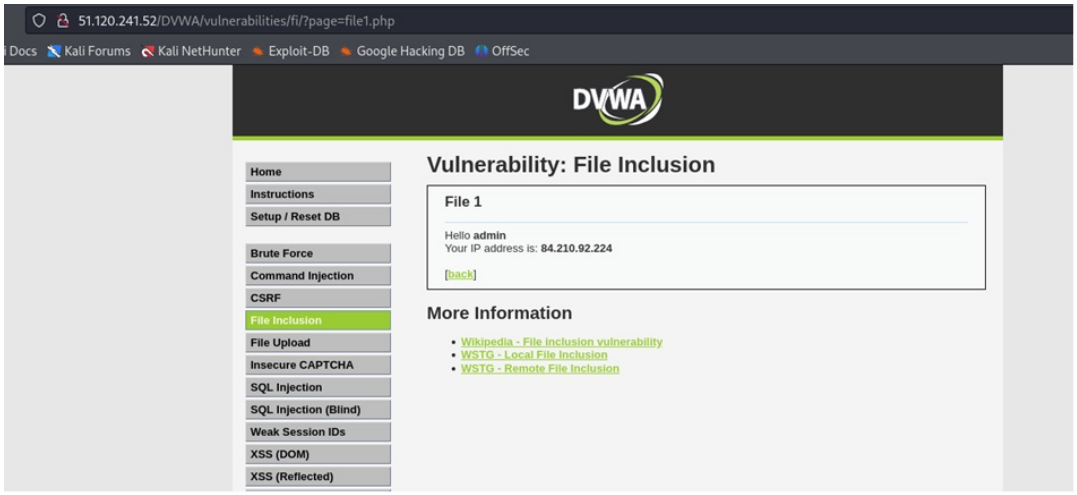

Figure B.25: Figure showing File Inclusion main tab

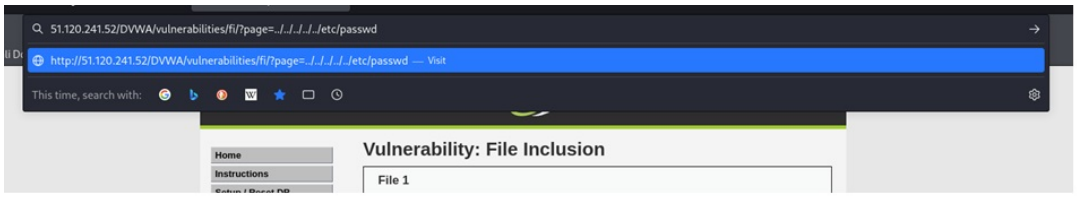

Figure B.26: Changing the URL as required

Figure B.27: Output showing the successful file inclusion

Medium Level: Testing the low-difficulty exploits further reveals that the directory traversal method is no longer viable for file access, showing improved server security with the filtering of '../' or '..́patterns in figure B.28 . The next step is to try to access the file without using '../' or '..ás shown in figure B.29 .

Figure B.28: Page source at medium level

Replace include.php to /etc/passwd

Figure B.29: Replacing include.php to /etc/passwd

File Upload

1. Description:

2. Proof of Concept: Since it is a PHP application, it allows for the upload of a PHP shell. Figure B.29shows the main tab of the lab in File Upload Kali contains numerous webshells. When uploading a webshell named “simple-backdoor.php,” the upload is successful as shown in figure B.31 and figure B.32. Upon accessing the URL as in figure B.33, the shell is obtained, enabling the execution of commands on the remote server. Figure B.34 reveals detailed information about the file, including a list of commands or permissions the user possesses. This constitutes remote code execution on the server machine.

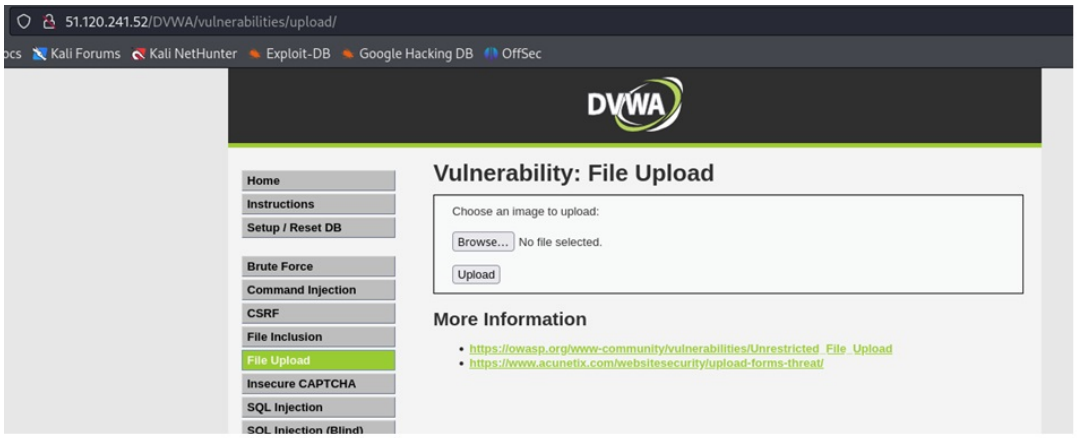

Figure B.30: File Upload tab in DVWA

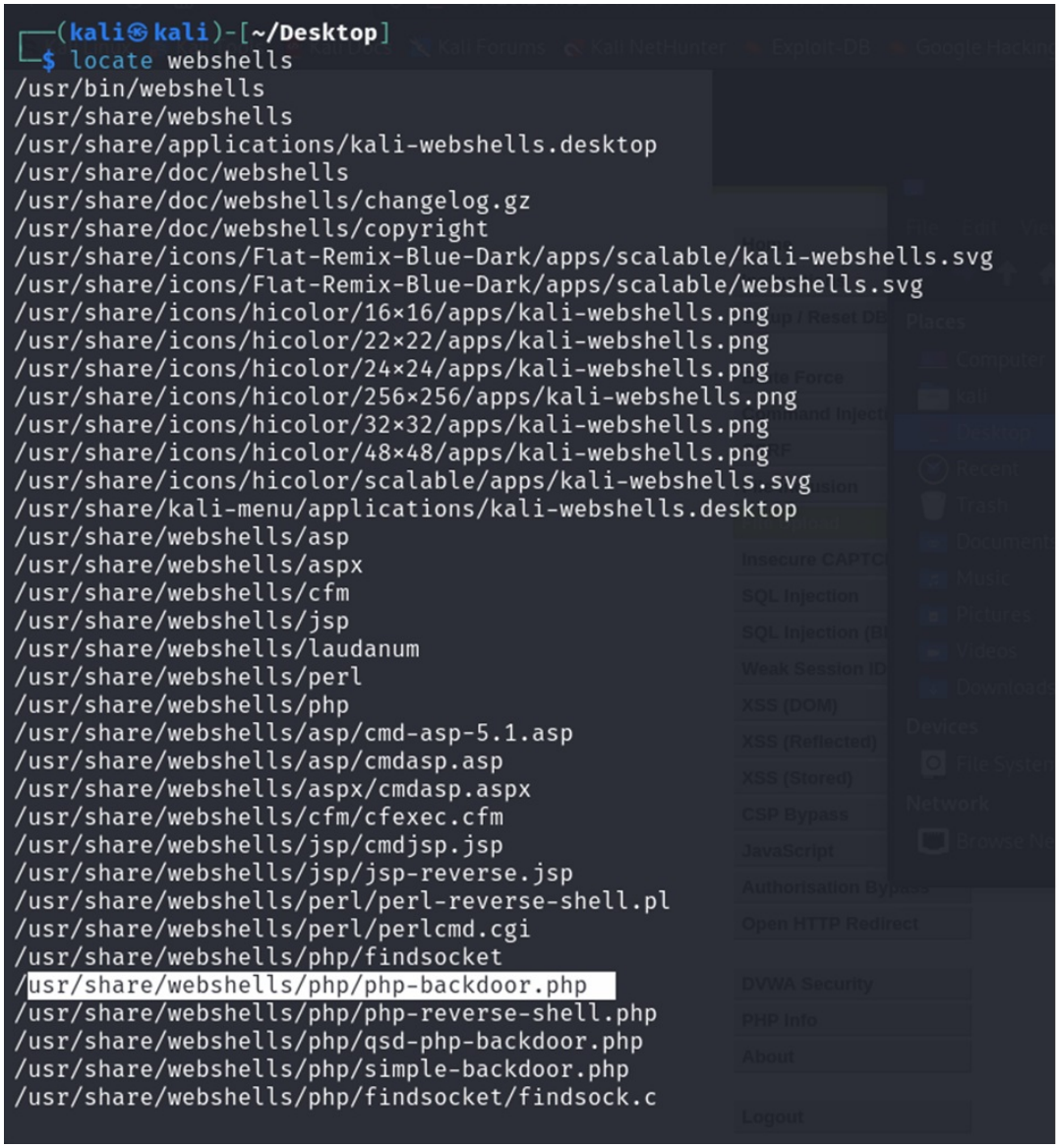

Figure B.31: simple-backdoor.php file in Kali

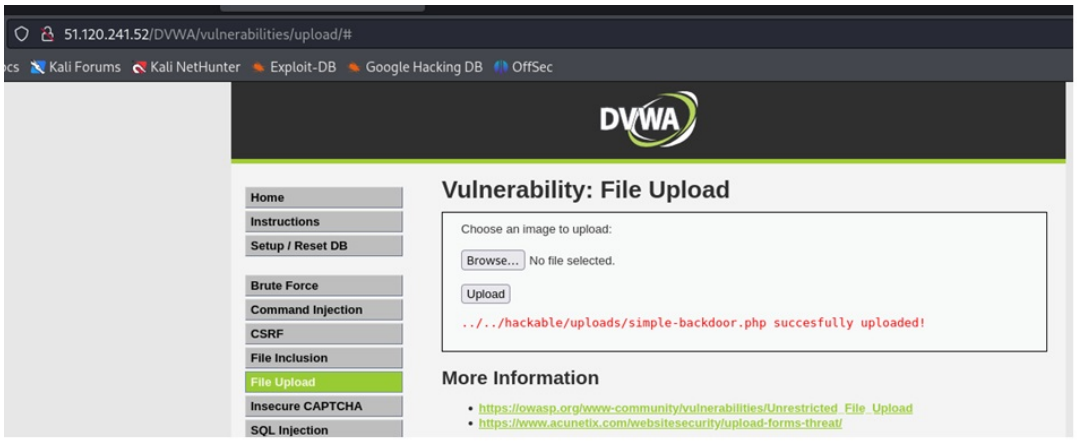

Figure B.32: File successfully uploaded

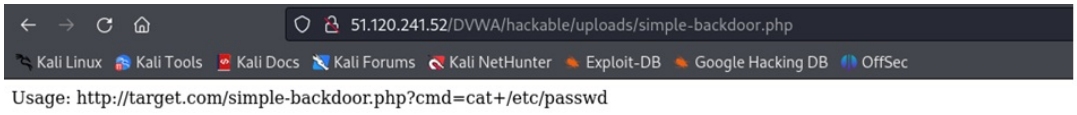

Figure B.33: Showing Remote Code execution

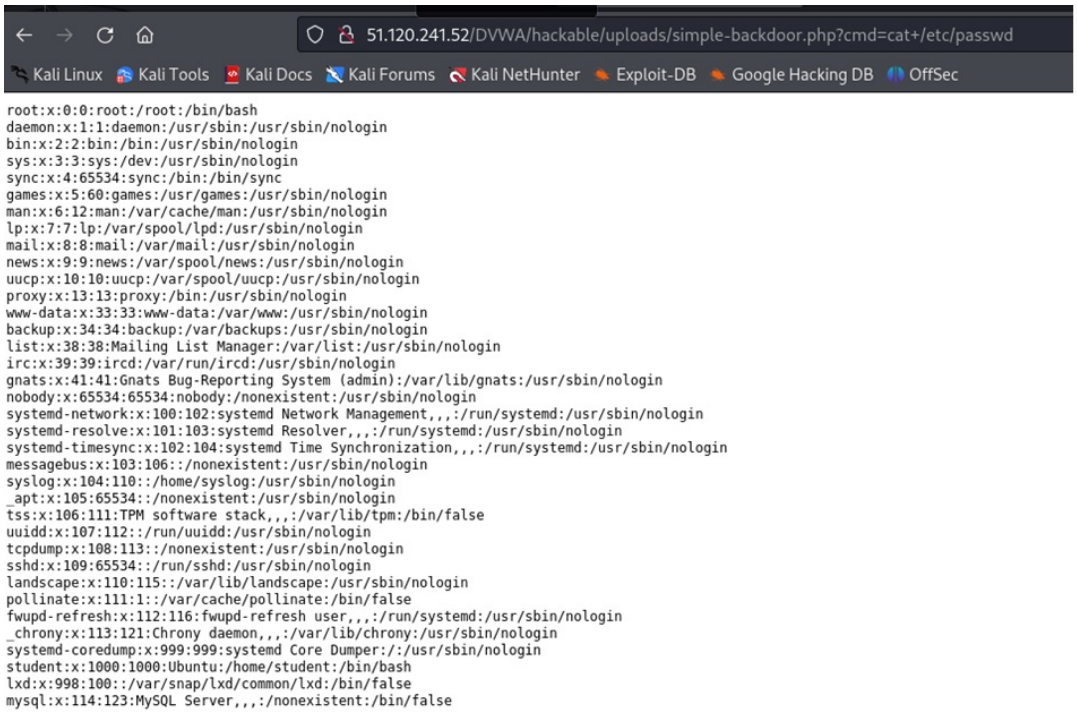

Figure B.34: Output providing detailed list of commands and permissions

Insecure CAPTCHA

1. Description: CAPTCHAs are employed by web applications to prevent the inclusion of automated inputs from attackers, be it user or any bot that can negatively impact the functionality. Utilizing insecure CAPTCHA makes attackers to bypass the anti-automation protections and gives users a false impression of security.

2. Proof of Concept:

   For this vulnerability, the scenario was reCAPTCHA API key was missing and needed to register for the key in figure B.35. For that follow the link provided and register for the key as in figure B.36and

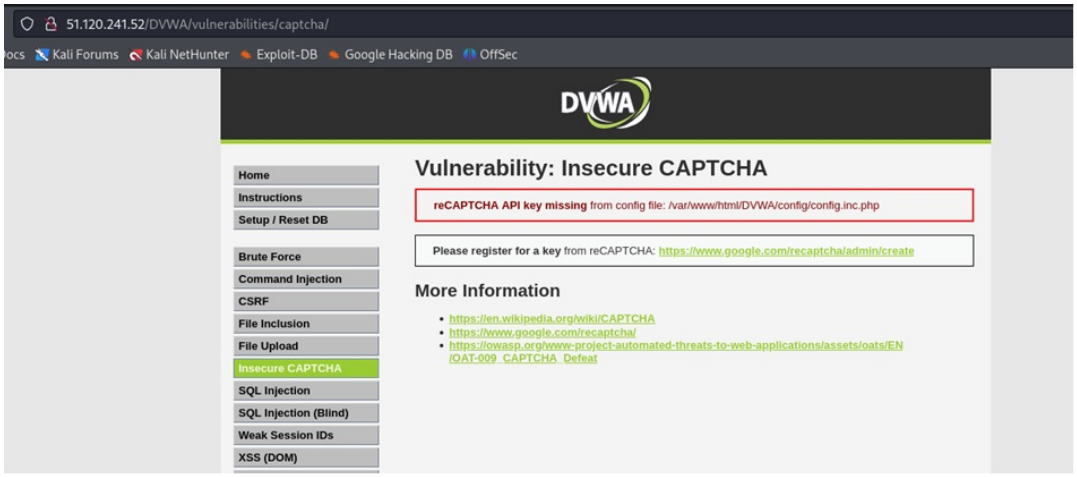

Figure B.35: Insecure CAPTCHA lab main view showing link to register key

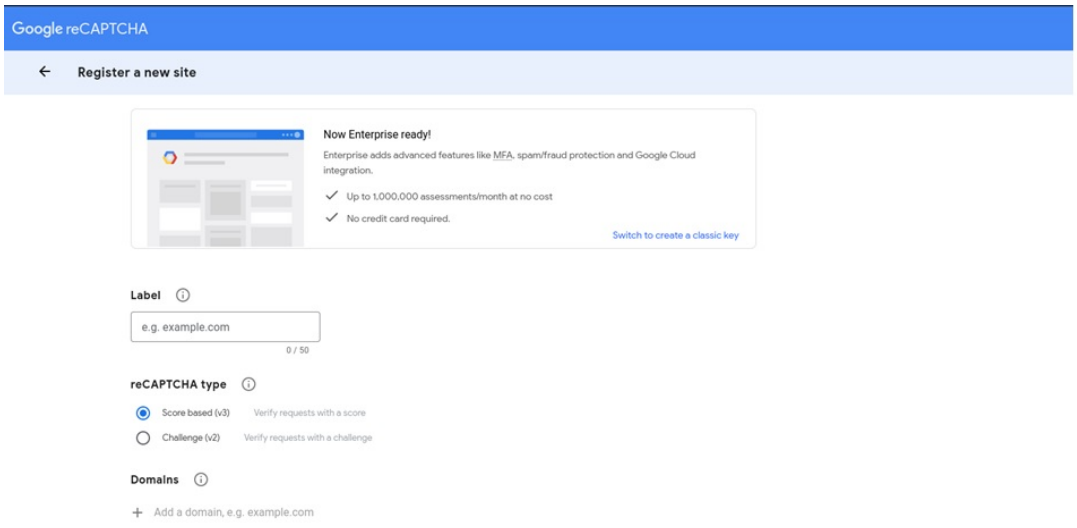

Figure B.36: Registering for key

Copy the public and private keys provided and upload them in the configuration file as shown in figure B.37. Insert the keys in the config file.

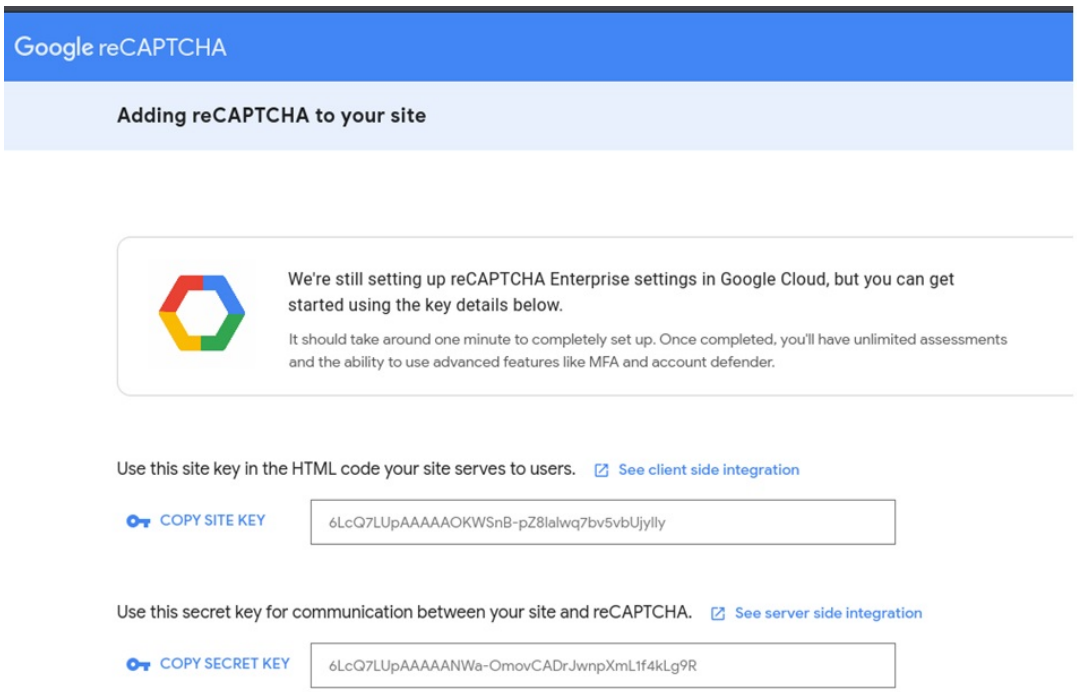

Figure B.37: Keys to generate reCAPTCHA

Now refresh the page and the reCAPTCHA will be working which is shown in figure B.38 and figure B.39.

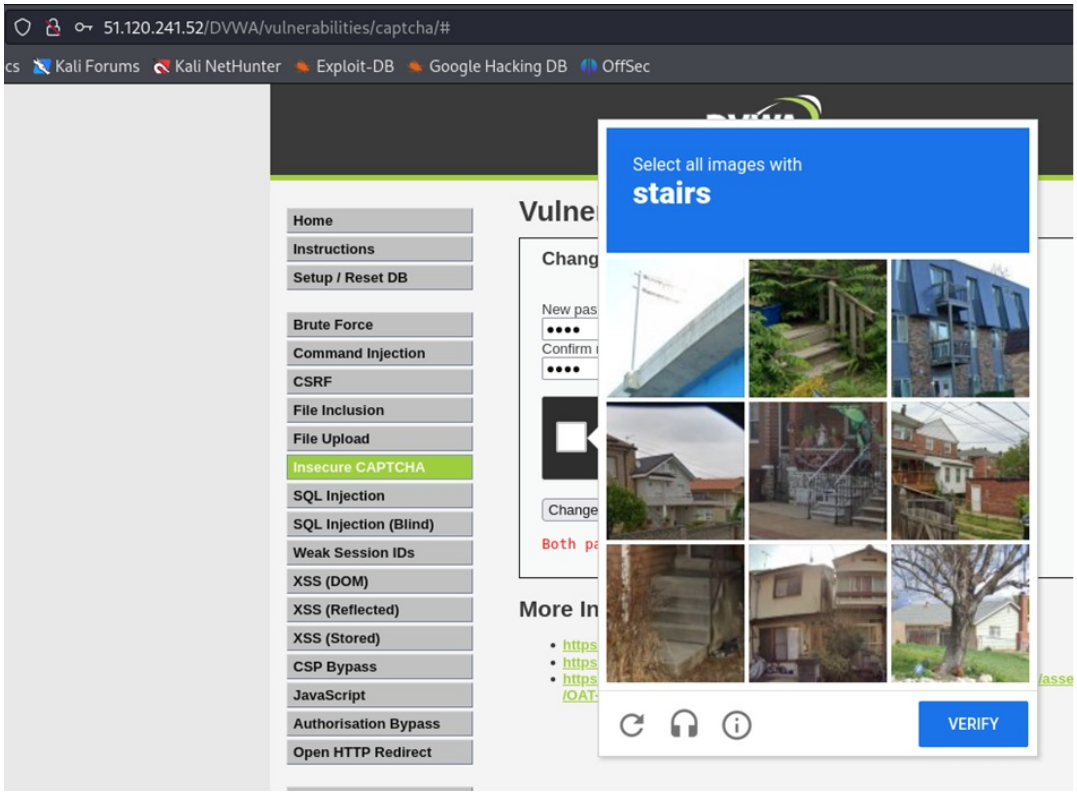

Figure B.38: reCAPTCHA page is working

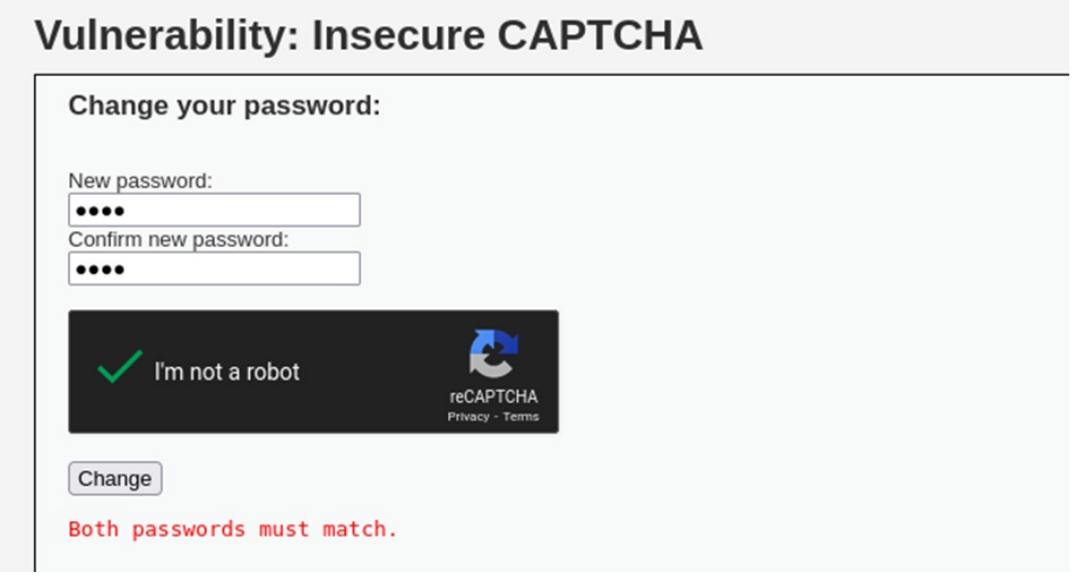

Figure B.39: A website login page with a form titled “Change your password”

SQL Injection

1. Description: SQL injection allows attackers to change the queries made by the program and with this vulnerability attacker can access the sensitive information without any authorization (Crespo et al., 2023).
2. Proof of Concept: Low Level: figure B.29 shows the main lab page of SQL injection. When entering a random number in the user ID as in figure B.41, it provides details of the user.

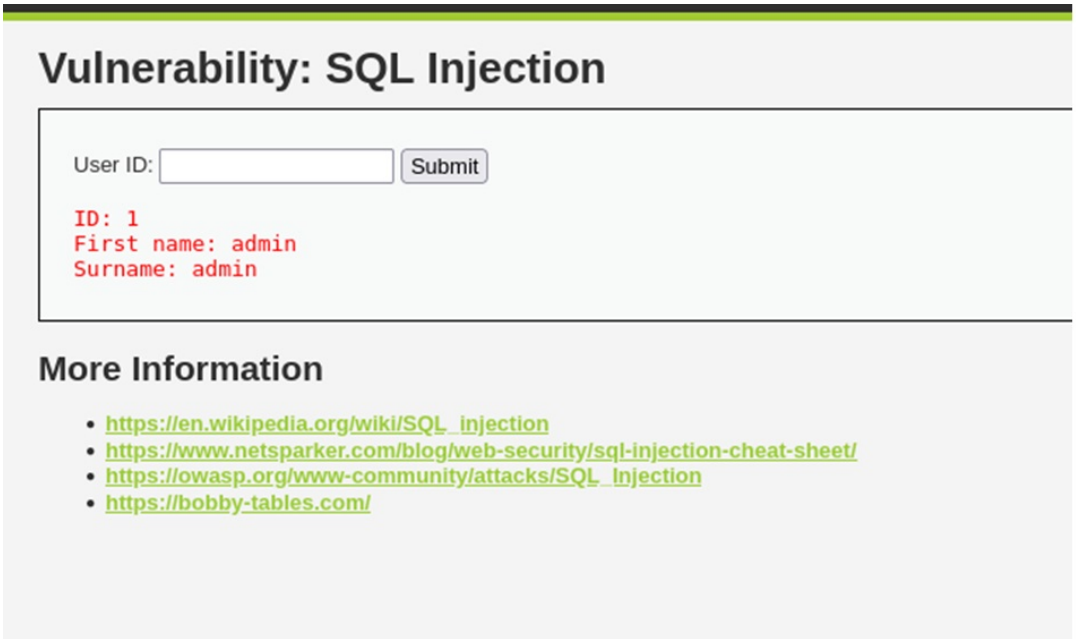

Figure B.40: SQl injection lab page

When entering the payload, it accepts and returns the following details,and figure B.42 shows the password list.

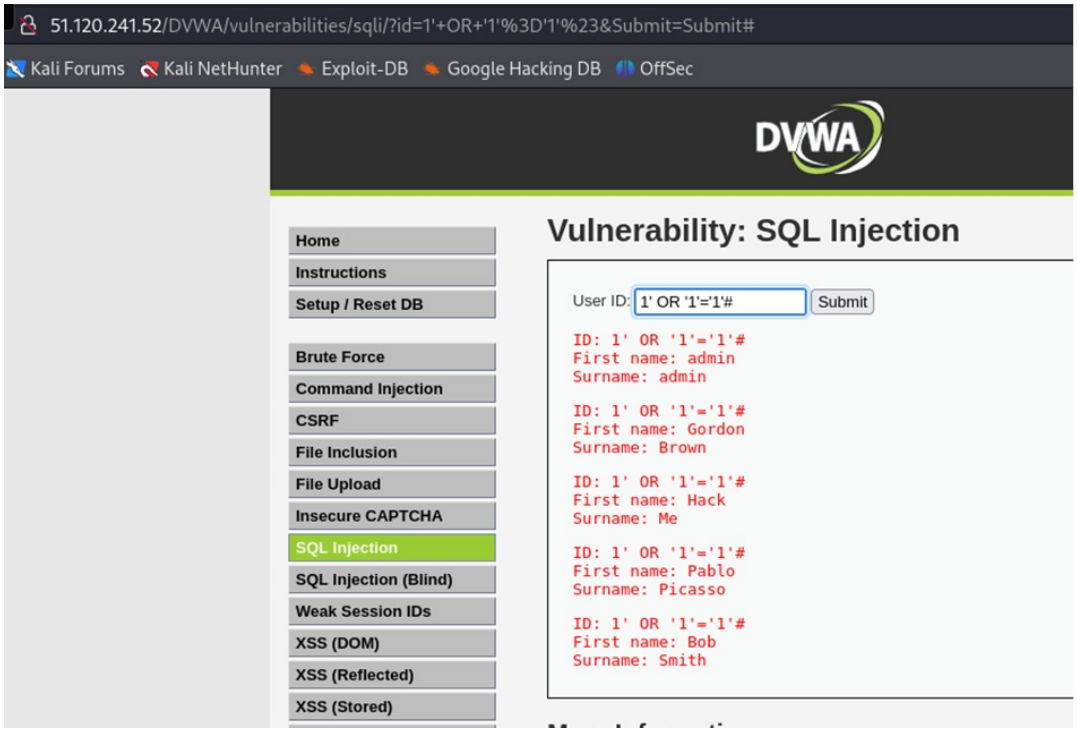

Figure B.41: With payload, database provides list of users

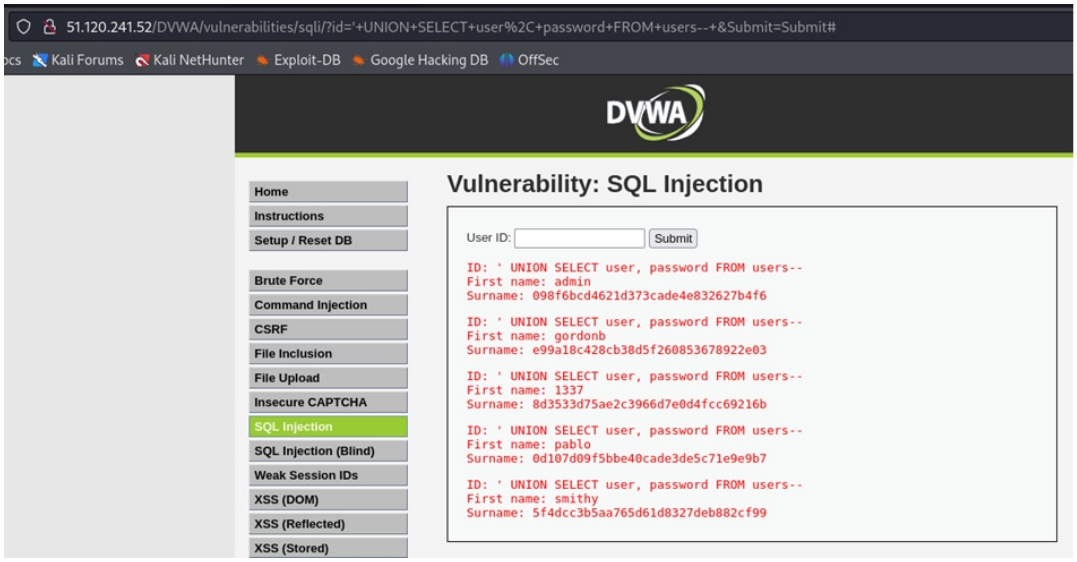

Figure B.42: Password list with details

Medium level: In this level, no input field allows to injection of the payload as can be seen in figure B.43, Then it is needed to go to the inspect and view the details as in figure B.44. Then insert the payload and check the output as shown in figure B.45 and figure B.46

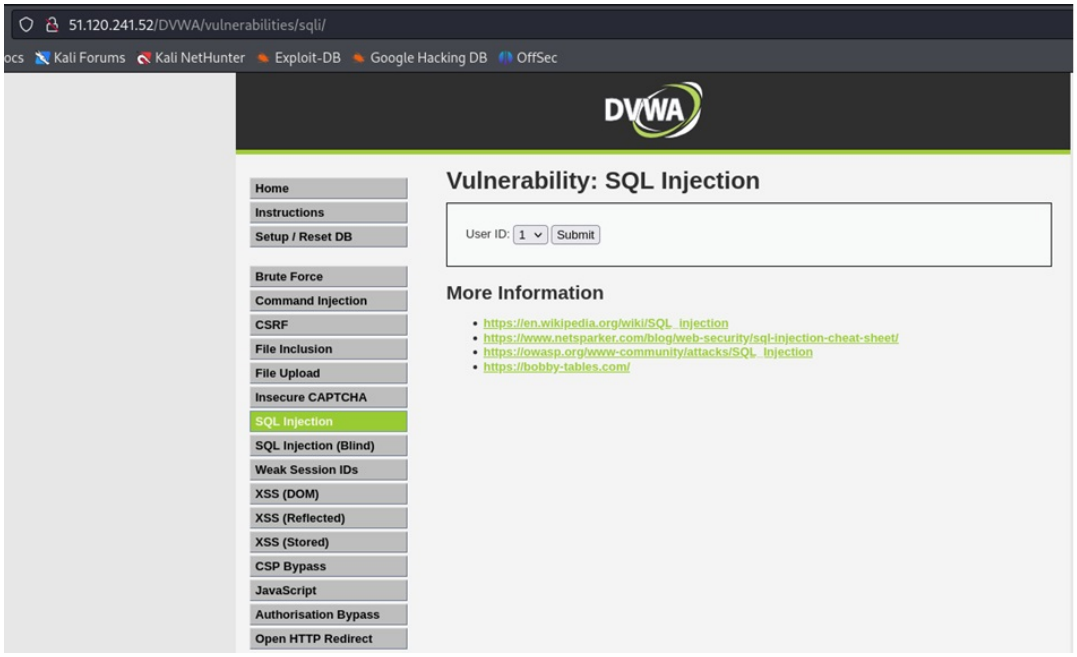

Figure B.43: Medium level does not have input section

Go to inspect and view the details,

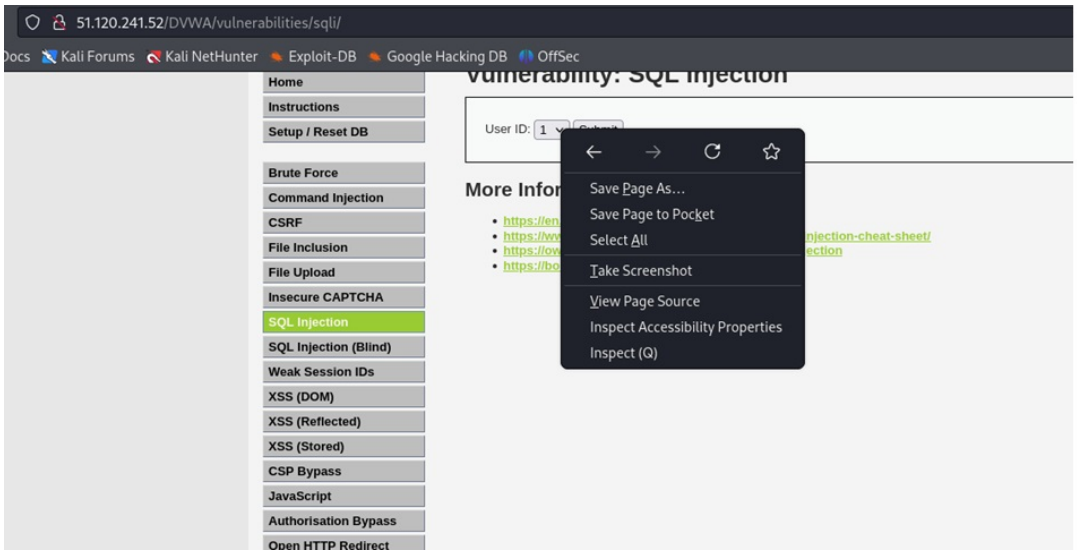

Figure B.44: Figure showing inspect and other options

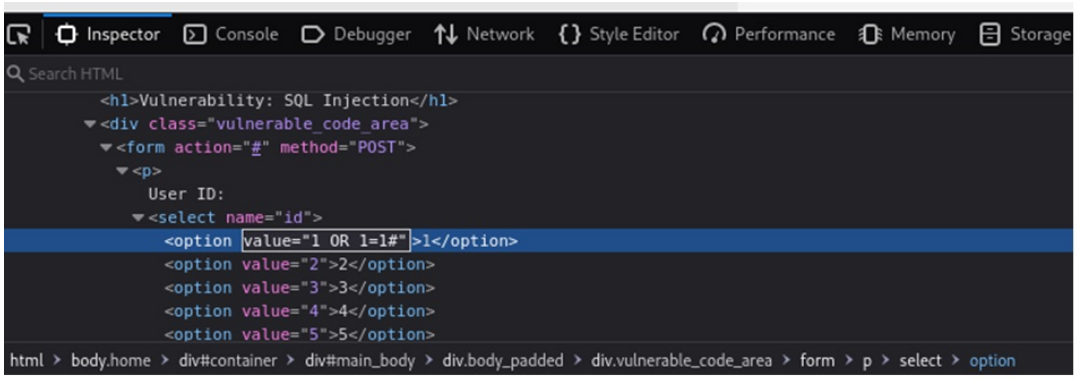

Figure B.45: Inserting payload in the ID values

Here is an output that shows the details of the users.

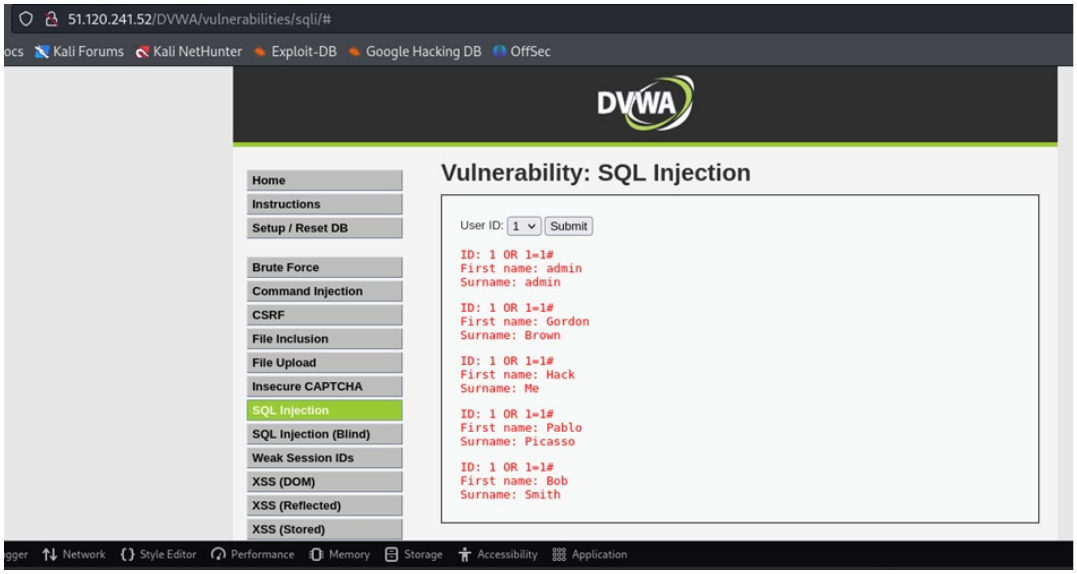

Figure B.46: Inserting payload in the ID values

SQL Injection(Blind)

1. Description: Blind SQL injection is the most intricate and sophisticated and happens when other methods fail to attack. To obtain the data from the victim, the attacker needs to use creativity and include boolean queries, like true or false (Crespo et al., 2023) .

2. Proof of Concept: In this attack at a low level, there is an input field as in figure B.47. But as in previous case, it does not allow input of the payload. For this sqlmap tool is used. Figure B.47 shows SQlmap command. Figure B.49 shows the databases retrieved and figure B.50 shows the fetched tables and columns in figure B.51 and figure B.52 shows the data of the users with passwords.

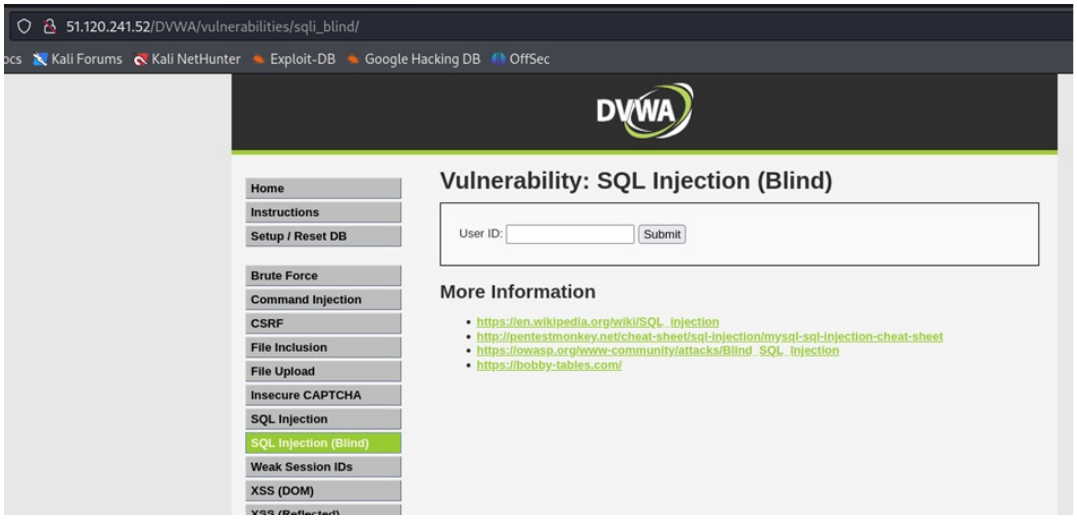

Figure B.47: Main page showing Blind SQL Injection

Figure B.48: Running sqlmap to retrieve information

Following databases retrieved.

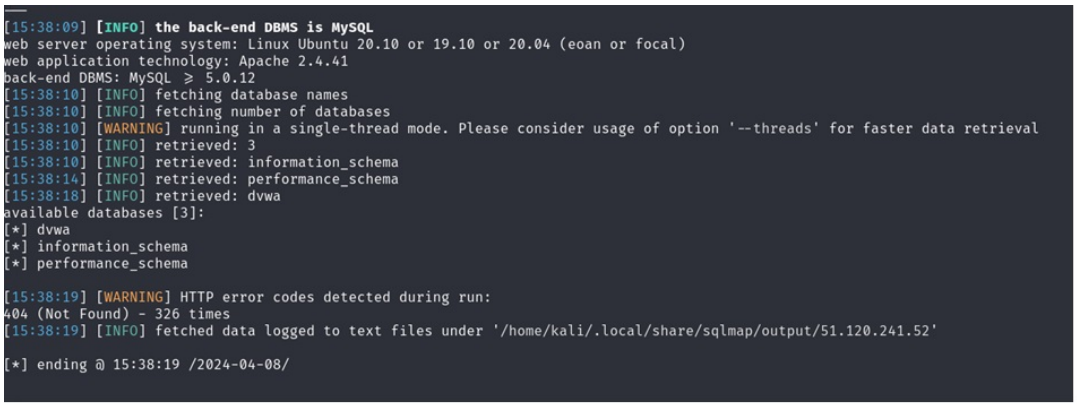

Figure B.49: retrieved databases are visible

Figure B.50: sqlmap command to fetch tables with output

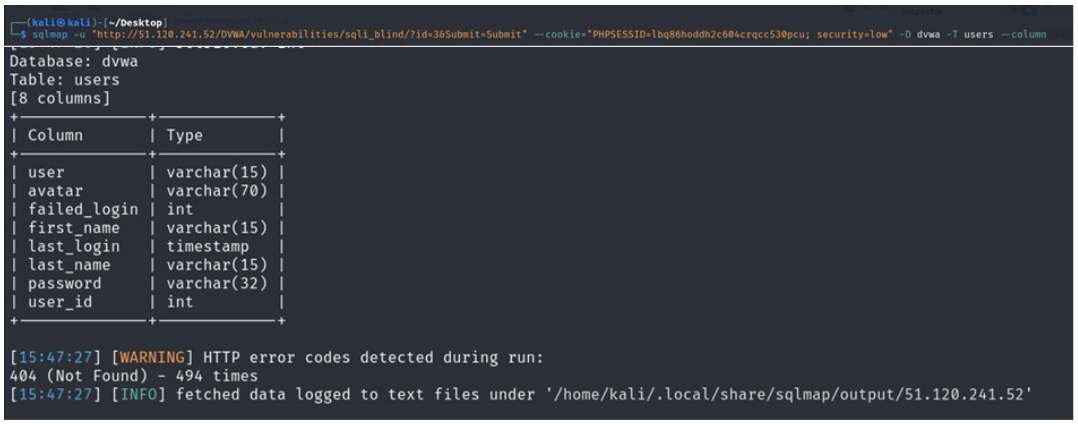

Figure B.51: retrieved Columns with data

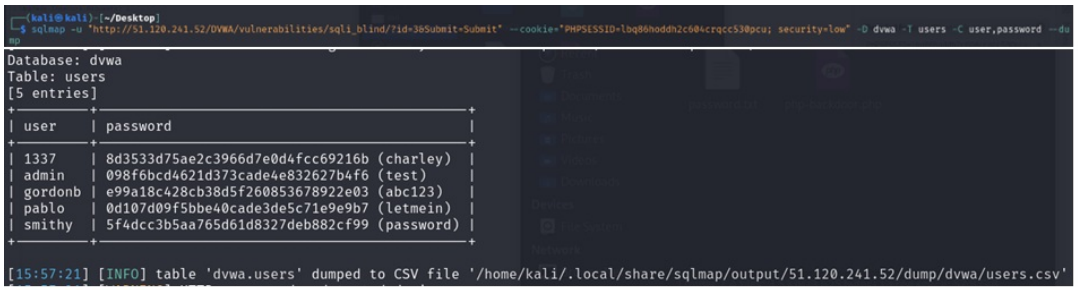

Figure B.52: retrieved databases are visible

Medium Level: For medium level lab, there is no input field in the blind sql test lab as in figure B.53. The cookie is accessed by intercepting the request in Burp Suite

and giving the.following command is used to retrieve the results that are shown in figure B.54. The retrieved databases are shown in figure B.53 and figure B.53 shows the columns that were retrieved.

Figure B.53: Figure showing main page of the lab

Figure B.54: retrieved databases are visible

```
Database: dvwa
Table: users
[8 columns]
+---------------+---------------+
| Column        | Type          |
+---------------+---------------+
| user          | varchar(15)   |
| avatar        | varchar(70)   |
| failed_login  | int           |
| first_name    | varchar(15)   |
| last_login    | timestamp     |
| last_name     | varchar(15)   |
| password      | varchar(32)   |
| user_id       | int           |
+---------------+---------------+
```

Figure B.55: retrieved columns are visible

Weak Session IDs

1. Description: In general, when a user accesses a website, the session ID remains traceable. An identifier for a session that isn't sufficiently unpredictable or random is called a Weak session ID. IT is easier for attackers to find or guess or modify session IDs as in turn it leads to session hijacking (Burgers et al., 2013).

2. Proof of Concept: In the low level, go to the weak session ID tab, and press the "Generate" button shown in figure B.56, every time it is pressed the cookie gets a new value.

Figure B.56: Figure shows main page lab of Weak session IDs

When the request is intercepted in the Burp Suite, this can be clearly seen in figure B.57, figure B.58, figure B.59 and figure B.60shows the iteration of the cookie session ID with increments of 1 respectively.

Figure B.57: Request intercepted in Burp Suite with cookie showing dvwaSession ID

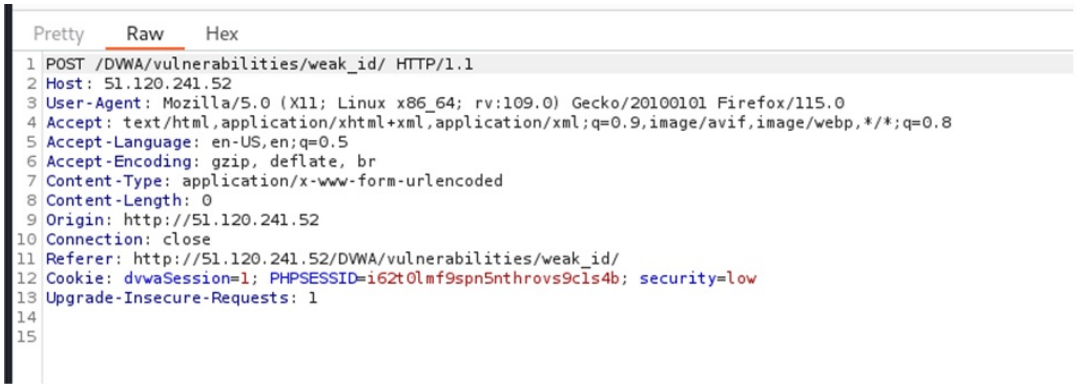

Figure B.58: Request intercepted in Burp Suite with session=1

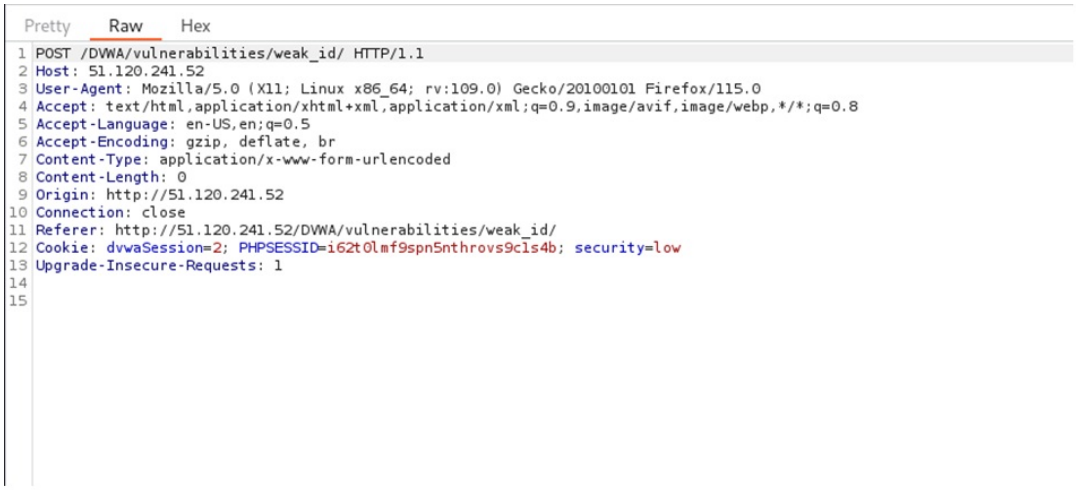

Figure B.59: Request intercepted in Burp Suite with cookie showing dvwaSession ID =2

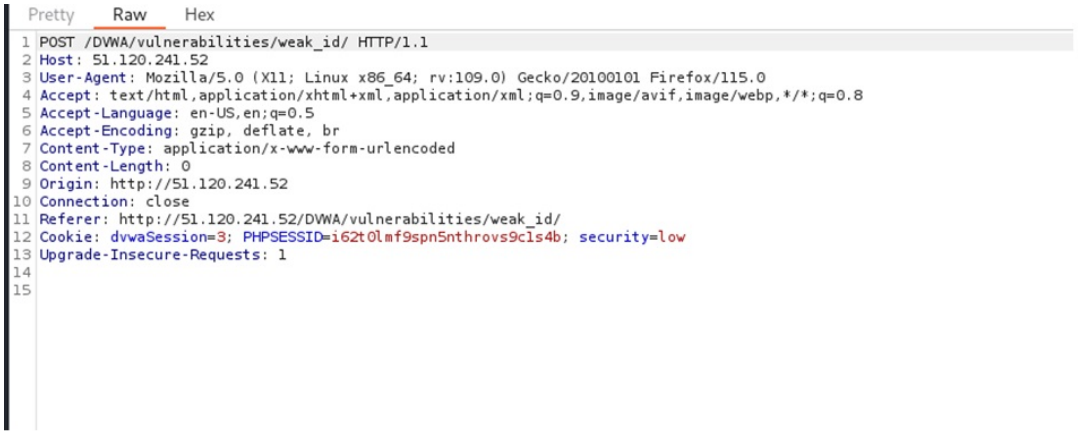

Figure B.60: Request intercepted in Burp Suite with cookie showing dvwaSession ID =3

When the page source is viewed it can be seen that that the server-side code increments the session ID by one with the statement "_SESSION['last_session_id']++;" in figure B.61

Figure B.61: Page source of Weak Session ID

Cross-Site Scripting (XSS(DOM))

1. Description: Cross-Site Scripting (XSS) occurs when malicious web code—typically in the form of scripts—is received or executed through the victim's web applications on their computer browser. Through this execution, user cookies may be taken or personal data may be filtered, allowing identity theft in a fake session.This also gives hackers the chance to potentially take over other machines or steal sensitive data (Wibowo & Sulaksono, 2021).

2. Proof of Concept:

   When select button is pressed, monitor the URL. The parameter "default=English" is there. Modifying the selection box modifies the "default" parameter as shown in figure B.62.

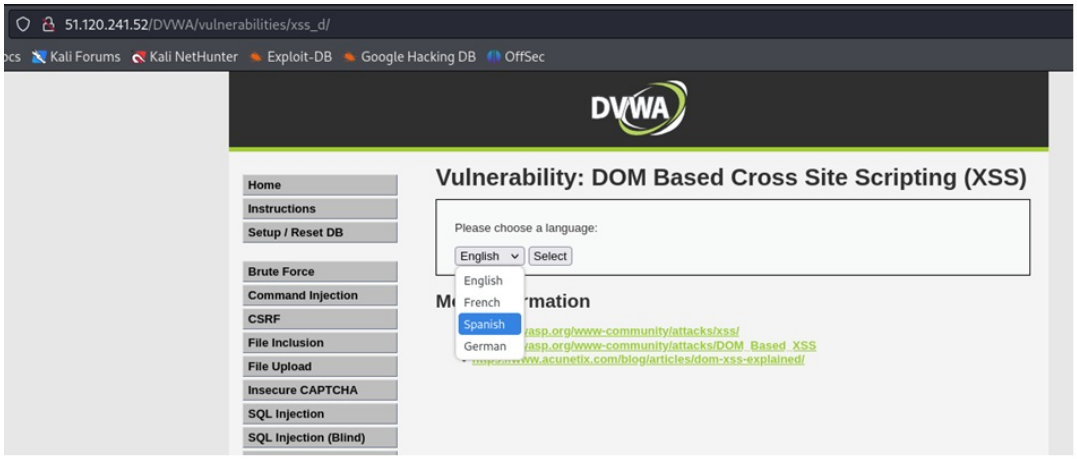

Figure B.62: Main page of XSS DOM based

When entering the payload “<script>alert(‘DOM XXS*)</script>” in the URL the output provides a pop-up box showing the XSS vulnerability which can be seen in figure B.63.

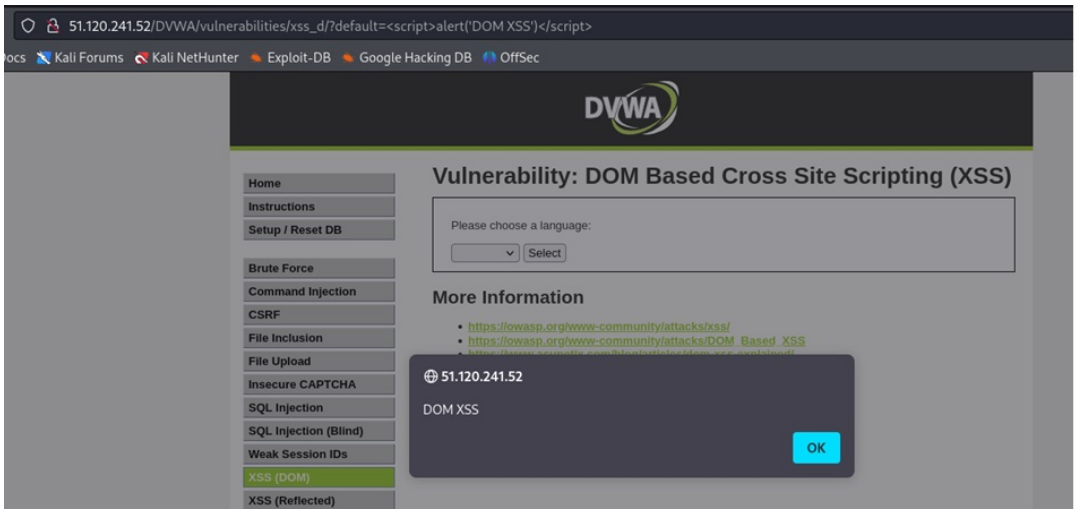

Figure B.63: Inserted payload providing output

Medium: For medium level, set the level to medium level and in the URL inject the payload shown in the figure B.64and the level is cleared.

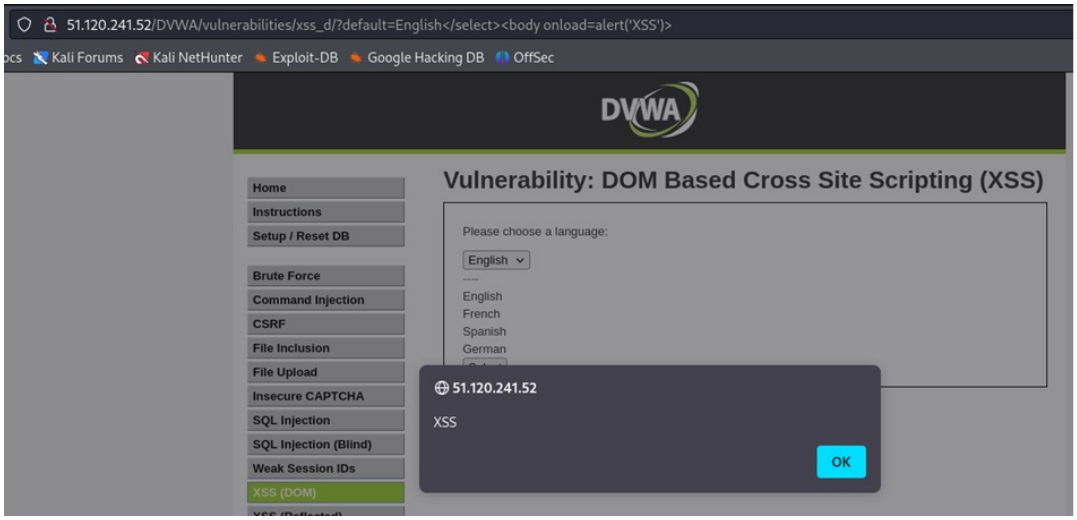

Figure B.64: Output with the payload

Cross-Site Scripting (XSS(Reflected))

1. Description: In a reflected cross-site scripting attack, a hacker inserts malicious code into a website in order to obtain a victim's session cookies. This kind of attack eliminates the requirement for a password by enabling the attacker to assume the victim's identity and carry out actions with their authorization. Forms, URLs, cookies, as well as multimedia material like flash and videos are examples of common injection locations. To get a response from a website ,the attack depends on data supplied by the user. A transport mechanism, like spoofing, is required to run the malicious code in order for the attack to be successful. In particular, if the application includes an XSS vulnerability, this can fool the user into clicking a link or button that triggers JavaScript and reroutes the victim's traffic to the attacker (Rodríguez et al., 2020).

2. Proof of Concept: In the Low level figure B.65, the payload is injected in the input box and the vulnerability is visible.

Figure B.65: Figure shows XSS reflected attack

For the medium level, the payload can be injected into the URL can be seen in figure B.66 and it reflects the output as a POP-UP box

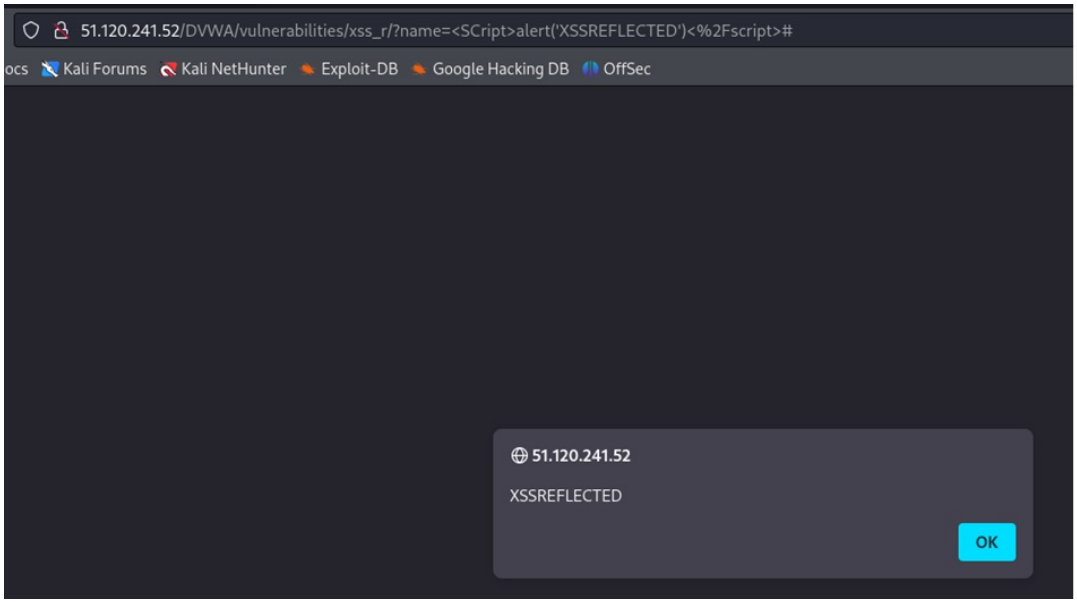

Figure B.66: Figure shows XSS reflected attack at medium level

Cross-Site Scripting (XSS(Stored))

1. Description: The Stored XSS occurs when an application obtains data from a source that is not reliable and trusted and also uses the data in a harmful way in subsequent HTTP responses.

2. Proof of Concept:

   For stored XSS at a low level, the payload can be injected in the input description box shown in figure B.67. The payload further displays a POP-up box showing “StoredXSSAttack “as a payload and message to the user.

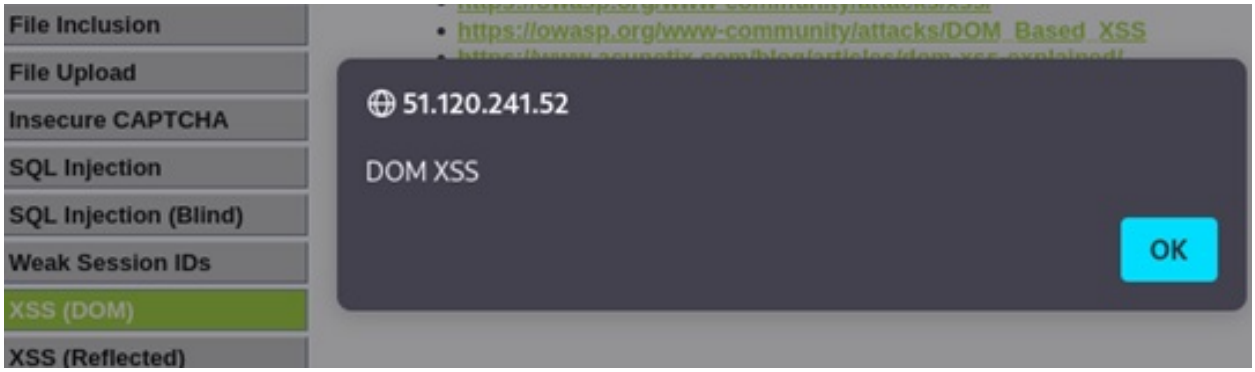

Figure B.67: Figure shows XSS stored attack

For medium level, shown in figure B.68increase the security to the medium level and insert the payload in the description box.

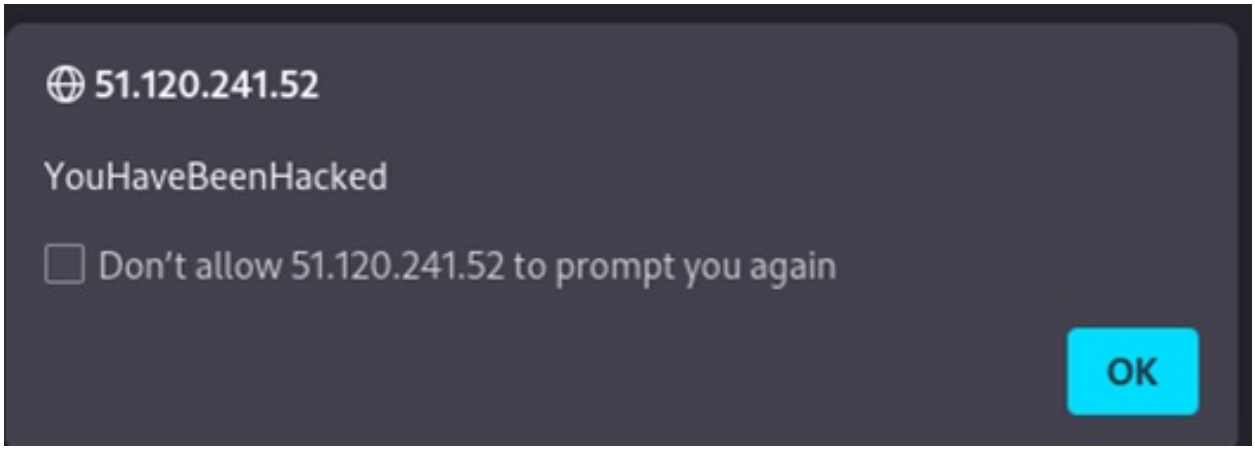

Figure B.68: Figure shows XSS stored attack at medium level

Content Security Policy (CSP) Bypass

1. Description: A content security policy is a set of guidelines that uses a white-listing technique to limit what material can be included on websites. Lists of sources are connected to content types via instructions from which resources of that particular kind may be included in a web page secured by CSP. For example, a web page can only load pictures via the host a.com over the HTTPS protocol if the directive img-src https://a.com is used. The content security policies (CSP) for an HTML page can be defined using meta elements or HTTP(S) headers. CSP-capable web browsers enforce these policies on a per-page basis, allowing different pages within the same website to have distinct security policies specified (Weichselbaum et al., 2016).

2. Proof of Concept: In this Content Security Bypass, it was needed to explore it with URL from Pastebin or Hastebin, but both were not responding as expected. But when tried with the scripts provided below the URL input area that can be seen in figure B.69, it worked and gave a popup box with the cookie message shown in figure B.70.

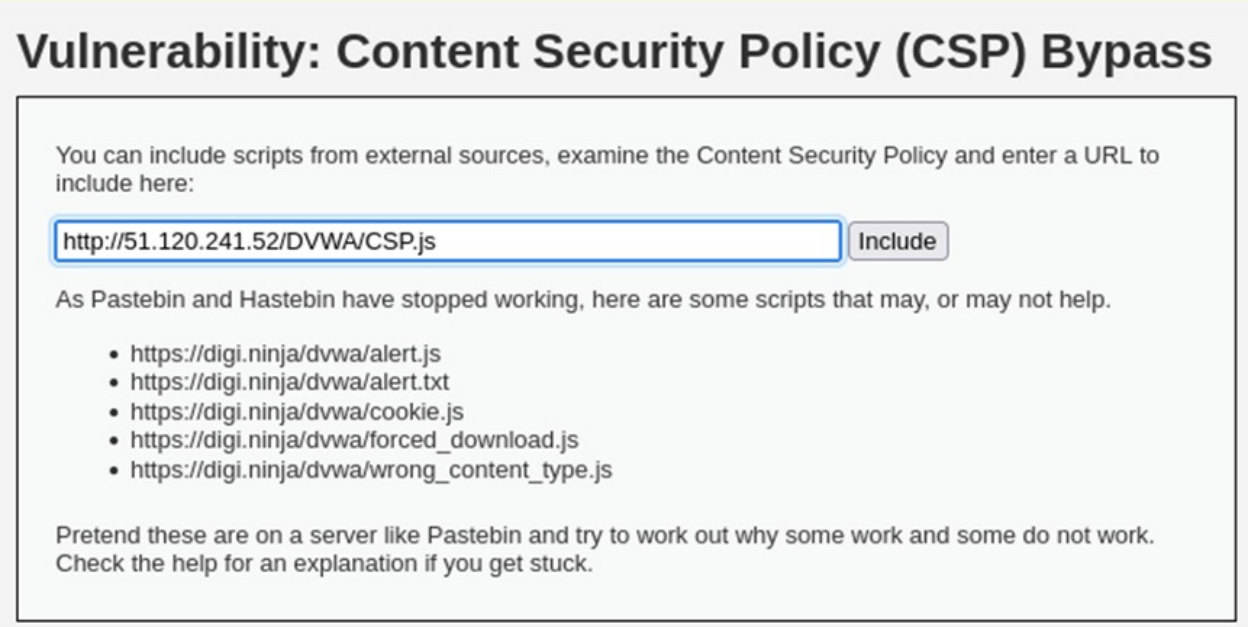

Figure B.69: Figure showing CSP lab main page with link as payload

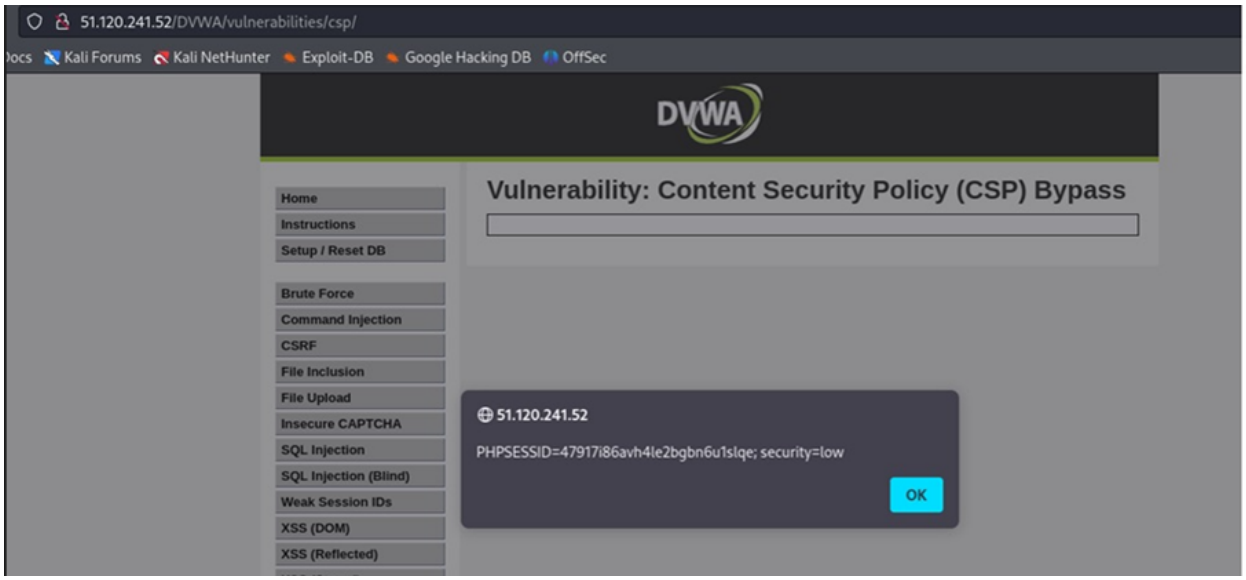

Figure B.70: Figure showing CSP payload with the POP-UP box

JavaScript

1. Description: The ability of an application to execute arbitrary JavaScript in the target's browser leads to a JavaScript vulnerability that is further a kind of XSS. This vulnerability can lead to various outcomes, including the exposure of session cookies, which in return can give the identity of the victim and may also enable the attacker to change the targeted application's behavior or page contents (OWASP Foundation, 2024).
2. Proof of Concept: In this vulnerability, the MD5 hash of the input was checked. Figure B.71 shows main lab of JavaScript, when attempting to submit the word "success" as requested and intercepting the request, it was observed that the hash value of the

token was incorrect.

Figure B.71: Figure showing JavaScript lab main page

In the figure B.72 source code it seems like it is In the page's source code, the JavaScript function 'generate_token()' is invoked. This function retrieves the value of the element "phrase" and calculates the MD5 hash of the phrase encoded using a Caesar cipher (ROT13).

```
function rot13(inp) {
    return inp.replace(/[a-zA-Z]/g,function(c){return String.fromCharCode(
}

function generate_token() {
    var phrase = document.getElementById("phrase").value;
    document.getElementById("token").value = md5(rot13(phrase));
}

generate_token();
/script>
```

Figure B.72: Source code of JavaScript vulnerability

After calculating the word "success" after ROT13 gives a new token. And the old token in the request was of the word "changeme". The result of success after ROT13 an cand be seen in figure B.73.

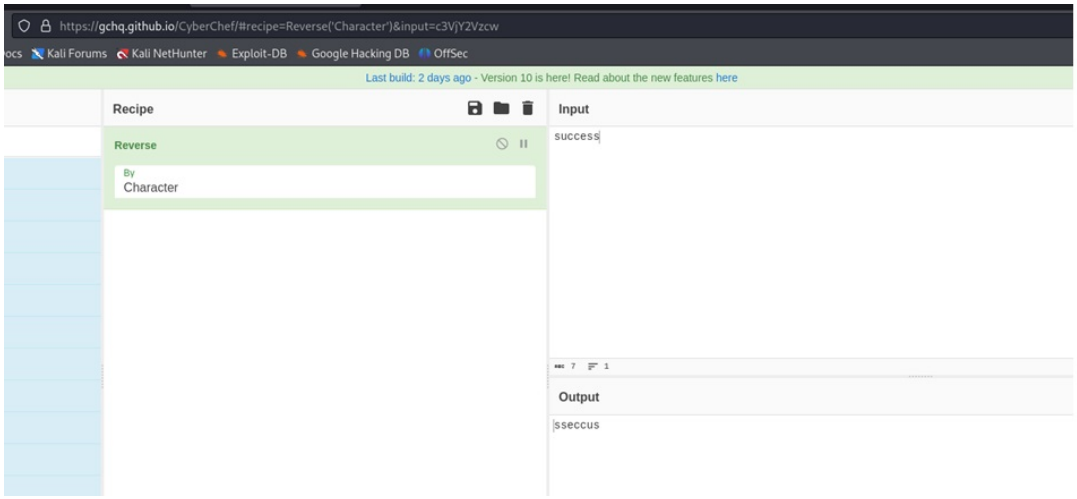

Figure B.73: Cyberchef with result

```
POST /DVWA/vulnerabilities/javascript/ HTTP/1.1
Host: 51.120.241.52
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 66
Origin: http://51.120.241.52
Connection: close
Referer: http://51.120.241.52/DVWA/vulnerabilities/javascript/
Cookie: PHPSESSID=47917i86avh4le2bgbn6ulslqe; security=low
Upgrade-Insecure-Requests: 1

token=38581812b435834ebf84ebcc2c6424d6&phrase=success&send=Submit
```

Figure B.74: Figure showing token value in BurpSuite request

When the token is fixed which was intercepted from Burp Suite in figure B.74, the correct value was gained that can be seen in figure B.75.

```
POST /DVWA/vulnerabilities/javascript/ HTTP/1.1
Host: 51.120.241.52
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 66
Origin: http://51.120.241.52
Connection: close
Referer: http://51.120.241.52/DVWA/vulnerabilities/javascript/
Cookie: PHPSESSID=47917i86avh4le2bgbn6ulslqe; security=low
Upgrade-Insecure-Requests: 1

token=38581812b435834ebf84ebcc2c6424d6&phrase=success&send=Submit
```

Figure B.75: Figure shows correct value entered

In the medium level, it is not possible to put the payload in input description box or in URL, so it was tried in the inspect area in the code. After changing it from there it worked as in figure B.76.

Figure B.76: Figure shows entering the value in inspect area

Authorisation Bypass

1. Description: In any situation in which someone uses a program to access functionality or content they shouldn't be able to is called as an authorization bypass. This vulnerability includes situations in which an individual can access or alter content that is not appropriate for their level of authorization. Such situations include vertical movement, also known as privilege escalation, in which an individual obtains access to content outside the scope of their role's authorization, and lateral movement, in which an individual obtains data from another account at the same level of privilege (Johnson, 2019).

2. Proof of Concept: Begin by accessing the Authorisation Bypass tab as in figure B.77. Subsequently, within a table format, display user details such as login ID and surname. Each user's information, encompassing their username, full name, and email address, is presented. Additionally, provide edit and delete functionalities adjacent to each user entry.

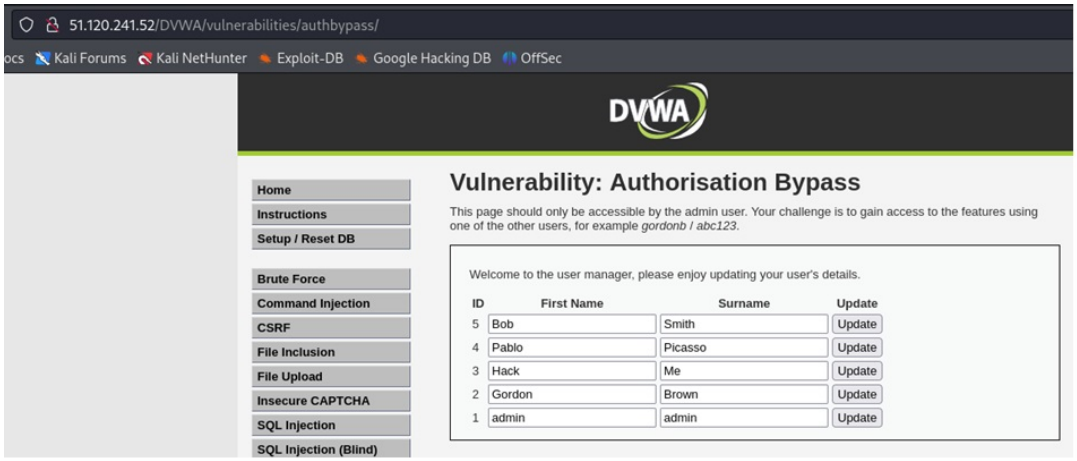

Figure B.77: Figure shows Authorisation Bypass lab with entries

Access the DVWA via a private browser with the credentials as gordnob/ abc123 that is shown in figure B.78.

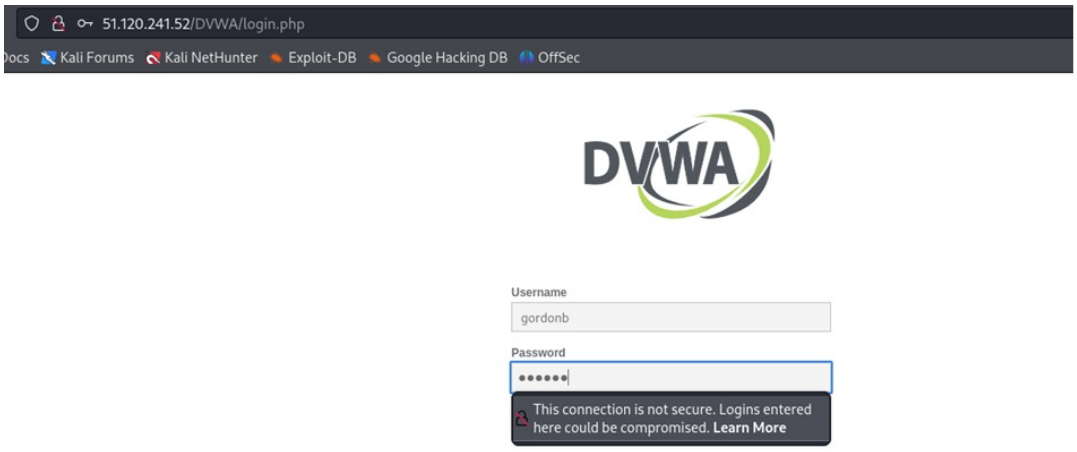

Figure B.78: Figure shows Authorisation Bypass lab with entries in a different login

In figure B.79 the authorization bypass was not visible when accessed from a non-administrative account.

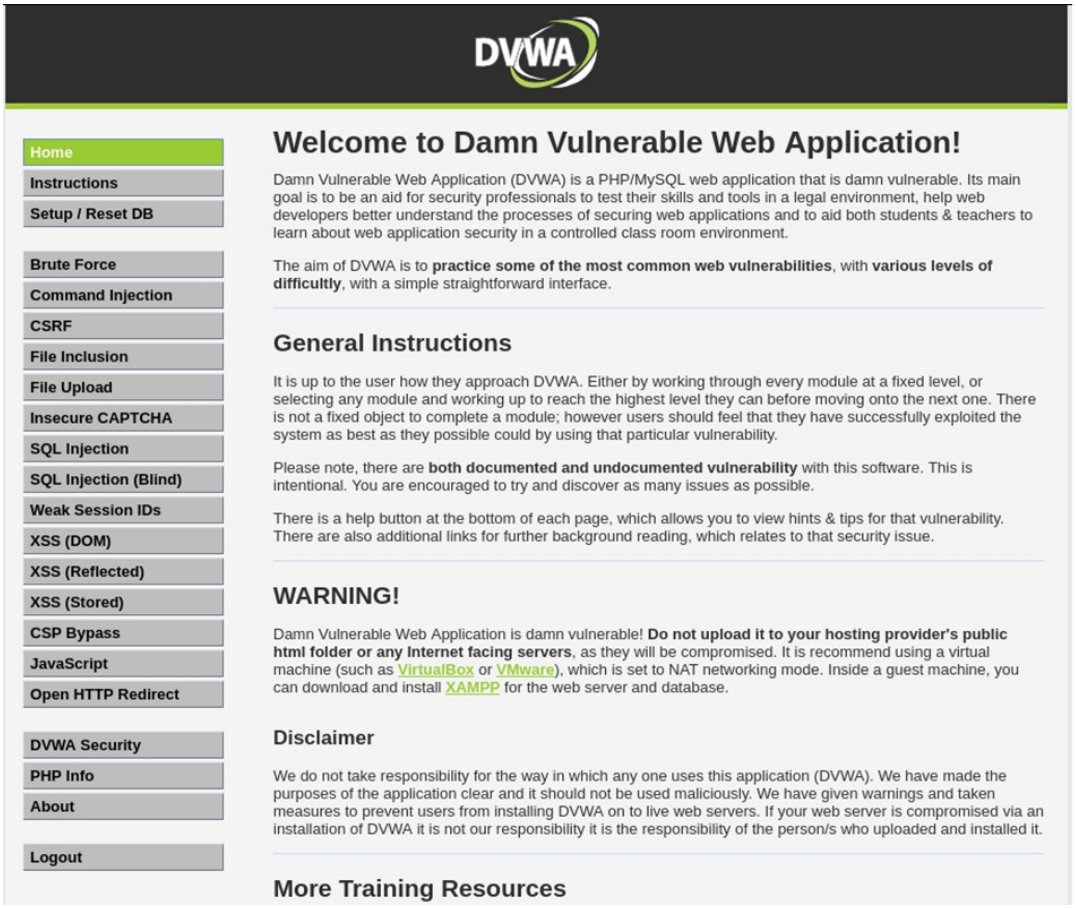

Figure B.79: Figure shows Authorisation Bypass lab with entries

When the “inspect” section of the web page is checked, select any vulnerability in the inspected area and add the “authorization bypass” and the path as shown in the figure B.80, and hit “Enter”. The Admin page is accessible now and can be seen in figure B.81 and figure B.82 shows the result.

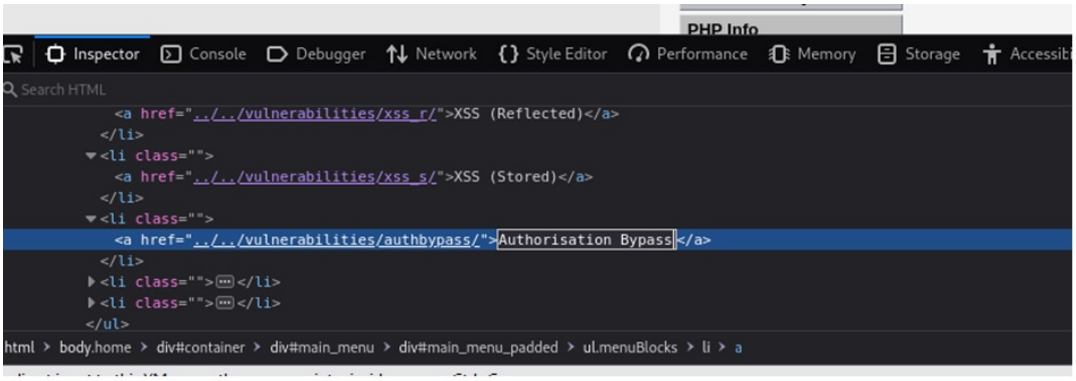

Figure B.80: Inserting payload in the inspect area

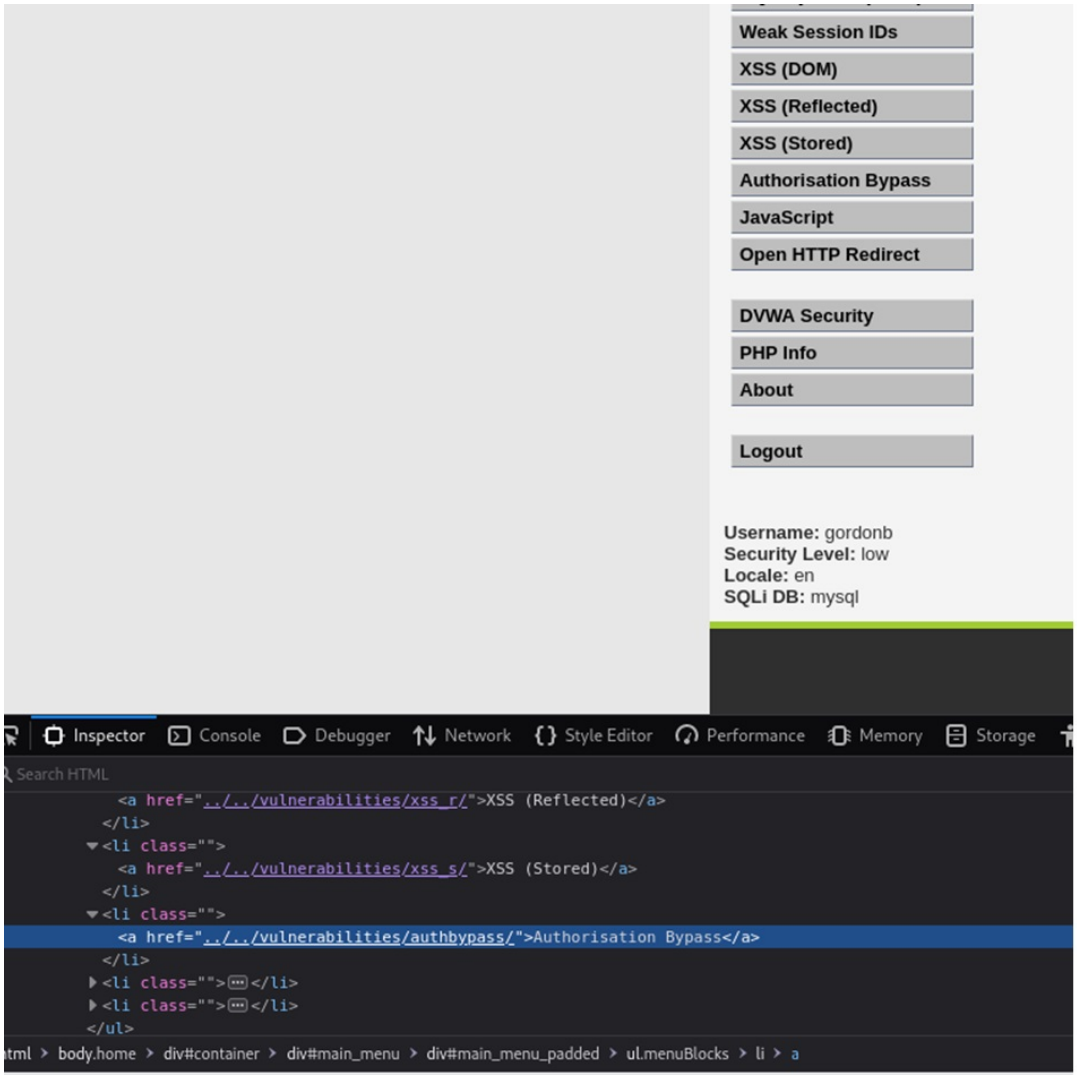

Figure B.81: Figure shows Authorisation Bypass is visible

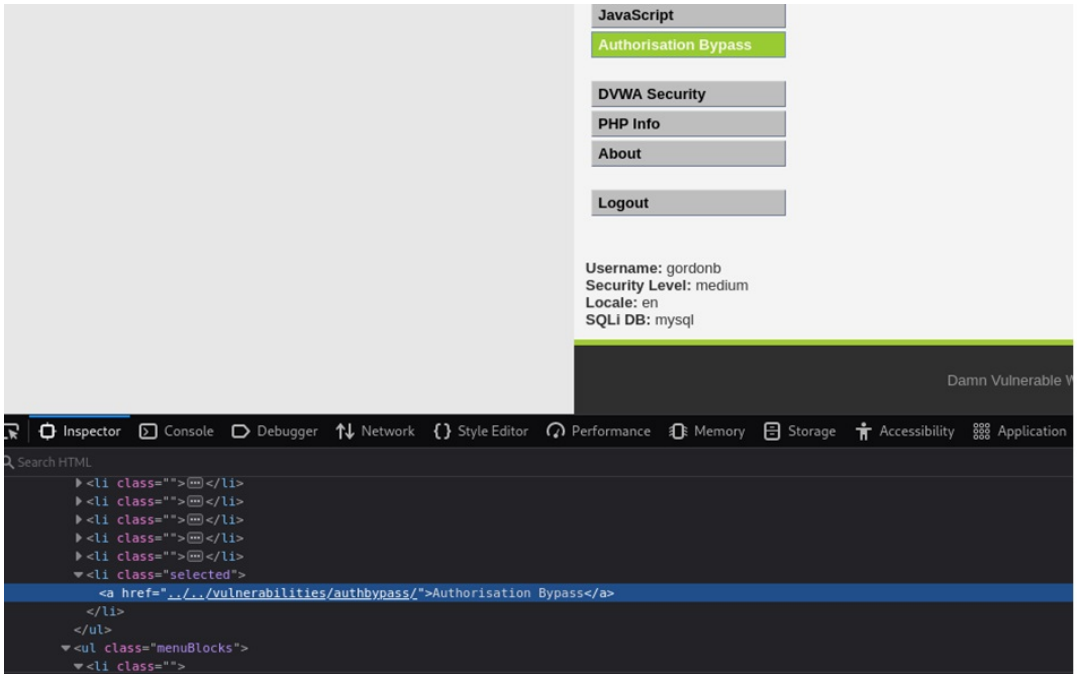

Figure B.82: Figure shows Authorisation Bypass in the entry

Open HTTP Redirect

1. Description: An attacker uses this bypass technique to send a URL that points to a page that is hosted on a server they own. The page would be accessible since this server's public IP address would pass any verification checks. As a result, the vulnerable server is sent to this page, where it requests an internal service (Jabiyev et al., 2021).

2. Proof of Concept: It appears in figure B.83 that the URL here is a link undergoing redirection. Intercept the request in BurpSuite as shown in figure B.84 to inspect it further.

Figure B.83: Figure shows lab of HTTP Reditect

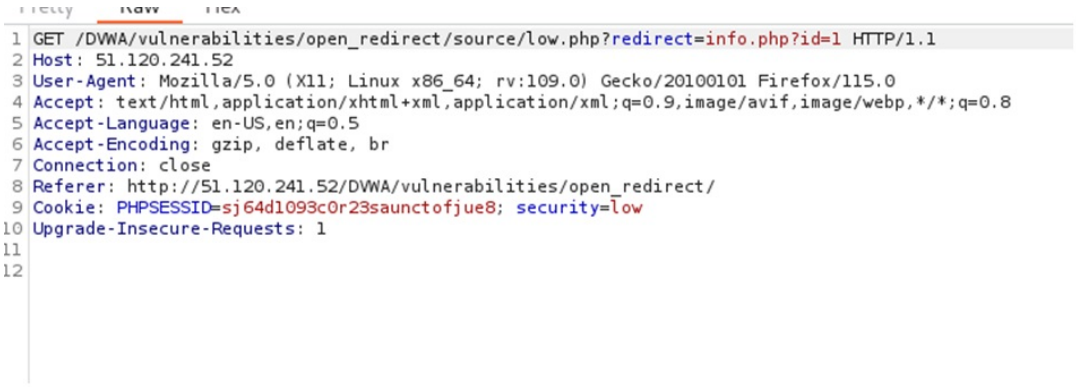

Figure B.84: Figure shows request intercepted in Burp Suite

The redirect can be modified to a new URL. When the URL is followed in the browser, the given URL can be accessed. This shows that the URLs can be modified and can be made malicious links to the users. Fig B.93 shows the redirection can be modified to a new URL. When the link of BWAPP used as example to change the URL, was pasted in the burp request in figure B.85, it redirected the page to a new URL that

was given in the request which is in figure B.86.

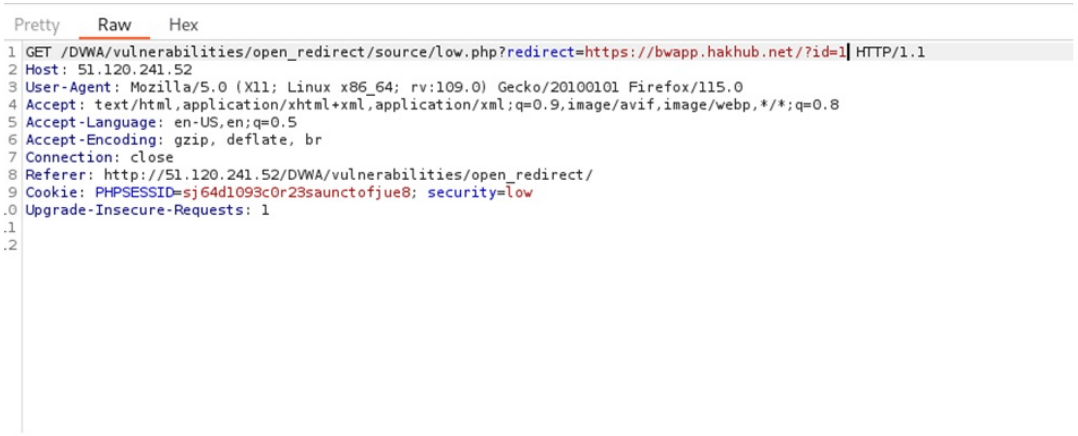

Figure B.85: Figure shows modified URL

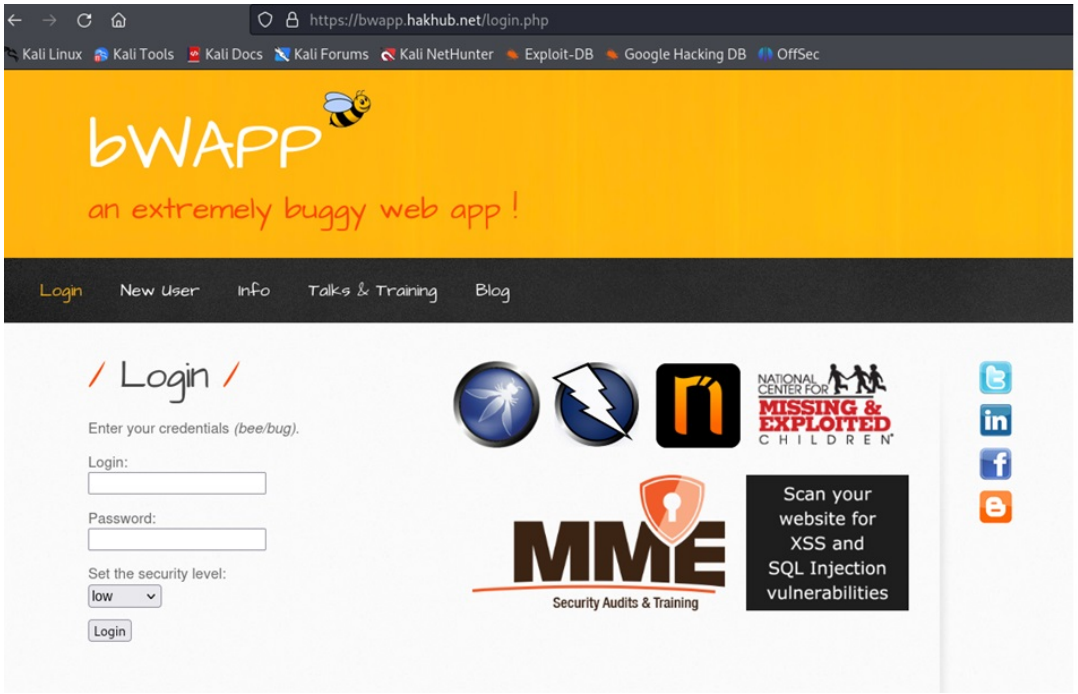

Figure B.86: Figure shows modified URL

Medium Level: Start by setting the level to medium and intercept the request in BurpSuite. Send the request to repeater. The redirect does not accept absolute URLs in this case, so tried without http:// and enter “evil.com” as in figure B.87.

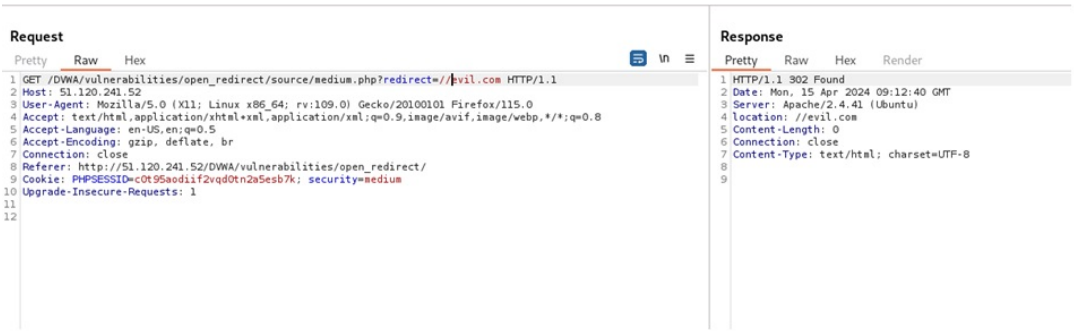

Figure B.87: Figure shows modified URL in medium level

Word count metrics

NUC Bachelor Project Word Count:
Total Sum count: 29898 Words in text: 28754 Words in headers: 371 Words outside text (captions, etc.): 773 Number of headers: 128 Number of floats/tables/figures: 150 Number of math inlines: 0 Number of math displayed: 0
NOTE: References are excluded.